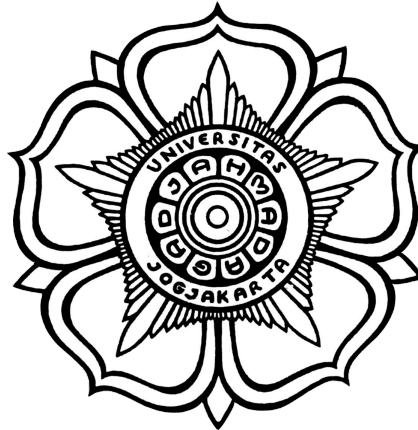


SKRIPSI

**PENGEMBANGAN SISTEM FUZZING UNTUK PENGUJIAN KUALITAS
REST API MENGGUNAKAN MULTI-AGENT REINFORCEMENT
LEARNING DAN SEMANTIC OPERATION DEPENDENCY GRAPH**

***FUZZING SYSTEM DEVELOPMENT FOR REST API QUALITY TESTING
USING MULTI-AGENT REINFORCEMENT LEARNING AND SEMANTIC
OPERATION DEPENDENCY GRAPH***



David Lois

22/498373/PA/21509

**PROGRAM STUDI S1 ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

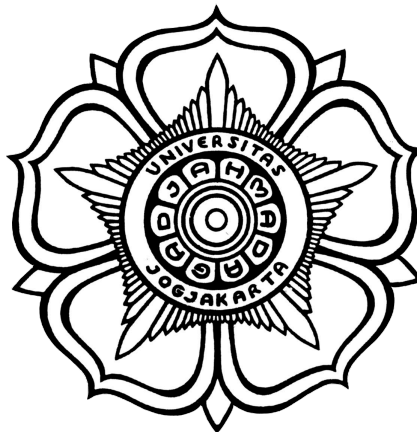
2025

PROPOSAL TUGAS AKHIR

**PENGEMBANGAN SISTEM FUZZING UNTUK PENGUJIAN KUALITAS
REST API MENGGUNAKAN MULTI-AGENT REINFORCEMENT
LEARNING DAN SEMANTIC OPERATION DEPENDENCY GRAPH**

***FUZZING SYSTEM DEVELOPMENT FOR REST API QUALITY TESTING
USING MULTI-AGENT REINFORCEMENT LEARNING AND SEMANTIC
OPERATION DEPENDENCY GRAPH***

Diajukan untuk memenuhi salah satu syarat memperoleh derajat Sarjana Ilmu
Komputer/Sarjana Sains Elektronika dan Instrumentasi



David Lois

22/498373/PA/21509

**PROGRAM STUDI S1 ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2025

HALAMAN PENGESAHAN

SKRIPSI

PENGEMBANGAN SISTEM FUZZING UNTUK PENGUJIAN KUALITAS REST API MENGGUNAKAN MULTI-AGENT REINFORCEMENT LEARNING DAN SEMANTIC OPERATION DEPENDENCY GRAPH

Telah dipersiapkan dan disusun oleh

DAVID LOIS

22/498373/PA/21509

Telah dipertahankan di depan Tim Penguji
pada tanggal ...

Susunan Tim Penguji

Nama Pembimbing I
Pembimbing I/Penguji

Nama Penguji I
Penguji

Nama Pembimbing II
Pembimbing II/Penguji

Nama Penguji II
Penguji

Penguji

Nama Penguji III (*jika ada*)

PERNYATAAN

Dengan ini saya menyatakan bahwa skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka.

Jakarta, 19 Desember 2025

David Lois

PRAKATA

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan skripsi yang berjudul “Pengembangan Sistem Fuzzing untuk Pengujian Kualitas REST API Menggunakan Multi-Agent Reinforcement Learning dan Semantic Operation Dependency Graph”. Skripsi ini disusun untuk memenuhi salah satu syarat memperoleh derajat Sarjana Ilmu Komputer pada Program Studi S1 Ilmu Komputer, Departemen Ilmu Komputer dan Elektronika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Gadjah Mada.

Penulis menyadari bahwa keberhasilan dalam penyelesaian skripsi ini tidak terlepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih yang tulus kepada:

1. Bapak Arif Nurwidyantoro, S.Kom., M.Cs., Ph.D., selaku Dosen Pembimbing yang telah memberikan arahan, saran, serta bimbingan yang sangat berharga selama proses penelitian dan penyusunan skripsi ini.
2. Kedua orang tua dan keluarga besar penulis yang senantiasa memberikan dukungan moral, doa, serta motivasi yang tidak terhingga dalam setiap tahapan studi penulis.
3. Rekan-rekan mahasiswa Program Studi Ilmu Komputer angkatan 2022 dan semua pihak yang telah membantu secara langsung maupun tidak langsung dalam proses penyelesaian tugas akhir ini.

Penulis berharap skripsi ini dapat memberikan manfaat bagi pengembangan ilmu pengetahuan, khususnya dalam bidang pengujian perangkat lunak otomatis.

Jakarta, 19 Desember 2025

David Lois

DAFTAR ISI

HALAMAN PENGESAHAN	2
PERNYATAAN	3
PRAKATA	4
DAFTAR ISI	5
DAFTAR TABEL	9
DAFTAR GAMBAR	10
DAFTAR SIMBOL	11
INTISARI	12
BAB I	1
1.1. Latar Belakang	1
1.2. Perumusan Produk	4
1.3. Batasan Produk	5
1.4. Tujuan Penelitian	5
1.5. Manfaat Penelitian	6
BAB II	7
2.1. Identifikasi Masalah	7
2.1.1. Hambatan Aksesibilitas dan Kompleksitas Operasional Alat Fuzzing Akademik	7
2.1.2. Kesenjangan Integrasi dalam Alur Kerja Industri (CI/CD)	8
2.1.3. Beban Kognitif dalam Interpretasi dan Manajemen Hasil Pengujian	8
2.2. Pengembangan Ide Inovatif	9
2.2.1. Demokratisasi Pengujian API Cerdas melalui Antarmuka Web	9
2.2.2. Integrasi Rekomendasi Proaktif dalam Alur Kerja CI/CD	9
2.2.3. Arsitektur Terintegrasi untuk Operasional Skala Industri	10
2.2.4. Aksesibilitas dan Visualisasi Hasil sebagai Fokus Desain	10
BAB III	11
3.1. REST API	11
3.1.1. Arsitektur Representational State Transfer (REST)	11
3.1.2. OpenAPI Specification (OAS)	11
3.2. Fuzzing REST API	12
3.2.1. Konsep Fuzz Testing	12
3.2.2. Tantangan Pengujian API Stateful	12
3.3. Semantic Operation Dependency Graph (SODG)	12
3.3.1. Pemodelan Dependensi Operasi	12
3.3.2. Ekstraksi Semantik dari Dokumentasi API	13

3.4. Multi-Agent Reinforcement Learning (MARL)	13
3.4.1. Reinforcement Learning (RL) dalam Pengujian	13
3.4.2. Agen Terspesialisasi dalam AutoRestTest	13
3.4.3. Parameter Optimasi RL	14
3.5. Large Language Models (LLM) dalam Otomasi Pengujian	14
3.5.1. Pemahaman Konteks Semantik	15
3.5.2. Integrasi LLM Engine	15
3.6. Jenis Pengujian Spesifikasi REST API	15
3.6.1. Pengujian REST API Berbasis Spesifikasi	15
3.6.2. Pengujian REST API Stateful dan Berbasis Model	16
3.6.3. Pengujian REST API Berbasis Reinforcement Learning (RL)	17
3.7. AutoRestTest	18
3.7.1. Integrasi SODG, MARL, dan LLM	18
3.7.2. Perbandingan dengan Alat Pengujian Tradisional	19
3.8. Sintesis dan Posisi Penelitian	19
3.9. Arsitektur Aplikasi Web dan Komunikasi Asinkron	20
3.9.1. Framework Next.js dan Server-Side Rendering (SSR)	20
3.9.2. FastAPI dan Python Backend	20
3.9.3. Orkestrasi Task (Trigger.dev)	20
3.10. Persistensi Data dan Optimasi Performa	21
3.10.1. PostgreSQL dan Basis Data Relasional	21
3.10.2. Prisma ORM	21
3.10.3. Sistem Caching dan In-Memory Database (Upstash Redis)	21
3.11. Continuous Integration dan Continuous Deployment (CI/CD)	22
3.11.1. Siklus Umpan Balik Kontinu	22
3.11.2. Alur Kerja GitHub Actions	22
3.12. Evaluasi Kualitas Perangkat Lunak	22
3.12.1. Task-Based Usability Testing	23
3.12.2. Metrik Google Lighthouse	23
BAB IV	24
4.1. Deskripsi Umum	24
4.2. Alur Pengembangan Produk	25
4.3. Analisis Solusi Terkait dan Posisi Strategis	26
4.4. Requirements Engineering	28
4.4.1. Functional Requirements	28
4.4.2. Non-Functional Requirements	29
4.5. Perancangan Desain Antarmuka Pengguna (UI/UX)	30

4.5.1. Pembentukan Persona Arketipe	30
4.5.2. Pembentukan User Flow	30
4.5.3. Arsitektur Informasi dan Tata Letak Antarmuka	34
4.6. Perancangan Arsitektur Sistem	35
4.6.1. Arsitektur Informasi dan Tata Letak Antarmuka	35
4.6.2. Alur Data dan Orkestrasi Asinkron	37
4.7. Perancangan Basis Data	40
4.7.1. Entity Relationship Diagram (ERD)	40
4.7.2. Definisi Skema Data	41
4.8. Strategi Evaluasi dan Validasi	42
4.8.1. Pengujian Fungsional	42
4.8.2. Pengujian Non-Fungsional	42
4.8.3. Pengujian Usabilitas (Task-Based Usability Testing)	43
4.8.4. Kuesioner Persepsi Subjektif	44
4.8.5. Matriks Penilaian dan Mekanisme Penskalaan	44
4.8.6. Audit Kualitas Web (Google Lighthouse)	44
BAB V	46
5.1. Lingkungan Implementasi	46
5.1.1. Spesifikasi Perangkat Keras	46
5.1.2. Spesifikasi Perangkat Lunak	46
5.2. Implementasi Basis Data	47
5.2.1. Skema Data Prisma	48
5.2.2. Detail Struktur Tabel dan Relasi	49
5.2.3. Transformasi ke PostgreSQL	49
5.3. Implementasi Backend dan Orkestrasi Tugas	50
5.3.1. Implementasi Task Runner (Triggr.dev)	50
5.3.2. Implementasi API Wrapper (FastAPI)	52
5.4. Implementasi Antarmuka Pengguna (Frontend)	53
5.4.1. Formulir Konfigurasi Pengujian (TestForm.tsx)	54
5.4.2. Dasbor Riwayat dan Pemantauan (JobHistory.tsx)	57
5.4.3. Visualisasi Hasil Pengujian (JobDetailsDataExplorer.tsx)	58
5.5. Implementasi Integrasi CI/CD	61
5.5.1. Otomasi Setup Workflow (Octokit)	61
5.5.2. Mekanisme Triggering CI/CD	63
5.6. Implementasi Keamanan dan Autentikasi	63
5.6.1. Manajemen Autentikasi Pengguna (Clerk)	63
5.6.2. Perlindungan Rute dengan Middleware	64

5.6.3. Keamanan Komunikasi Antar-Layanan (API Key)	65
5.7. Akses Layanan dan Repositori	66
BAB VI	67
6.1. Hasil Pengujian Fungsional	67
6.1.1. Hasil Unit Testing	67
6.1.2. Hasil Validasi Fitur Utama	69
6.1.3. Hasil Validasi Integrasi CI/CD	70
6.2. Hasil Pengujian Non-Fungsional	71
6.3.1. Analisis Performa API (Postman)	72
6.3.2. Audit Kualitas Web (Google Lighthouse)	73
6.3. Analisis Task-Based Usability Testing	74
6.3.1. Profil Demografis Partisipan	74
6.3.2. Hasil Observasi dan Tingkat Penyelesaian Tugas	75
6.3.3. Proses Normalisasi Skor Perilaku	76
6.3.4. Analisis Skor Komponen Berbobot	77
6.3.5. Kalkulasi Skor Akhir Usabilitas	77
6.3.6. Analisis Masukan Peserta dan Perbaikan Fitur	78
6.4. Analisis Kuesioner Pasca-Sesi	80
6.4.1. Interpretasi Data Persepsi Subjektif	81
6.4.2. Sintesis Masukan dan Sentimen Kualitatif	82
6.5. Pembahasan Hasil dan Dampak Penelitian	83
6.5.1. Penjembatanaan Teori dan Praktik Melalui Antarmuka Web	83
6.5.2. Peran Integrasi CI/CD dalam Siklus Penjaminan Kualitas	83
6.5.3. Kualitas Teknis dan Keberterimaan Pengguna	84
6.6. Kesimpulan dan Saran	85
6.6.1. Kesimpulan	85
6.6.2. Saran	86
DAFTAR PUSTAKA	87
LAMPIRAN	89

DAFTAR TABEL

Tabel 6.1 Ringkasan metrik performa API (Waktu Respons, Throughput, dan Error Rate).	72
Tabel 6.2 Profil demografis partisipan usability testing	75
Tabel 6.3 Rekapitulasi skor mentah hasil observasi task-based usability testing (Skala 0-2)	75
Tabel 6.4 Hasil normalisasi skor perilaku partisipan ke dalam standar penilaian (Skala 1-5)	76
Tabel 6.5 Rekapitulasi skor kuesioner kepuasan pengguna berdasarkan parameter kepuasan dan kegunaan (Skala 1-5).	81

DAFTAR GAMBAR

Gambar 4.1 Diagram alur pengembangan produk	25
Gambar 4.2 Diagram user flow (flowchart)	31
Gambar 4.3 Diagram arsitektur end-to-end sistem	36
Gambar 4.4 Diagram fuzzing task orchestration sequence	37
Gambar 4.5 Logika integrasi CI/CD	39
Gambar 4.6 Diagram hubungan entitas (ERD)	40
Gambar 5.1 Antarmuka Halaman Dashboard yang Menampilkan Formulir Pembuatan Pekerjaan Baru (Create a New Test) dan Tabel Riwayat Pekerjaan (Job History).	56
Gambar 5.2 Antarmuka Halaman Dashboard yang Menampilkan Formulir Pembuatan CI Setup dan Tabel Riwayat Pekerjaan (Job History).	56
Gambar 5.3 Antarmuka yang memungkinkan pengguna mengatur parameter cerdas seperti temperature dan learning rate.	57
Gambar 5.4 Tabel yang berisi daftar pengujian yang pernah dilakukan, lengkap dengan label status (seperti Queued, Processing, atau Completed), dan waktu pembuatan.	58
Gambar 5.5 Tampilan landing halaman job details (JobDetailsDataExplorer.tsx)	59
Gambar 5.6 Tampilan data explorer job details (JobDetailsDataExplorer.tsx)	60
Gambar 5.7 Tampilan server errors job details (JobDetailsDataExplorer.tsx)	60
Gambar 5.8 Tampilan kombinasi body dan parameter job details (JobDetailsDataExplorer.tsx)	61
Gambar 6.1 Code coverage frontend dan backend for frontend	68
Gambar 6.2 Code coverage model wrapper API	68
Gambar 6.3 Tampilan form one-time test yang telah diisi oleh pengguna	69
Gambar 6.4 Visualisasi laporan hasil pengujian API (features service) yang menunjukkan statistik permintaan, durasi, dan broken endpoints	70
Gambar 6.5 Tampilan log keberhasilan eksekusi workflow AutoRestTest pada antarmuka Github Actions	71
Gambar 6.6 Hasil pengujian performa API menggunakan Postman yang menunjukkan grafik waktu respons dan tingkat kesalahan	72
Gambar 6.7 Skor hasil audit kualitas web (performance, accessibility, best practices, seo) menggunakan Google Lighthouse	74

DAFTAR SIMBOL

Kode 5.1 Definisi skema database dengan menggunakan Prisma ORM	48
Kode 5.2 Implementasi Task Runner menggunakan SDK Trigger.dev v3	51
Kode 5.3 Implementasi API Wrapper menggunakan FastAPI untuk eksekusi model	53
Kode 5.4 Implementasi komponen formulir konfigurasi pengujian menggunakan React Hook Form.	55
Kode 5.5 Implementasi komponen grafik distribusi kode status respons menggunakan pustaka Recharts.	59
Kode 5.6 Implementasi otomasi penyisipan berkas workflow GitHub Actions menggunakan Octokit.	62
Kode 5.7 Implementasi Clerk Provider pada rute akar (root layout) aplikasi.	64
Kode 5.8 Implementasi Middleware Next.js untuk proteksi rute dasbor dan API.	65
Kode 5.9 Implementasi fungsi validasi API Key untuk keamanan komunikasi antar-layanan.	66

INTISARI

Pengembangan Sistem Fuzzing untuk Pengujian Kualitas REST API Menggunakan Multi-Agent Reinforcement Learning dan Semantic Operation Dependency Graph

Oleh

David Lois

22/498373/PA/21509

Pengujian REST API menghadapi tantangan signifikan akibat kompleksitas dependensi antar-operasi dan ruang input yang luas. Alat pengujian *state-of-the-art* seperti AutoRestTest menawarkan solusi efektif melalui *Semantic Operation Dependency Graph* (SODG) dan *Multi-Agent Reinforcement Learning* (MARL), namun penggunaannya terbatas karena antarmuka baris perintah yang kompleks dan minimnya integrasi alur kerja. Penelitian ini mengembangkan sistem *fuzzing* berbasis web yang mengintegrasikan AutoRestTest dengan arsitektur asinkron dan otomatisasi CI/CD. Pengembangan dilakukan secara *end-to-end* mencakup antarmuka manajemen, orkestrasi *task runner*, dan *wrapper* model cerdas. Hasil pengujian performa menunjukkan sistem mampu menangani beban tinggi dengan rata-rata waktu respons 149 ms dan tingkat kesalahan internal 0,01%. Validasi fungsional pada layanan target berhasil mendeteksi 77,8% *broken endpoints* dengan total 16.739 permintaan dalam 60 detik. Evaluasi usability menghasilkan skor akhir 4,86 dari skala 5,00, yang membuktikan bahwa sistem ini efektif menjembatani kesenjangan antara alat riset canggih dengan kebutuhan praktis industri perangkat lunak. Secara nyata, sistem ini berkontribusi pada peningkatan keandalan layanan digital sehari-hari dengan memungkinkan pengembang mendeteksi dan memperbaiki celah keamanan kritis secara dini sebelum berdampak pada pengguna akhir.

Kata Kunci: Fuzzing REST API, Aplikasi Web, Multi-Agent Reinforcement Learning (MARL), Semantic Operation Dependency Graph (SODG), Pengujian Perangkat Lunak Otomatis, Pengujian API Stateful.

ABSTRACT

Fuzzing System Development for REST API Quality Testing Using Multi-Agent Reinforcement Learning and Semantic Operation Dependency Graph

By

David Lois

22/498373/PA/21509

REST API testing faces significant challenges due to complex inter-operation dependencies and a vast input space. State-of-the-art testing tools like AutoRestTest offer effective solutions through Semantic Operation Dependency Graph (SODG) and Multi-Agent Reinforcement Learning (MARL); however, their adoption is often limited by complex command-line interfaces and minimal workflow integration. This research develops a web-based fuzzing system that integrates AutoRestTest with an asynchronous architecture and CI/CD automation. The development is conducted end-to-end, encompassing a management interface, task runner orchestration, and intelligent model wrappers. Performance testing results demonstrate the system's capability to handle high loads with an average response time of 149 ms and an internal error rate of 0.01%. Functional validation on the target service successfully detected 77.8% of broken endpoints with a total of 16,739 requests generated in 60 seconds. Usability evaluation yielded a final score of 4.86 out of 5.00, proving that this system effectively bridges the gap between advanced research tools and the practical needs of the software industry. In practice, this system contributes to the reliability of everyday digital services by enabling developers to detect and fix critical security vulnerabilities early before they impact end-users.

Keywords: REST API Fuzzing, Web Application, Multi-Agent Reinforcement Learning (MARL), Semantic Operation Dependency Graph (SODG), Automated Software Testing, Stateful API Testing.

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam ekosistem perangkat lunak modern, *Application Programming Interface* (API) dengan gaya arsitektur *Representational State Transfer* (REST) telah menjadi standar *de facto* untuk komunikasi antar layanan (*service-to-service*) dan antara klien dan server (Postman, 2023). Keberhasilan fungsionalitas, keamanan, dan keandalan aplikasi sangat bergantung pada kualitas REST API yang mendasarinya. Oleh karena itu, pengujian API yang efektif dan efisien merupakan aspek krusial dalam siklus hidup pengembangan perangkat lunak untuk memastikan kualitas dan mencegah celah keamanan.

Tantangan utama dalam pengujian REST API terletak pada kompleksitas dan cakupan pengujian. Banyak pendekatan awal dan beberapa alat otomatis bergantung pada spesifikasi formal API, umumnya dalam format *OpenAPI Specification* (OAS) atau Swagger, untuk memandu proses pengujian. Sebagai contoh, RESTTESTGEN (Viglianisi et al., 2020) merupakan salah satu pendekatan yang secara otomatis menghasilkan *test case* untuk skenario nominal (penggunaan normal) dan skenario *error* berdasarkan definisi antarmuka dalam OAS. Namun, ketergantungan pada OAS memiliki keterbatasan fundamental. Spesifikasi ini seringkali hanya mendefinisikan aspek sintaksis dan struktural API, namun gagal menangkap informasi tersirat, seperti logika bisnis, urutan operasi implisit yang diperlukan (dependensi logis), atau batasan nilai input yang tidak terdokumentasi secara eksplisit (Corradini & Pasqua, 2024). Keterbatasan ini menghalangi kemampuan alat pengujian *black-box* untuk menghasilkan *test case* yang benar-benar mendalam dan efektif.

Selain keterbatasan dokumentasi, tantangan signifikan lainnya adalah menghasilkan urutan (*sequence*) pemanggilan operasi API yang bermakna dan valid. Banyak operasi API bersifat *stateful*, di mana hasil dari satu operasi menjadi prasyarat atau input untuk operasi berikutnya. Memahami dan memodelkan dependensi antar operasi ini sangat penting untuk pengujian yang mendalam.

Menjawab tantangan ini, dan sebagai salah satu upaya untuk melakukan *fuzzing*—yaitu strategi pengujian dengan mengirimkan beragam variasi input secara otomatis ke sistem target guna menemukan perilaku tak terduga sekaligus menjawab berbagai tantangan pengujian REST API— pendekatan seperti MOREST (Liu et al., 2022) mencoba membangun dan memelihara model perilaku layanan REST secara dinamis dalam bentuk *RESTful-service Property Graph* (RPG). RPG ini diperbarui berdasarkan umpan balik eksekusi (*execution feedback*) dan digunakan untuk memandu generasi urutan panggilan API, dengan tujuan meningkatkan kesadaran (*awareness*) tentang bagaimana semua API saling terhubung dan berinteraksi.

Lebih lanjut, ruang pencarian (*search space*) dalam pengujian REST API sangatlah luas, mencakup pemilihan operasi dari puluhan atau ratusan kemungkinan, pemilihan parameter yang relevan untuk setiap operasi, dan pengambilan sampel nilai dari ruang input parameter yang bisa jadi tak terbatas (Kim et al., 2023). Melakukan eksplorasi secara acak atau menyeluruh seringkali tidak efisien. Untuk mengatasi hal ini, teknik adaptif diperlukan. ARAT-RL (Kim et al., 2023) mengusulkan penggunaan *Reinforcement Learning* (RL) untuk secara adaptif memprioritaskan eksplorasi operasi dan parameter berdasarkan analisis dinamis data permintaan dan respons. Pendekatan ini bertujuan membuat eksplorasi lebih efisien, terutama ketika spesifikasi (misalnya, skema respons) tidak lengkap atau dinamis.

Munculnya teknik *machine learning*, khususnya *Reinforcement Learning* (RL), memang menawarkan arah baru yang menjanjikan untuk otomatisasi pengujian REST API yang lebih cerdas. Selain ARAT-RL yang fokus pada prioritisasi adaptif, DeepREST (Corradini & Pasqua, 2024) memanfaatkan *Deep Reinforcement Learning* (DRL) dengan strategi *curiosity-driven learning* untuk secara spesifik mencoba mengungkap batasan-batasan implisit yang tersembunyi dari dokumentasi. Meskipun pendekatan-pendekatan berbasis RL ini menunjukkan kemajuan signifikan dalam menangani aspek-aspek tertentu (prioritisasi, batasan implisit, adaptasi), tantangan untuk secara komprehensif

menangani semua kompleksitas secara simultan—termasuk dependensi eksplisit dan implisit, eksplorasi terstruktur di berbagai dimensi (operasi, parameter, nilai, header), serta generasi nilai yang semantik—masih tetap ada.

Dalam konteks inilah, AutoRestTest (Stennett et al., 2025) diperkenalkan sebagai sebuah pendekatan yang mengintegrasikan *Semantic Operation Dependency Graph* (SODG) untuk memodelkan dependensi antar operasi secara eksplisit, dengan *Multi-Agent Reinforcement Learning* (MARL) yang menggunakan lima agen terspesialisasi untuk mengelola eksplorasi yang kompleks dan terstruktur pada berbagai aspek API (operasi, parameter, nilai, dependensi, header). Pendekatan ini juga diperkaya dengan *Large Language Models* (LLMs) untuk meningkatkan pemahaman semantik dan generasi data uji yang relevan. Evaluasi awal yang dilaporkan menunjukkan bahwa AutoRestTest secara signifikan mengungguli beberapa alat *state-of-the-art* lainnya, termasuk ARAT-RL dan MOREST (serta RESTler dan EvoMaster), dalam hal cakupan operasi yang berhasil dieksekusi pada *benchmark* yang digunakan (Stennett et al., 2025). Hal ini menunjukkan potensi superior AutoRestTest dalam menavigasi dan menguji konfigurasi API yang kompleks.

Melihat potensi dan hasil menjanjikan dari AutoRestTest sebagai representasi terkini (*state-of-the-art*) dalam *fuzzing* REST API cerdas, terdapat peluang untuk mengadopsi dan mengintegrasikannya ke dalam bentuk yang lebih mudah diakses oleh praktisi. Saat ini, AutoRestTest (seperti banyak alat riset lainnya) hadir sebagai *tool* baris perintah atau pustaka kode yang membutuhkan pemahaman teknis mendalam untuk digunakan. Selain dari itu, pengembang AutoRestTest juga tidak menyediakan metode praktis untuk integrasi dengan *workflow* CI/CD (Stennett et al., 2025). Penelitian ini bertujuan untuk mengisi celah tersebut dengan mengembangkan sebuah sistem API *fuzzer* berbasis web yang didasarkan pada prinsip-prinsip inti AutoRestTest (SODG, MARL terspesialisasi, pemanfaatan LLM). Aplikasi web ini dirancang untuk menyediakan antarmuka pengguna yang intuitif, memungkinkan pengembang atau *tester* untuk mendefinisikan target API, mengonfigurasi dan menjalankan proses

fuzzing, serta menganalisis hasilnya. Dengan demikian, penelitian ini berkontribusi dengan menjembatani riset mutakhir dalam otomatisasi pengujian API dengan kebutuhan praktis di industri serta menyediakan alat bantu yang kuat namun mudah digunakan untuk meningkatkan kualitas dan keamanan REST API.

1.2. Perumusan Produk

Permasalahan mendasar dalam pengujian kualitas REST API saat ini adalah adanya kesenjangan antara kecanggihan teknik pengujian berbasis riset dengan aksesibilitasnya bagi praktisi di industri, di mana alat fuzzer *state-of-the-art* seringkali hanya tersedia dalam bentuk baris perintah (*terminal*) yang kompleks dan sulit diintegrasikan ke dalam alur kerja pengembangan rutin. Untuk menjawab tantangan tersebut, penelitian ini merumuskan produk berupa **Sistem Fuzzing REST API Berbasis Web Terintegrasi** yang mengimplementasikan model *AutoRestTest* dengan pendekatan *end-to-end*.

Dalam konteks ini, pendekatan *end-to-end* dipahami sebagai pengembangan infrastruktur lengkap yang menjembatani logika pengujian komputasi yang kompleks meliputi *Semantic Operation Dependency Graph* (SODG) dan *Multi-Agent Reinforcement Learning* (MARL) menjadi sebuah layanan aplikasi yang dapat digunakan oleh pengguna secara intuitif. Secara umum, pengembangan sistem ini mencakup komponen antarmuka pengguna berbasis web untuk mempermudah konfigurasi dan pemantauan proses pengujian, komponen *backend* yang mengelola orkestrasi tugas pengujian secara asinkron, serta penyediaan solusi integrasi otomatis ke dalam *pipeline* CI/CD melalui GitHub Actions.

Keunggulan dari produk yang dirumuskan ini terletak pada kemampuannya untuk mentransformasi alat riset murni menjadi solusi praktis yang siap digunakan di lingkungan produksi. Dengan mengintegrasikan mekanisme cerdas dari *AutoRestTest*, sistem ini mampu menyajikan hasil pengujian yang lebih mendalam pada API yang bersifat *stateful* dan kompleks, sekaligus mempercepat siklus umpan balik (*feedback*) bagi pengembang melalui otomatisasi pengujian yang berkelanjutan.

1.3. Batasan Produk

Untuk memastikan fokus dan keberhasilan penelitian dalam mencapai tujuan yang telah ditetapkan, perlu ditetapkan batasan-batasan produk yang jelas. Pengembangan sistem *fuzzing* REST API berbasis web dalam konteks penelitian ini melibatkan beberapa aspek teknis dengan batasan sebagai berikut:

1. Sistem hanya mendukung pembacaan dan pemrosesan spesifikasi REST API dalam format *OpenAPI Specification* (OAS) versi 2 atau 3 dalam format file JSON.
2. Implementasi logika inti *fuzzer* didasarkan pada prinsip-prinsip model *AutoRestTest* yang mengintegrasikan mekanisme *Semantic Operation Dependency Graph* (SODG) dan *Multi-Agent Reinforcement Learning* (MARL), tanpa melakukan modifikasi pada logika dasar algoritma agen aslinya.
3. Solusi integrasi alur kerja CI/CD dibatasi secara eksklusif pada platform GitHub Actions, dengan memanfaatkan izin akses modifikasi konten repositori melalui protokol GitHub OAuth2.
4. Arsitektur aplikasi dikembangkan menggunakan kerangka kerja Next.js untuk sisi *frontend* dan *backend* API server, serta skrip Python berbasis FastAPI untuk komponen *worker* yang menjalankan proses *fuzzing*.
5. Evaluasi kegunaan (*usability*) aplikasi dilakukan melalui sesi *task-based usability testing* terbatas dengan melibatkan partisipan yang representatif seperti mahasiswa informatika atau pengembang perangkat lunak.
6. Pengujian performa aplikasi web difokuskan pada pemenuhan standar metrik usabilitas yang ditetapkan melalui alat audit Google Lighthouse.

1.4. Tujuan Penelitian

Selaras dengan rumusan masalah yang telah dipaparkan sebelumnya, tujuan dari penelitian ini adalah sebagai berikut:

1. Merancang dan mengimplementasikan arsitektur *web application* API *fuzzer* yang mampu mengintegrasikan *AutoRestTest*, memanfaatkan

Semantic Operation Dependency Graph (SODG) dan *Multi-Agent Reinforcement Learning* (MARL) dengan agen terspesialisasi.

2. Mengembangkan antarmuka pengguna (*user interface*) untuk *web application API fuzzer* untuk memudahkan pengguna dalam melakukan konfigurasi, eksekusi, pemantauan, dan interpretasi hasil proses *fuzzing* API.
3. Menyediakan solusi integrasi API *fuzzing* dengan CI/CD pipeline yang tidak memerlukan banyak keterlibatan *developer*.
4. Melakukan evaluasi terhadap *web application API fuzzer* yang telah dikembangkan untuk mengukur efektivitasnya dalam menjalankan sekuens operasi API yang kompleks dan mengidentifikasi potensi *fault*, serta menilai aspek kegunaan (*usability*) aplikasi dari perspektif pengguna.

1.5. Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat sebagai berikut:

1. Menjembatani kesenjangan antara riset akademis dan kebutuhan industri dengan menyediakan prototipe aplikasi web *fuzzer* yang mengemas kompleksitas *AutoRestTest*, SODG, dan MARL menjadi antarmuka intuitif bagi pengembang dan *Quality Assurance* (QA).
2. Meningkatkan kualitas dan keamanan REST API melalui alat pengujian cerdas yang mampu mendeteksi *bug*, *fault*, atau celah keamanan pada skenario *stateful* secara lebih akurat dibandingkan metode konvensional.
3. Mempercepat siklus umpan balik (*feedback loop*) dalam pengembangan perangkat lunak melalui integrasi otomatis ke dalam alur kerja CI/CD, sehingga memungkinkan deteksi dini masalah API demi efisiensi waktu dan sumber daya.

BAB II

IDENTIFIKASI MASALAH DAN IDE INOVATIF

2.1. Identifikasi Masalah

Berdasarkan penjabaran pada latar belakang, pengujian kualitas REST API dalam ekosistem pengembangan perangkat lunak modern menghadapi tantangan yang menghambat efektivitas dan efisiensi penjaminan mutu aplikasi. Permasalahan tersebut muncul seiring dengan meningkatnya kompleksitas arsitektur perangkat lunak yang menuntut strategi pengujian yang lebih cerdas dan terintegrasi. Secara umum, permasalahan-permasalahan tersebut dapat dikelompokkan menjadi beberapa aspek utama sebagai berikut.

2.1.1. Hambatan Aksesibilitas dan Kompleksitas Operasional Alat Fuzzing Akademik

Meskipun penelitian di bidang otomatisasi pengujian telah menghasilkan berbagai pendekatan *state-of-the-art* seperti AutoRestTest yang menunjukkan keunggulan dalam cakupan pengujian, adopsinya oleh praktisi di industri masih sangat terbatas. Hambatan utama terletak pada sifat alat-alat riset tersebut yang umumnya dikembangkan sebagai perangkat lunak baris perintah (*command-line interface* atau CLI) atau pustaka kode mentah yang memerlukan pemahaman teknis mendalam untuk dioperasikan.

Ketergantungan pada antarmuka berbasis terminal ini menimbulkan kompleksitas operasional bagi penguji perangkat lunak (*tester*) maupun pengembang dalam beberapa hal. Pertama, proses pendefinisian target API dan konfigurasi parameter *fuzzing* yang kompleks menjadi kurang intuitif karena dilakukan melalui berkas konfigurasi manual atau argumen baris perintah. Kedua, pelaksanaan proses pengujian dan pemantauan status *fuzzing* secara *real-time* sulit dilakukan tanpa bantuan antarmuka visual yang informatif. Ketiga, hasil pengujian yang disajikan sering kali berupa *log* mentah yang sulit diinterpretasikan, sehingga menghambat proses analisis temuan *bug* atau celah keamanan secara cepat. Kondisi ini menyebabkan kesenjangan yang signifikan antara kecanggihan metode

pengujian berbasis AI dengan kemudahan penggunaannya dalam praktik pengembangan sehari-hari.

2.1.2. Kesenjangan Integrasi dalam Alur Kerja Industri (CI/CD)

Dalam praktik rekayasa perangkat lunak modern, efisiensi penjaminan mutu sangat bergantung pada otomatisasi pengujian yang terintegrasi secara mulus ke dalam alur kerja *Continuous Integration/Continuous Delivery* (CI/CD). Namun, terdapat kesenjangan teknis yang signifikan pada alat-alat *fuzzing* API tingkat riset. Sebagai contoh, pengembang *AutoRestTest* belum menyediakan metode praktis yang memungkinkan alat tersebut untuk diintegrasikan secara langsung dengan *workflow* CI/CD di industri.

Kondisi ini diperparah dengan belum tersedianya solusi integrasi yang siap pakai untuk platform *software workflow* populer seperti GitHub Actions. Tanpa adanya integrasi ini, proses *fuzzing* API cerdas tetap bersifat manual dan sporadis, alih-alih menjadi bagian dari pengujian otomatis yang berjalan setiap kali terdapat perubahan kode. Akibatnya, penguji tidak mendapatkan umpan balik secara kontinu, sehingga potensi *fault* atau perilaku tak terduga pada API sering kali baru terdeteksi di tahap akhir pengembangan. Hal ini meningkatkan risiko teknis dan biaya perbaikan perangkat lunak karena deteksi dini kegagalan sistem tidak terjadi secara otomatis dalam siklus pengembangan.

2.1.3. Beban Kognitif dalam Interpretasi dan Manajemen Hasil Pengujian

Selain masalah aksesibilitas dan integrasi, tantangan lain muncul pada tahap manajemen dan analisis hasil pengujian. Alat *fuzzer* akademik yang berbasis terminal umumnya menyajikan luaran dalam bentuk *log* teks mentah yang sangat panjang dan kompleks. Antarmuka berbasis terminal tersebut menyulitkan penguji dalam melakukan interpretasi terhadap hasil pengujian yang disajikan.

Ketiadaan representasi visual dan ringkasan statistik yang terstruktur memaksa pengguna untuk membedah data mentah secara manual guna menemukan sekuens operasi spesifik yang memicu kegagalan sistem. Hal ini menimbulkan beban kognitif yang tinggi bagi pengembang maupun penguji, terutama saat mencoba memahami perilaku API yang memiliki banyak *endpoint*

dengan dependensi antar-operasi yang rumit. Tanpa adanya dasbor hasil yang intuitif untuk pemantauan dan fitur penyimpanan riwayat pengujian, proses evaluasi kualitas REST API menjadi tidak efisien dan sulit untuk dilakukan secara berkelanjutan dalam jangka panjang.

2.2. Pengembangan Ide Inovatif

Berdasarkan identifikasi masalah yang telah diuraikan, diperlukan suatu pendekatan yang tidak hanya meningkatkan efektivitas deteksi kesalahan pada API, tetapi juga memprioritaskan kemudahan akses operasional bagi praktisi di lingkungan industri. Penelitian ini merumuskan pengembangan Sistem API *Fuzzer* Berbasis Web Terintegrasi yang memanfaatkan model *AutoRestTest* sebagai inti sistem. Perumusan ide inovatif ini dikembangkan melalui empat aspek utama yang mencerminkan kebutuhan teknis, operasional, dan pengalaman pengguna sebagai berikut.

2.2.1. Demokratisasi Pengujian API Cerdas melalui Antarmuka Web

Inovasi fundamental dari penelitian ini adalah transformasi alat pengujian REST API dari bentuk baris perintah (*command-line interface*) yang umumnya eksklusif bagi kalangan peneliti menjadi sebuah aplikasi web yang intuitif bagi khalayak luas. Sistem yang dirancang menempatkan aksesibilitas sebagai elemen fundamental dengan menyediakan antarmuka pengguna (UI) berbasis Next.js yang memandu pengguna melalui proses konfigurasi parameter pengujian yang kompleks. Melalui antarmuka ini, teknis pengujian mutakhir seperti *Semantic Operation Dependency Graph* (SODG) dan *Multi-Agent Reinforcement Learning* (MARL) dapat dioperasikan dengan mudah tanpa mengharuskan pengguna memahami kerumitan matematis di baliknya. Pendekatan ini secara efektif menjembatani kesenjangan antara hasil riset akademis dan kebutuhan praktis di industri.

2.2.2. Integrasi Rekomendasi Proaktif dalam Alur Kerja CI/CD

Berbeda dengan alat *fuzzing* konvensional yang bersifat sporadis dan manual, sistem yang diajukan mengadopsi paradigma pengujian proaktif melalui integrasi langsung dengan *pipeline* CI/CD, khususnya GitHub Actions. Setelah

pengguna melakukan otorisasi melalui GitHub OAuth2, sistem dapat secara otomatis membuat dan menyisipkan file *workflow* ke dalam repositori target. Dengan mekanisme ini, sesi *fuzzing* akan dipicu secara otomatis oleh kejadian tertentu, seperti aktivitas *push* kode ke cabang utama (*main branch*). Inovasi ini memastikan adanya siklus umpan balik (*feedback*) yang kontinu dan otomatis, sehingga pengembang dapat mendeteksi potensi *fault* atau kerentanan keamanan pada tahap paling awal dalam siklus pengembangan perangkat lunak.

2.2.3. Arsitektur Terintegrasi untuk Operasional Skala Industri

Untuk menangani proses pengujian yang intensif dan durasi komputasi yang lama, sistem ini mengadopsi arsitektur asinkron terdistribusi yang dirancang untuk skala produksi. Inovasi arsitektural ini memisahkan tanggung jawab antara API Server berbasis Python untuk manajemen trafik yang efisien dengan sekumpulan *Worker* berbasis Python yang menjalankan logika inti *AutoRestTest*. Penggunaan *Message Queue* berbasis RabbitMQ bertindak sebagai penyangga (*buffer*) yang memastikan stabilitas sistem dalam menangani antrean pekerjaan pengujian yang masuk secara bersamaan (*concurrent*). Pendekatan ini menjamin antarmuka pengguna tetap responsif dan bebas dari hambatan (*bottleneck*) meskipun proses *fuzzing* yang berat sedang berjalan di latar belakang.

2.2.4. Aksesibilitas dan Visualisasi Hasil sebagai Fokus Desain

Sebagai solusi atas sulitnya menginterpretasikan luaran *log* mentah dari alat pengujian tradisional, sistem ini menghadirkan dasbor hasil yang komprehensif dan terstruktur. Inovasi ini diwujudkan melalui penyajian visual terhadap statistik pengujian, kode status operasi, representasi *Q-tables* dari agen RL, hingga rincian *body request* yang memicu kesalahan server. Selain itu, sistem mengimplementasikan persistensi data menggunakan PostgreSQL untuk menyimpan riwayat sesi pengujian secara permanen. Hal ini memungkinkan pengguna terautentikasi untuk melacak progres kualitas API mereka dari waktu ke waktu, melakukan audit terhadap temuan masalah sebelumnya, dan mengelola hasil pengujian secara lebih efisien.

BAB III

KAJIAN ILMIAH

3.1. REST API

Sebelum membahas metode pengujian, pemahaman mendalam mengenai objek yang akan diuji menjadi fondasi yang krusial. Subbab ini akan menguraikan prinsip dasar arsitektur komunikasi layanan yang menjadi standar industri saat ini, serta format dokumentasi yang digunakan sistem untuk memahami struktur layanan tersebut. Rincian mengenai arsitektur dan spesifikasi tersebut dijelaskan dalam poin-poin berikut.

3.1.1. Arsitektur Representational State Transfer (REST)

Dalam ekosistem perangkat lunak modern, *Application Programming Interface* (API) dengan gaya arsitektur *Representational State Transfer* (REST) telah menjadi standar *de facto* untuk komunikasi antar layanan (*service-to-service*) maupun antara klien dan server. Keberhasilan fungsionalitas, keamanan, dan keandalan aplikasi sangat bergantung pada kualitas REST API yang mendasarinya. Arsitektur ini mengandalkan protokol HTTP yang bersifat *stateless*, di mana setiap permintaan dari klien harus memuat semua informasi yang diperlukan oleh server untuk memahami dan memproses permintaan tersebut (Richardson & Ruby, 2007).

3.1.2. OpenAPI Specification (OAS)

Untuk memandu proses pengujian otomatis, seringkali digunakan spesifikasi formal API dalam format *OpenAPI Specification* (OAS) atau yang sebelumnya dikenal sebagai Swagger. OAS mendefinisikan aspek sintaksis dan struktural dari sebuah API, termasuk daftar *endpoint*, metode HTTP yang didukung, serta parameter masukan dan skema respons. Dalam penelitian ini, OAS menjadi sumber data utama yang diolah oleh sistem untuk membangun pemahaman mengenai target API yang akan diuji (Miller et al., 2021).

3.2. Fuzzing REST API

Strategi pengujian otomatis diperlukan untuk menemukan kesalahan yang mungkin terlewatkan oleh pengujian manual. Bagian ini akan membahas konsep dasar dari teknik pengujian yang memanfaatkan input acak, serta mengeksplorasi tantangan spesifik yang muncul ketika teknik tersebut diterapkan pada API yang memiliki ketergantungan urutan operasi (*stateful*). Penjelasan mengenai konsep dan tantangan tersebut dijabarkan di bawah ini.

3.2.1. Konsep Fuzz Testing

Fuzzing atau *fuzz testing* adalah strategi pengujian perangkat lunak dengan cara mengirimkan beragam variasi data *input* secara otomatis ke sistem target guna menemukan perilaku tak terduga, *bug*, atau celah keamanan. Teknik ini sangat efektif dalam mendeteksi kesalahan yang mungkin terlewatkan melalui pengujian manual atau pengujian unit tradisional (Godefroid, 2020).

3.2.2. Tantangan Pengujian API Stateful

Salah satu tantangan signifikan dalam pengujian REST API adalah sifatnya yang seringkali bersifat *stateful*. Dalam API yang *stateful*, hasil dari satu operasi sering menjadi prasyarat atau *input* untuk operasi berikutnya, sehingga urutan (*sequence*) pemanggilan API menjadi sangat krusial. Alat pengujian konvensional seringkali gagal menangkap dependensi logis ini, yang mengakibatkan rendahnya cakupan pengujian pada logika bisnis yang kompleks.

3.3. Semantic Operation Dependency Graph (SODG)

Untuk mengatasi tantangan pada API yang bersifat *stateful*, diperlukan sebuah model yang mampu memetakan hubungan antar operasi secara eksplisit. Subbab ini menjelaskan mekanisme pemodelan dependensi yang digunakan untuk menjaga validitas urutan pemanggilan API, mulai dari konsep pemodelan hingga teknik ekstraksi informasinya dari dokumentasi.

3.3.1. Pemodelan Dependensi Operasi

Untuk mengatasi masalah pada API *stateful*, diperlukan pemodelan yang dapat menggambarkan keterhubungan antar operasi. *Semantic Operation Dependency Graph* (SODG) diperkenalkan sebagai pendekatan untuk

memodelkan dependensi antar operasi secara eksplisit. SODG memungkinkan sistem pengujian untuk memahami operasi mana yang bertindak sebagai produsen data (*producer*) dan operasi mana yang bertindak sebagai konsumen (*consumer*) (Stennett et al., 2025).

3.3.2. Ekstraksi Semantik dari Dokumentasi API

SODG bekerja dengan cara mengekstraksi informasi dari dokumentasi API (OAS) dan menganalisis hubungan antar parameter. Dengan memetakan dependensi ini ke dalam struktur graf, sistem dapat menghasilkan sekuens pemanggilan API yang lebih bermakna dan valid dibandingkan dengan pemanggilan secara acak. Penggunaan *cached graph* dalam sistem ini bertujuan untuk meningkatkan efisiensi proses pengujian dengan menghindari rekonstruksi graf yang berulang untuk target API yang sama (Stennett et al., 2025).

3.4. Multi-Agent Reinforcement Learning (MARL)

Selain pemodelan dependensi, efisiensi pengujian sangat bergantung pada strategi eksplorasi. Bagian ini membahas penerapan teknik pembelajaran mesin yang memungkinkan sistem belajar dari interaksi sebelumnya. Pembahasan mencakup konsep dasar *Reinforcement Learning*, pembagian tugas pada agen-agen yang terspesialisasi, serta parameter yang mempengaruhi proses optimasi agen tersebut.

3.4.1. Reinforcement Learning (RL) dalam Pengujian

Reinforcement Learning (RL) merupakan cabang dari *machine learning* yang berfokus pada bagaimana agen cerdas mengambil tindakan dalam suatu lingkungan untuk memaksimalkan akumulasi imbalan (*reward*). Dalam konteks pengujian perangkat lunak, RL memungkinkan agen untuk mempelajari strategi pengujian optimal melalui interaksi langsung dengan API target. Agen belajar memprioritaskan operasi dan parameter yang lebih penting atau kompleks untuk diuji berdasarkan umpan balik dinamis dari respons API (Sutton & Barto, 2020).

3.4.2. Agen Terspesialisasi dalam AutoRestTest

Sistem ini mengadopsi pendekatan *Multi-Agent Reinforcement Learning* (MARL) yang diusulkan dalam model *AutoRestTest*. Pendekatan ini menggunakan

lima jenis agen yang memiliki tugas terspesialisasi untuk mengelola eksplorasi pada berbagai dimensi pengujian API, yaitu:

1. **Agen Operasi:** Bertugas memilih metode HTTP dan *endpoint* yang akan diuji.
2. **Agen Parameter:** Mengelola pemilihan parameter yang relevan untuk setiap operasi.
3. **Agen Nilai:** Bertanggung jawab atas pengambilan sampel nilai masukan dari ruang *input* yang luas.
4. **Agen Dependensi:** Mengelola keterkaitan antar operasi berdasarkan model SODG.
5. **Agen Header:** Mengatur pengujian pada bagian *header* permintaan.

3.4.3. Parameter Optimasi RL

Implementasi sistem ini menyediakan antarmuka bagi pengguna untuk mengatur parameter optimasi agen RL guna menyesuaikan intensitas dan perilaku pengujian. Parameter tersebut meliputi:

1. **Learning Rate:** Menentukan seberapa besar informasi baru akan memperbarui nilai pengetahuan agen saat ini.
2. **Discount Factor:** Mengatur tingkat kepentingan imbalan di masa depan dibandingkan dengan imbalan instan.
3. **Max Exploration:** Mengontrol keseimbangan antara eksplorasi (*exploration*) untuk mencoba tindakan baru dan eksploitasi (*exploitation*) terhadap tindakan yang sudah diketahui efektif.

3.5. Large Language Models (LLM) dalam Otomasi Pengujian

Pemahaman terhadap semantik data merupakan kunci untuk menghasilkan input pengujian yang relevan dan bermakna. Subbab ini menguraikan peran model bahasa besar dalam meningkatkan kualitas data uji, meliputi kemampuan pemahaman konteks bisnis dari dokumentasi serta teknis integrasi mesin LLM ke dalam sistem pengujian.

3.5.1. Pemahaman Konteks Semantik

Integrasi *Large Language Models* (LLM) bertujuan untuk meningkatkan pemahaman semantik terhadap dokumentasi API. Berbeda dengan metode tradisional yang menghasilkan data secara acak, LLM memungkinkan sistem untuk menghasilkan data uji yang lebih relevan dan bermakna secara semantik berdasarkan konteks bisnis yang didefinisikan dalam OAS.

3.5.2. Integrasi LLM Engine

Dalam implementasi sistem ini, fungsionalitas LLM diintegrasikan melalui layanan API OpenAI, sebagaimana tercantum dalam dependensi *backend*. Pengguna dapat memilih jenis *engine* (misalnya GPT-4) dan mengatur parameter *temperature* melalui antarmuka aplikasi. Parameter *temperature* ini menentukan tingkat kreativitas atau keacakan LLM dalam menghasilkan variasi data uji.

3.6. Jenis Pengujian Spesifikasi REST API

Terdapat beragam pendekatan dalam otomatisasi pengujian REST API yang telah dikembangkan sebelumnya. Bagian ini menyajikan tinjauan literatur mengenai berbagai kategori pengujian untuk memosisikan penelitian ini dalam ranah keilmuan yang ada. Tinjauan ini mencakup pendekatan berbasis spesifikasi murni, pengujian berbasis model untuk API *stateful*, hingga pendekatan terkini yang memanfaatkan *Reinforcement Learning*.

3.6.1. Pengujian REST API Berbasis Spesifikasi

Pendekatan umum dalam otomatisasi pengujian REST API adalah memanfaatkan spesifikasi formal API, seperti *OpenAPI Specification* (OAS) atau GraphQL schemas. Spesifikasi ini menjadi sumber informasi utama mengenai struktur API.

Berbagai alat memanfaatkan spesifikasi ini dengan cara berbeda. RESTTESTGEN (Viglianisi et al., 2020), misalnya, secara otomatis menganalisis file OAS untuk menghasilkan *test case* yang menargetkan skenario penggunaan normal dan skenario *error*, menggunakan *oracle* khusus untuk mendeteksi *fault* berdasarkan definisi antarmuka (Viglianisi et al., 2020). Pendekatan lain yang juga berbasis skema adalah Schemathesis (Hatfield-Dodds & Dygalo, 2022).

Schemathesis menerapkan prinsip *property-based testing* yang diturunkan dari skema OpenAPI atau GraphQL. Tujuannya tidak hanya mencari *crash* tetapi juga menemukan *error* semantik (pelanggaran terhadap properti yang diharapkan dari API) dengan cara menghasilkan input yang beragam namun tetap valid sesuai skema (Hatfield-Dodds & Dygalo, 2022). Evaluasi yang dilaporkan menunjukkan efektivitas Schemathesis dalam menemukan *defect* pada berbagai layanan web (Hatfield-Dodds & Dygalo, 2022).

Meskipun demikian, ketergantungan pada spesifikasi memiliki batasan. Spesifikasi mungkin tidak lengkap, tidak akurat, atau gagal menangkap logika bisnis dan batasan implisit yang penting (Corradini & Pasqua, 2024). Oleh karena itu, meskipun alat berbasis spesifikasi seperti RESTTESTGEN dan Schemathesis berguna, mereka mungkin tidak cukup untuk menguji perilaku API yang kompleks secara mendalam, terutama yang melibatkan urutan operasi dan manajemen *state*.

3.6.2. Pengujian REST API Stateful dan Berbasis Model

Menyadari bahwa banyak REST API bersifat *stateful* dan memerlukan urutan operasi yang benar untuk pengujian mendalam, penelitian telah berfokus pada penanganan dependensi dan *state*.

RESTler (Atlidakis et al., 2019) merupakan salah satu *fuzzer* pertama yang secara eksplisit dirancang untuk pengujian REST API *stateful*. RESTler menganalisis spesifikasi API untuk **menginferensi dependensi *producer-consumer*** antar operasi (misalnya, operasi B membutuhkan ID yang dihasilkan oleh operasi A). Informasi ini digunakan untuk membangun urutan operasi yang valid. Selain itu, RESTler menggunakan **umpan balik dinamis** dari respons API selama eksekusi untuk mempelajari urutan mana yang valid atau tidak valid, sehingga dapat memangkas ruang pencarian dan memandu generasi *test sequence* berikutnya. RESTler terbukti berhasil menemukan *bug* nyata di layanan cloud skala besar seperti GitLab, Azure, dan Office365 (Atlidakis et al., 2019).

Pendekatan lain untuk menangani dependensi dan perilaku dinamis adalah melalui pembangunan model. MOREST (Liu et al., 2022) membangun dan memelihara **model graf dinamis** (*RESTful-service Property Graph* - RPG) berdasarkan umpan balik eksekusi. RPG ini memodelkan operasi, parameter, dan dependensi data yang teramati, bertujuan untuk mendapatkan pemahaman menyeluruh tentang bagaimana operasi-operasi saling terhubung dan memandu generasi urutan panggilan API (Liu et al., 2022).

Baik RESTler maupun MOREST menunjukkan pentingnya memahami dan memanfaatkan dependensi antar operasi untuk pengujian yang efektif. Namun, tantangan tetap ada dalam hal skalabilitas model, penanganan *state* yang sangat kompleks, dan penemuan dependensi yang tidak mudah diinferensi dari spesifikasi atau observasi langsung.

3.6.3. Pengujian REST API Berbasis Reinforcement Learning (RL)

Dalam beberapa tahun terakhir, *Reinforcement Learning* (RL) telah muncul sebagai teknik yang menjanjikan untuk mengatasi tantangan kompleksitas dan ruang pencarian yang luas dalam pengujian REST API. RL memungkinkan agen belajar strategi pengujian optimal melalui interaksi langsung dengan API.

Berbagai penelitian telah menerapkan RL dengan fokus yang berbeda-beda. ARAT-RL (Kim et al., 2023) menggunakan RL untuk mengatasi masalah inefisiensi eksplorasi. Dengan menganalisis data permintaan dan respons secara dinamis, agen RL dalam ARAT-RL belajar untuk memprioritaskan operasi dan parameter mana yang lebih penting atau kompleks untuk diuji, sehingga menjadikan proses pengujian lebih adaptif dan efisien, terutama ketika spesifikasi API tidak lengkap (Kim et al., 2023).

Sementara itu, DeepREST (Corradini & Pasqua, 2024) secara spesifik memanfaatkan *Deep Reinforcement Learning* (DRL) yang didorong oleh 'rasa ingin tahu' (*curiosity-driven learning*). Fokus utama DeepREST adalah agar agen RL secara aktif mencari dan mempelajari batasan-batasan implisit dalam API—aturan atau perilaku yang tidak terdokumentasi—dengan cara mencoba

berbagai interaksi dan belajar dari hasilnya (Corradini & Pasqua, 2024). Ini menargetkan kelemahan mendasar dari ketergantungan pada spesifikasi formal.

Pendekatan RL yang lebih terintegrasi dan komprehensif diusulkan dalam *AutoRestTest* (Stennett et al., 2025). *AutoRestTest* tidak hanya menggunakan RL, tetapi menggabungkannya dengan teknik lain untuk mengatasi berbagai tantangan secara bersamaan. Ia menggunakan *Semantic Operation Dependency Graph* (SODG) untuk memodelkan dependensi eksplisit antar operasi terlebih dahulu. Kemudian, ia menerapkan *Multi-Agent Reinforcement Learning* (MARL) dengan lima agen yang memiliki tugas terspesialisasi (mengelola pemilihan operasi, parameter, nilai, dependensi, dan *header*). Pembagian tugas ini memungkinkan eksplorasi yang lebih terstruktur dan efektif di berbagai dimensi pengujian. *AutoRestTest* juga mengintegrasikan *Large Language Models* (LLMs) untuk meningkatkan pemahaman semantik dan generasi data (Stennett et al., 2025). Secara signifikan, evaluasi yang dilaporkan menunjukkan *AutoRestTest* mengungguli beberapa alat *state-of-the-art* lainnya, termasuk RESTler, MOREST, dan ARAT-RL, dalam hal cakupan operasi pada *benchmark* yang digunakan (Stennett et al., 2025).

3.7. AutoRestTest

Penelitian ini mengadopsi model *AutoRestTest* sebagai landasan utama pengembangan sistem. Subbab ini akan membedah secara spesifik bagaimana model tersebut mengintegrasikan berbagai komponen cerdas yang telah dibahas sebelumnya, serta menyajikan bukti empiris mengenai keunggulannya dibandingkan alat pengujian konvensional berdasarkan studi terdahulu.

3.7.1. Integrasi SODG, MARL, dan LLM

AutoRestTest merepresentasikan pendekatan terkini (*state-of-the-art*) dalam *fuzzing* REST API. Keunggulan utamanya terletak pada integrasi tiga komponen utama: SODG untuk pemodelan dependensi eksplisit, MARL untuk eksplorasi terstruktur multi-aspek, dan LLM untuk pemahaman semantik data (Stennett et al., 2025). Sinergi ketiga komponen ini memungkinkan sistem untuk menavigasi konfigurasi API yang kompleks secara otomatis.

3.7.2. Perbandingan dengan Alat Pengujian Tradisional

Berdasarkan evaluasi awal, pendekatan *AutoRestTest* terbukti secara signifikan mengungguli beberapa alat pengujian populer lainnya seperti *RESTler*, *MOREST*, dan *ARAT-RL* dalam hal cakupan operasi yang berhasil dieksekusi (Stennett et al., 2025). Hal ini membuktikan efektivitas sistem dalam menangani skenario pengujian API yang bersifat *stateful* dan memiliki ketergantungan antar-operasi yang tinggi.

3.8. Sintesis dan Posisi Penelitian

Tinjauan pustaka ini memetakan beragam pendekatan dalam otomatisasi pengujian REST API, mulai dari yang berbasis spesifikasi (*RESTTESTGEN*, *Schemathesis*), fokus pada *stateful* dan pemodelan dinamis (*RESTler*, *MOREST*), hingga berbagai aplikasi *Reinforcement Learning* (*ARAT-RL*, *DeepREST*, *AutoRestTest*). Masing-masing pendekatan memberikan kontribusi dalam mengatasi aspek-aspek tertentu dari tantangan pengujian API.

AutoRestTest (Stennett et al., 2025) dipilih sebagai landasan metodologis utama untuk penelitian ini karena beberapa alasan: (1) Pendekatannya yang **terintegrasi**, menggabungkan pemodelan dependensi eksplisit (*SODG*), eksplorasi multi-aspek (*MARL* terspesialisasi), dan potensi pemahaman semantik (*LLM*). (2) **Hasil evaluasi awal yang menjanjikan**, menunjukkan keunggulannya dibandingkan beberapa alat *state-of-the-art* lain yang relevan (termasuk *RESTler*, *MOREST*, *ARAT-RL*) dalam hal cakupan pengujian pada studi kasus yang dilaporkan.

Penelitian yang dilakukan dalam skripsi ini, oleh karena itu, **tidak bertujuan menciptakan metode *fuzzing* baru**, melainkan berfokus pada **implementasi dan integrasi *AutoRestTest*** dalam sebuah **platform *web application***. Tujuannya adalah untuk meningkatkan aksesibilitas dan kemudahan penggunaan teknik *fuzzing* REST API bagi para pengembang dan *tester*, sehingga menjembatani kesenjangan antara hasil riset mutakhir dan kebutuhan praktis di industri.

3.9. Arsitektur Aplikasi Web dan Komunikasi Asinkron

Implementasi sistem ini memerlukan serangkaian teknologi yang mampu mendukung interaksi pengguna yang responsif sekaligus menangani beban komputasi berat. Bagian ini menjabarkan tumpukan teknologi (*tech stack*) yang digunakan, mulai dari kerangka kerja *frontend*, layanan *backend* berbasis Python, hingga mekanisme orkestrasi tugas di latar belakang.

3.9.1. Framework Next.js dan Server-Side Rendering (SSR)

Sistem antarmuka pengguna dibangun menggunakan Next.js 15, sebuah kerangka kerja React yang mendukung *Server-Side Rendering* (SSR) dan *Static Site Generation* (SSG). Penggunaan Next.js memungkinkan pemisahan yang jelas antara logika *frontend* dan *backend* melalui pola *App Router*, serta memastikan performa aplikasi tetap optimal melalui optimasi pemuatan aset dan *rendering* di sisi *server* (Next.js, 2025).

3.9.2. FastAPI dan Python Backend

Komponen *model wrapper* dikembangkan menggunakan FastAPI, sebuah kerangka kerja web Python berkinerja tinggi yang dirancang untuk membangun API secara asinkron. Pemilihan FastAPI didasarkan pada dukungannya yang sangat baik terhadap ekosistem *machine learning* Python, kemudahan validasi data menggunakan Pydantic, serta performa eksekusi yang mendekati Node.js dan Go. Dalam sistem ini, FastAPI bertindak sebagai jembatan yang mengekspos fungsionalitas model *AutoRestTest* melalui protokol HTTP (FastAPI, 2025).

3.9.3. Orkestrasi Task (Trigger.dev)

Mengingat proses *fuzzing* REST API merupakan tugas yang memakan waktu lama (*long-running task*) dan intensif secara komputasi, sistem ini menggunakan Trigger.dev untuk orkestrasi tugas asinkron. Trigger.dev menyediakan platform terkelola untuk mendefinisikan, menjadwalkan, dan memantau *background jobs* tanpa hambatan waktu habis (*timeout*) yang sering terjadi pada *server* web tradisional. Dengan memindahkan beban kerja pengujian ke Trigger.dev, aplikasi utama tetap responsif bagi pengguna sementara proses *fuzzing* berjalan di latar belakang (Trigger.dev, 2025).

3.10. Persistensi Data dan Optimasi Performa

Pengelolaan data yang efisien dan cepat sangat diperlukan untuk mendukung fitur riwayat pengujian dan eksekusi model yang berulang. Subbab ini menjelaskan infrastruktur penyimpanan data yang dipilih, teknik pemetaan objek-relasional, serta strategi *caching* yang diterapkan untuk mengoptimalkan kinerja sistem secara keseluruhan.

3.10.1. PostgreSQL dan Basis Data Relasional

Untuk penyimpanan data yang bersifat persisten dan terstruktur, sistem ini menggunakan PostgreSQL. PostgreSQL dipilih karena keandalannya dalam menjaga integritas data, dukungan terhadap tipe data kompleks (seperti JSONB untuk menyimpan hasil summary pengujian), dan performa yang stabil untuk menangani riwayat pekerjaan pengujian dalam jumlah besar.

3.10.2. Prisma ORM

Akses dan manipulasi basis data pada sisi aplikasi *web* dikelola menggunakan Prisma *Object-Relational Mapping* (ORM). Prisma memberikan abstraksi yang bersifat *type-safe* untuk bahasa pemrograman TypeScript, sehingga meminimalisir kesalahan kueri dan mempercepat proses pengembangan melalui sinkronisasi otomatis antara skema basis data dengan model aplikasi (Prisma, 2025).

3.10.3. Sistem Caching dan In-Memory Database (Upstash Redis)

Eksekusi model pengujian cerdas memerlukan waktu komputasi yang signifikan, terutama saat berinteraksi dengan LLM. Untuk memitigasi latensi tersebut, sistem mengimplementasikan strategi *caching* menggunakan Upstash Redis sebagai *in-memory database*. Upstash Redis memungkinkan penyimpanan sementara hasil keluaran model (*model output*) dan metadata pengujian lainnya untuk mempercepat akses data pada permintaan yang sama di masa mendatang. Strategi ini krusial untuk meningkatkan pengalaman pengguna dengan menghindari proses kalkulasi ulang yang berat pada data yang telah tersedia di *cache*.

3.11. Continuous Integration dan Continuous Deployment (CI/CD)

Integrasi pengujian ke dalam alur kerja pengembangan merupakan aspek vital dalam rekayasa perangkat lunak modern. Bagian ini membahas konsep siklus umpan balik otomatis dan implementasi teknis integrasi sistem *fuzzer* dengan platform otomasi alur kerja populer, guna memastikan pengujian berjalan secara berkesinambungan.

3.11.1. Siklus Umpan Balik Kontinu

Dalam praktik rekayasa perangkat lunak modern, *Continuous Integration* (CI) dan *Continuous Deployment* (CD) telah menjadi standar untuk mengotomatisasi siklus hidup pengembangan. Integrasi pengujian otomatis ke dalam *pipeline* CI/CD sangat krusial untuk memberikan umpan balik (*feedback*) yang cepat bagi pengembang setiap kali ada perubahan kode. Hal ini memungkinkan deteksi dini terhadap *bug* atau regresi fitur sebelum aplikasi masuk ke lingkungan produksi.

3.11.2. Alur Kerja GitHub Actions

GitHub Actions adalah platform otomatisasi alur kerja yang terintegrasi langsung dengan ekosistem GitHub. Sistem yang dikembangkan dalam penelitian ini memanfaatkan GitHub Actions untuk menjalankan sesi *fuzzing* secara otomatis melalui file *workflow* berformat YAML. Integrasi ini memungkinkan proses pengujian dipicu oleh kejadian tertentu (*trigger events*), seperti aktivitas *push* kode atau *pull request* pada repositori pengguna. Melalui penggunaan SDK Octokit, sistem dapat secara terprogram mengelola konfigurasi *workflow* tersebut untuk meminimalisir intervensi manual dari pengembang.

3.12. Evaluasi Kualitas Perangkat Lunak

Untuk memastikan sistem yang dikembangkan memenuhi standar kualitas, diperlukan metode pengukuran yang objektif dan terukur. Subbab ini menguraikan metodologi evaluasi yang digunakan dalam penelitian, mencakup pengujian usability yang melibatkan partisipan serta audit teknis otomatis untuk menilai performa aplikasi web.

3.12.1. *Task-Based Usability Testing*

Task-based usability testing adalah metode evaluasi yang melibatkan partisipan untuk menyelesaikan serangkaian tugas spesifik menggunakan sistem dalam lingkungan yang terkendali untuk mengidentifikasi masalah usability serta mengukur efektivitas dan efisiensi antarmuka. Berbeda dengan kuesioner murni yang hanya menangkap persepsi setelah penggunaan, metode ini memungkinkan peneliti untuk mengamati perilaku pengguna secara langsung, hambatan yang mereka temui, serta pola interaksi manusia-komputer yang terjadi selama proses berlangsung (Adamjak, 2024).

Salah satu teknik inti dalam pengujian berbasis tugas adalah protokol *think-aloud*. Dalam teknik ini, partisipan didorong untuk menyuarakan secara verbal pikiran, perasaan, dan alasan di balik setiap tindakan mereka saat mengerjakan tugas. Metode ini memberikan data kualitatif yang kaya mengenai model mental pengguna dan membantu peneliti memahami mengapa pengguna mengalami kesulitan pada bagian tertentu dari antarmuka.

3.12.2. **Metrik Google Lighthouse**

Google Lighthouse adalah alat audit otomatis bersifat *open-source* yang digunakan untuk meningkatkan kualitas halaman web. Dalam penelitian ini, Lighthouse digunakan untuk mengevaluasi kepatuhan aplikasi web terhadap standar usability dan performa teknis. Fokus evaluasi mencakup metrik-metrik vital seperti:

1. **Performance:** Mengukur kecepatan muat halaman, termasuk indikator *Largest Contentful Paint* (LCP).
2. **Accessibility:** Memastikan antarmuka pengguna dapat diakses dengan baik oleh berbagai kategori pengguna.
3. **Best Practices:** Mengevaluasi kepatuhan kode terhadap standar pengembangan web modern.
4. **SEO:** Menilai optimasi mesin pencari pada aplikasi.

BAB IV

PERANCANGAN PRODUK

4.1. Deskripsi Umum

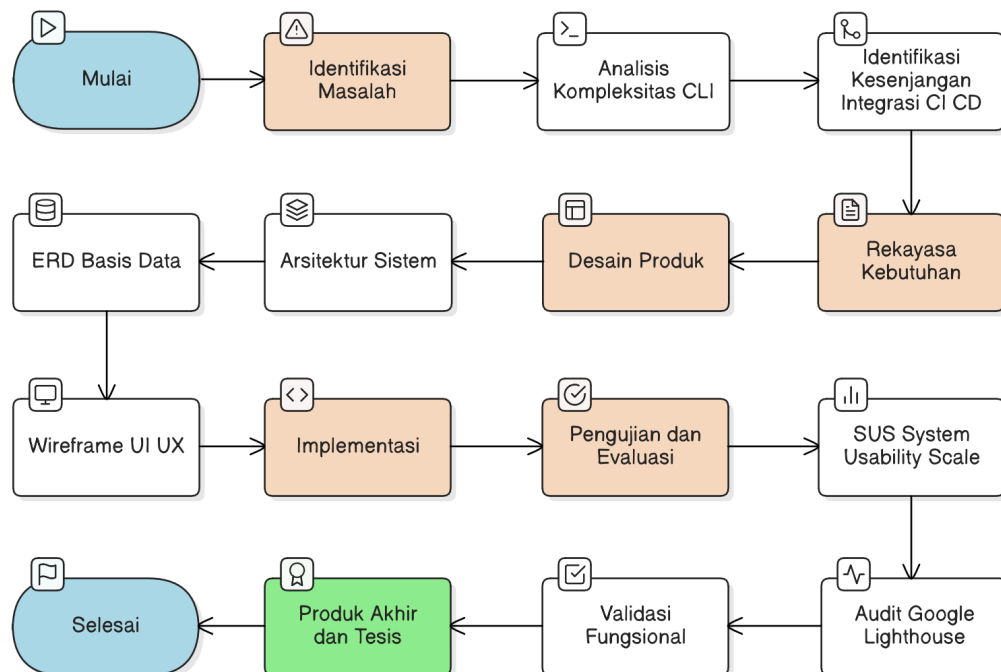
Sistem yang dirancang dalam penelitian ini adalah sebuah platform pengujian kualitas dan keamanan REST API berbasis web yang mengintegrasikan model cerdas *AutoRestTest*. Pengembangan sistem ini bertujuan untuk memfasilitasi penggunaan teknik *fuzzing* tingkat lanjut bagi para praktisi pengembang perangkat lunak dan penguji mutu (*QA Engineer*) tanpa memerlukan pemahaman teknis mendalam mengenai eksekusi baris perintah (*CLI*) yang kompleks.

Sistem ini berfungsi sebagai platform orkestrasi yang menjembatani interaksi manusia dengan mesin pengujian berbasis *Multi-Agent Reinforcement Learning* (MARL) dan *Large Language Models* (LLM). Secara fungsional, sistem memungkinkan pengguna untuk mendefinisikan target pengujian melalui spesifikasi OpenAPI (OAS) dan mengonfigurasi parameter pengujian secara visual. Selain pengujian manual sekali jalan (*one-time test*), sistem juga menyediakan fitur otomatisasi melalui integrasi dengan *pipeline* CI/CD pada platform GitHub.

Secara teknis, sistem mengadopsi arsitektur asinkron untuk menangani beban komputasi yang intensif selama proses *fuzzing* berjalan. Komponen-komponen utama sistem, hubungan antar-layanan, serta siklus pengolahan data dari tahap inisialisasi hingga penyimpanan hasil pengujian secara visual telah dipetakan di dalam **Diagram Arsitektur End-to-End Sistem**. Dengan adanya pemisahan antara antarmuka web, orkestrator tugas (Trigger.dev), dan pembungkus model (FastAPI), sistem ini dirancang untuk memberikan pengalaman pengguna yang responsif sekaligus menjaga reliabilitas eksekusi model di latar belakang. Luaran akhir yang diharapkan dari sistem ini adalah laporan hasil pengujian yang komprehensif, mencakup statistik distribusi kode status respons dan detail temuan kesalahan server yang disajikan melalui dasbor interaktif.

4.2. Alur Pengembangan Produk

Pengembangan sistem *API Fuzzer* berbasis web ini dilakukan dengan menggunakan pendekatan *Research and Development* (R&D). Metodologi ini dipilih karena penelitian berfokus pada pengembangan produk inovatif yang didasarkan pada temuan riset ilmiah sebelumnya, yaitu model *AutoRestTest*, untuk kemudian diimplementasikan ke dalam bentuk platform yang aplikatif. Secara sistematis, tahapan pengembangan produk ini digambarkan melalui **Gambar 4.1** yang terbagi ke dalam beberapa fase utama sebagai berikut.



Gambar 4.1 Diagram alur pengembangan produk

1. **Identifikasi Masalah (*Problem Identification*):** Tahap awal melibatkan analisis mendalam terhadap keterbatasan alat *fuzzing* akademik yang ada. Fokus utama adalah mengidentifikasi hambatan aksesibilitas pada alat berbasis CLI serta kesenjangan integrasi dalam alur kerja industri modern (CI/CD).
2. **Studi Literatur (*Literature Review*):** Melakukan kajian terhadap teori-teori inti yang mendukung sistem, meliputi mekanisme *stateful*

fuzzing menggunakan *Semantic Operation Dependency Graph* (SODG), pengoptimalan eksplorasi dengan *Multi-Agent Reinforcement Learning* (MARL), serta peran *Large Language Models* (LLM) dalam pemrosesan data uji secara semantik.

3. **Rekayasa Kebutuhan (*Requirements Engineering*):** Mendefinisikan kebutuhan fungsional sistem, seperti fitur manajemen pengujian asinkron dan otomasi *setup workflow* GitHub Actions. Selain itu, ditetapkan pula kebutuhan non-fungsional yang mencakup standar usabilitas dan performa aplikasi web.
4. **Perancangan Produk (*Product Design*):** Tahap ini mencakup perancangan arsitektur sistem *end-to-end* yang melibatkan integrasi antara Next.js, FastAPI, dan Trigger.dev. Selain itu, dilakukan perancangan basis data menggunakan skema Prisma dan perancangan antarmuka pengguna (*wireframe*) untuk memastikan alur kerja yang intuitif.
5. **Implementasi (*Implementation*):** Melakukan proses pengodean sistem berdasarkan rancangan yang telah dibuat. Tahap ini mencakup pengembangan *frontend* dengan TypeScript dan Next.js 15, pembangunan *backend* orkestrasi tugas asinkron menggunakan Trigger.dev, serta pembuatan *model wrapper* berbasis FastAPI untuk mengeksekusi *engine* AutoRestTest.
6. **Pengujian dan Evaluasi (*Testing & Evaluation*):** Tahap akhir untuk memvalidasi apakah produk yang dikembangkan telah memenuhi kebutuhan yang ditetapkan. Evaluasi dilakukan melalui validasi fungsional, audit performa dan aksesibilitas menggunakan Google Lighthouse, serta pengukuran tingkat kepuasan pengguna menggunakan kuesioner *System Usability Scale* (SUS).

4.3. Analisis Solusi Terkait dan Posisi Strategis

Untuk menentukan nilai kebaruan dan arah perancangan sistem, dilakukan analisis terhadap beberapa solusi pengujian API yang telah ada. Alat pengujian *state-of-the-art* seperti *RESTler*, *MOREST*, dan *ARAT-RL* memiliki performa yang

sangat baik dalam mendeteksi kesalahan secara algoritmik. Namun, alat-alat tersebut mayoritas dikembangkan sebagai perangkat lunak baris perintah (*Command Line Interface* atau CLI) yang bersifat *standalone*.

Berdasarkan analisis terhadap solusi-solusi tersebut, ditemukan beberapa celah operasional yang menjadi fokus penyelesaian dalam perancangan sistem ini:

1. **Kurangnya Persistensi Data:** Alat pengujian tradisional umumnya hanya menyajikan hasil dalam bentuk file *log* lokal. Sistem yang dirancang menempati posisi strategis dengan menyediakan basis data terpusat (PostgreSQL) untuk menyimpan riwayat pengujian secara permanen.
2. **Hambatan Integrasi Alur Kerja:** Penggunaan alat CLI memerlukan konfigurasi manual pada setiap lingkungan pengujian. Sistem ini mengambil posisi sebagai platform orkestrasi yang mengotomatiskan pembuatan file *workflow* untuk GitHub Actions, sehingga pengujian menjadi bagian integral dari siklus pengembangan.
3. **Beban Komputasi dan Responsivitas:** Menjalankan model AI yang intensif pada server web tradisional sering kali menyebabkan *timeout*. Posisi strategis sistem ini diperkuat dengan penggunaan arsitektur asinkron melalui Trigger.dev dan strategi *caching* Upstash Redis untuk menjamin responsivitas antarmuka meskipun proses pengujian yang berat sedang berjalan di latar belakang.

Melalui pemetaan ini, sistem yang dikembangkan tidak memposisikan diri sebagai kompetitor algoritma *AutoRestTest*, melainkan sebagai **Platform Manajemen Pengujian API Cerdas**. Posisi ini bertujuan untuk mendemokratisasi penggunaan teknologi pengujian AI tingkat lanjut agar lebih mudah diakses, dikelola, dan diintegrasikan ke dalam standar operasional industri perangkat lunak modern.

4.4. Requirements Engineering

Tahap *requirements engineering* dilakukan untuk mendefinisikan batasan dan kemampuan sistem secara spesifik. Penentuan kebutuhan ini didasarkan pada tujuan untuk meningkatkan aksesibilitas model *AutoRestTest* dan menyediakan integrasi alur kerja pengujian yang otomatis.

4.4.1. Functional Requirements

Google Lighthouse adalah alat audit otomatis bersifat *open-source* yang digunakan untuk meningkatkan kualitas halaman web. Dalam penelitian ini, Lighthouse digunakan untuk mengevaluasi kepatuhan aplikasi web terhadap standar usabilitas dan performa teknis. Fokus evaluasi mencakup metrik-metrik vital seperti:

1. **Manajemen Autentikasi:** Sistem harus mampu mengelola sesi pengguna secara aman menggunakan Clerk dan mendukung integrasi OAuth dengan GitHub untuk keperluan akses repositori.
2. **Definisi Target Pengujian:** Sistem harus menyediakan fitur pengunggahan spesifikasi OpenAPI (OAS) dalam format JSON guna membangun graf dependensi operasi (SODG).
3. **Konfigurasi Parameter Cerdas:** Sistem harus memungkinkan pengguna mengatur parameter model AI melalui antarmuka visual, meliputi pilihan *engine* LLM (GPT-4/GPT-3.5), tingkat *temperature*, serta parameter agen RL seperti *learning rate*, *discount factor*, dan *max exploration*.
4. **Eksekusi Pengujian Asinkron:** Sistem harus mampu menjalankan proses *fuzzing* yang intensif di latar belakang tanpa memblokir antarmuka pengguna, dengan memanfaatkan orkestrasi tugas dari Trigger.dev.
5. **Pemantauan Status Real-time:** Sistem harus menyediakan dasbor untuk memantau kemajuan pengujian, termasuk persentase progres, operasi yang sedang berjalan, dan riwayat pekerjaan pengujian yang tersimpan di PostgreSQL.

6. **Visualisasi Hasil Pengujian:** Sistem harus menyajikan ringkasan hasil melalui grafik interaktif, meliputi distribusi kode status respons dan detail teknis mengenai kesalahan server (5xx) yang ditemukan.
7. **Otomasi Integrasi CI/CD:** Sistem harus mampu secara otomatis menyisipkan berkas *workflow* GitHub Actions ke dalam repositori pengguna untuk menjalankan sesi *fuzzing* proaktif pada setiap aktivitas *push* kode.
8. **Mekanisme Caching:** Sistem harus mengimplementasikan penyimpanan sementara menggunakan Upstash Redis untuk menyimpan keluaran model guna mempercepat eksekusi pengujian berulang pada target yang sama.

4.4.2. *Non-Functional Requirements*

Kebutuhan non-fungsional menetapkan kriteria kualitas dan batasan operasional sistem untuk menjamin performa dan pengalaman pengguna yang optimal. Kriteria tersebut meliputi:

1. **Usabilitas (*Usability*):** Sistem harus mencapai skor persepsi kegunaan yang baik berdasarkan evaluasi kuesioner *System Usability Scale* (SUS) dari perspektif pengembang dan penguji perangkat lunak.
2. **Performa Teknis:** Aplikasi web harus memenuhi standar optimasi industri yang diukur melalui audit Google Lighthouse, dengan target skor tinggi pada metrik *Performance*, *Accessibility*, dan *Best Practices*.
3. **Reliabilitas Tugas:** Sistem harus mampu menangani tugas pengujian yang memakan waktu lama (hingga 3600 detik) tanpa mengalami kegagalan koneksi (*timeout*), yang dicapai melalui pemisahan beban kerja antara *web server* dan *task runner*.
4. **Keamanan API:** Akses terhadap titik akhir eksekusi model pada *backend* FastAPI harus dilindungi menggunakan skema autentikasi API Key guna mencegah penggunaan yang tidak sah.

5. **Skalabilitas Data:** Basis data harus mampu menangani penyimpanan log hasil pengujian yang kompleks dalam format JSONB untuk mendukung fleksibilitas pelaporan.

4.5. Perancangan Desain Antarmuka Pengguna (UI/UX)

Perancangan antarmuka pengguna bertujuan untuk menciptakan pengalaman yang intuitif bagi praktisi pengembang dalam mengoperasikan model pengujian yang kompleks. Fokus utama desain adalah pada kemudahan konfigurasi parameter serta kejelasan penyajian hasil pengujian.

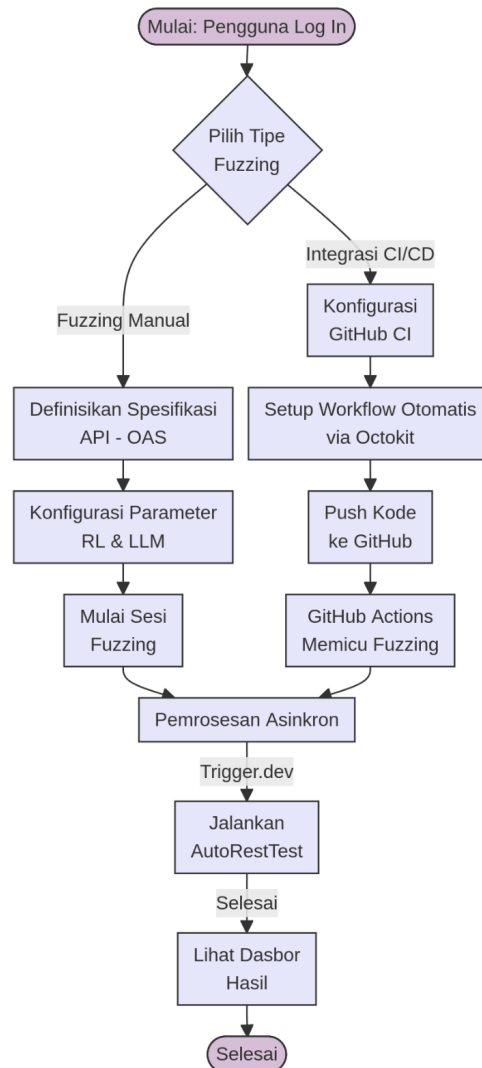
4.5.1. Pembentukan Persona Arketipe

Untuk memastikan desain sistem tepat sasaran, didefinisikan dua persona arketipe utama sebagai representasi pengguna target:

1. **Backend Developer:** Membutuhkan alat yang dapat diintegrasikan ke dalam repositori GitHub guna memastikan kualitas API secara otomatis pada setiap pembaruan kode melalui fitur CI/CD.
2. **Quality Assurance (QA) Engineer:** Membutuhkan fleksibilitas untuk melakukan pengujian mendalam secara manual (*one-time test*) dengan kebebasan mengatur parameter RL guna mengeksplorasi skenario kegagalan API yang spesifik.

4.5.2. Pembentukan *User Flow*

Alur interaksi pengguna dirancang sedemikian rupa untuk mengakomodasi dua skenario utama sistem. Detail langkah-langkah teknis dan percabangan keputusan pengguna dalam sistem ini telah dipetakan di dalam **Gambar 4.2**.



Gambar 4.2 Diagram *user flow* (flowchart)

Secara garis besar, terdapat dua alur utama:

Skenario 1: Pengguna Melakukan API Fuzzing Manual Sekali Jalan

Alur ini ditujukan bagi pengguna yang ingin melakukan pengujian *fuzzing* secara langsung dan interaktif pada target API tertentu.

1. **Mulai (Pengguna Telah Log In):** Pengguna diasumsikan telah memiliki akun dan berhasil masuk (log in) ke dalam aplikasi web *fuzzer*. Titik awal adalah halaman utama atau dasbor aplikasi.

2. **Memilih Jenis Fuzzing (Manual):** Dari halaman utama atau menu navigasi, pengguna memilih opsi untuk melakukan "Manual Fuzzing" atau *fuzzing* sekali jalan. Pilihan ini mengarahkan pengguna ke alur konfigurasi pengujian yang lebih interaktif.
3. **Definisi Spesifikasi API Target:** Pengguna akan diarahkan ke antarmuka untuk mendefinisikan REST API yang akan diuji. Hal ini dapat dilakukan dengan cara mengunggah file spesifikasi API (format OpenAPI Specification/OAS) atau dengan memasukkan URL *endpoint* dari spesifikasi API tersebut. Sistem akan mem-parsing spesifikasi ini untuk memahami struktur API.
4. **Konfigurasi Parameter Fuzzing:** Setelah API target terdefinisi, pengguna mengonfigurasi parameter-parameter untuk sesi *fuzzing*. Ini mencakup pengaturan umum seperti durasi pengujian, intensitas, serta parameter spesifik yang terkait dengan mekanisme AutoRestTest (misalnya, pengaturan terkait SODG atau perilaku agen MARL jika diekspos ke pengguna).
5. **Memulai Sesi Fuzzing:** Dengan semua konfigurasi telah diatur, pengguna menekan tombol untuk memulai proses *fuzzing*. Sistem akan menginisialisasi *engine fuzzer* berdasarkan parameter yang diberikan dan mulai mengirimkan sekuens permintaan ke API target.
6. **Proses Fuzzing dan Pengalihan ke Dasbor Hasil:** Selama proses *fuzzing* berjalan, pengguna dapat memantau progres secara *real-time* (tergantung desain antarmuka). Setelah sesi *fuzzing* selesai (baik karena durasi tercapai, target tertentu terpenuhi, atau dihentikan manual), sistem akan secara otomatis mengalihkan pengguna ke Dasbor Hasil (*Results Dashboard*).
7. **Dasbor Hasil:** Pada dasbor ini, pengguna dapat melihat laporan detail dari sesi *fuzzing* yang baru saja selesai, mencakup statistik pengujian, daftar *error* atau anomali yang ditemukan, sekuens permintaan yang memicu masalah, dan informasi relevan lainnya.

8. **Selesai:** Pengguna telah berhasil melakukan sesi *fuzzing* manual dan menganalisis hasilnya.

Skenario 2: Pengguna Mengintegrasikan API Fuzzing dengan CI/CD Pipeline

Alur ini ditujukan bagi pengguna yang ingin mengotomatisasi proses *fuzzing* API sebagai bagian dari alur kerja CI/CD mereka, sehingga pengujian dapat dilakukan secara berkala atau setiap kali ada perubahan pada kode API.

1. **Mulai (Pengguna Telah Log In):** Sama seperti skenario pertama, pengguna telah log in ke aplikasi.
2. **Memilih Jenis Fuzzing (Integrasi CI/CD):** Dari halaman utama atau menu navigasi, pengguna memilih opsi untuk "CI/CD Integrated Fuzzing" atau pengaturan integrasi.
3. **Definisi Spesifikasi API Target:** Mirip dengan skenario manual, pengguna mendefinisikan REST API yang akan diuji secara otomatis oleh *pipeline* CI/CD. Pengaturan ini disimpan sebagai konfigurasi yang dapat dipicu dari luar.
4. **Konfigurasi Parameter Fuzzing (untuk CI/CD):** Pengguna mengatur parameter *fuzzing* yang akan digunakan ketika sesi dipicu oleh CI/CD. Parameter ini berbeda dari *fuzzing* manual (misalnya, durasi lebih singkat untuk *feedback* cepat). Pengguna juga akan mendapatkan detail teknis (misalnya, API key, *webhook URL*, atau perintah CLI) yang diperlukan untuk memicu *fuzzing* dari *pipeline* CI/CD.
5. **Menambahkan Job ke CI/CD Pipeline:** Pengguna beralih ke sistem CI/CD mereka. Menggunakan informasi yang didapatkan dari langkah sebelumnya, sistem akan menambahkan *job* atau *stage* baru dalam *pipeline* pengguna yang bertugas memanggil atau memicu aplikasi web *fuzzer*. *Job* ini biasanya dieksekusi setelah tahap *build* dan *deploy* ke lingkungan pengujian (*staging*).
6. **CI/CD Pipeline Memicu Fuzzing:** Ketika *pipeline* CI/CD berjalan (misalnya, setelah ada *commit code* baru), *job* yang telah dikonfigurasi

akan secara otomatis memanggil aplikasi web *fuzzer* untuk memulai sesi pengujian pada API target dengan parameter yang telah ditentukan.

7. **Proses Fuzzing Berjalan (di Sisi Aplikasi Web Fuzzer):** Aplikasi web *fuzzer* menerima pemicu, melakukan autentikasi, dan menjalankan sesi *fuzzing* secara otomatis di latar belakang.
8. **Penyediaan Tautan Hasil pada Eksekusi Job CI/CD:** Setelah sesi *fuzzing* otomatis selesai, aplikasi web *fuzzer* akan menghasilkan laporan. Informasi mengenai hasil ini, idealnya berupa tautan langsung (*link*) ke Dasbor Hasil yang relevan di dalam aplikasi web *fuzzer*, akan dikomunikasikan kembali atau dicatat dalam log eksekusi *job* CI/CD.
9. **Pengguna Mengakses Hasil Melalui Tautan:** Pengguna dapat memantau hasil *build* CI/CD mereka. Jika *job fuzzing* telah selesai, pengguna dapat mengklik tautan yang disediakan dalam log atau output *job* CI/CD untuk langsung diarahkan ke Dasbor Hasil spesifik di aplikasi web *fuzzer* dan menganalisis temuan.
10. **Selesai:** Pengguna telah berhasil mengintegrasikan *fuzzing* API ke dalam *pipeline* CI/CD dan dapat mengakses hasilnya secara terpusat.

4.5.3. Arsitektur Informasi dan Tata Letak Antarmuka

Struktur informasi sistem dibagi menjadi tiga bagian utama yang dirancang untuk mengurangi beban kognitif pengguna:

1. **Formulir Konfigurasi Intuitif (TestForm):** Menggunakan elemen kontrol visual seperti *slider* untuk parameter numerik (misalnya *Learning Rate* dan *Temperature*) serta *radio group* untuk pemilihan jenis pengujian. Hal ini bertujuan menggantikan kompleksitas penulisan berkas konfigurasi teks manual.
2. **Manajemen Riwayat Pekerjaan (JobHistory):** Menyajikan status pengujian dalam bentuk tabel yang interaktif, memungkinkan pengguna untuk melakukan pemantauan terhadap banyak sesi pengujian sekaligus.

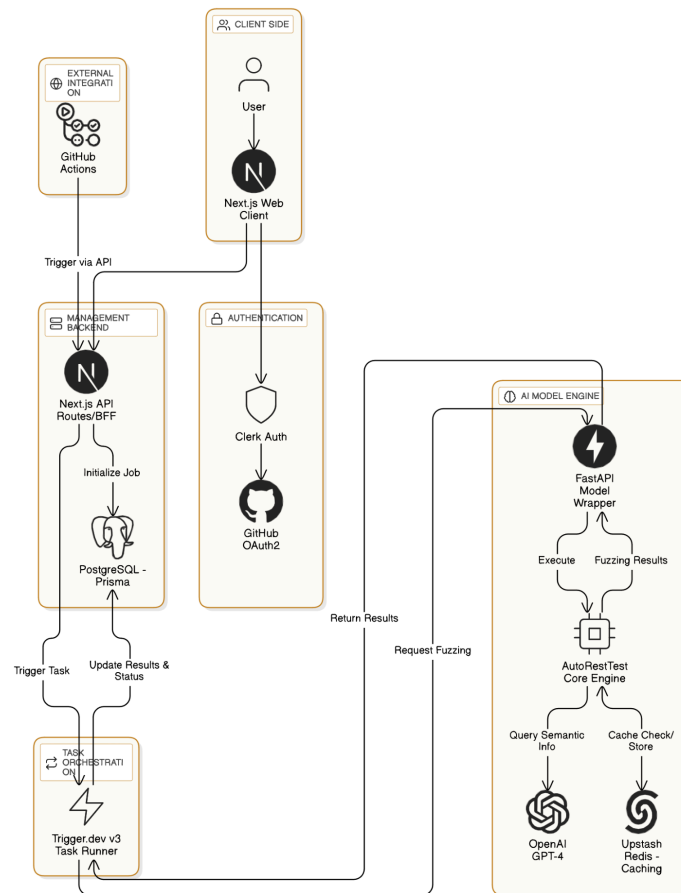
3. **Dasbor Hasil Interaktif (JobDetailsDataExplorer):** Hasil pengujian yang kompleks dari model *AutoRestTest* diubah menjadi representasi visual menggunakan grafik distribusi kode status respons dan tampilan detail kesalahan server yang dapat dieksplorasi secara hierarkis.

4.6. Perancangan Arsitektur Sistem

Perancangan arsitektur sistem ini difokuskan pada pemisahan beban kerja (*separation of concerns*) untuk memastikan platform tetap responsif saat menjalankan model pengujian AI yang intensif. Arsitektur ini mengintegrasikan berbagai layanan terkelola dan *framework* modern guna mencapai reliabilitas tinggi.

4.6.1. Arsitektur Informasi dan Tata Letak Antarmuka

Secara keseluruhan, topologi sistem dibagi menjadi empat lapisan utama yang saling terhubung melalui protokol HTTP/REST dan akses basis data terpusat. Detail visual dari struktur ini dapat dilihat pada **Gambar 4.3**. Adapun rincian komponennya adalah sebagai berikut:



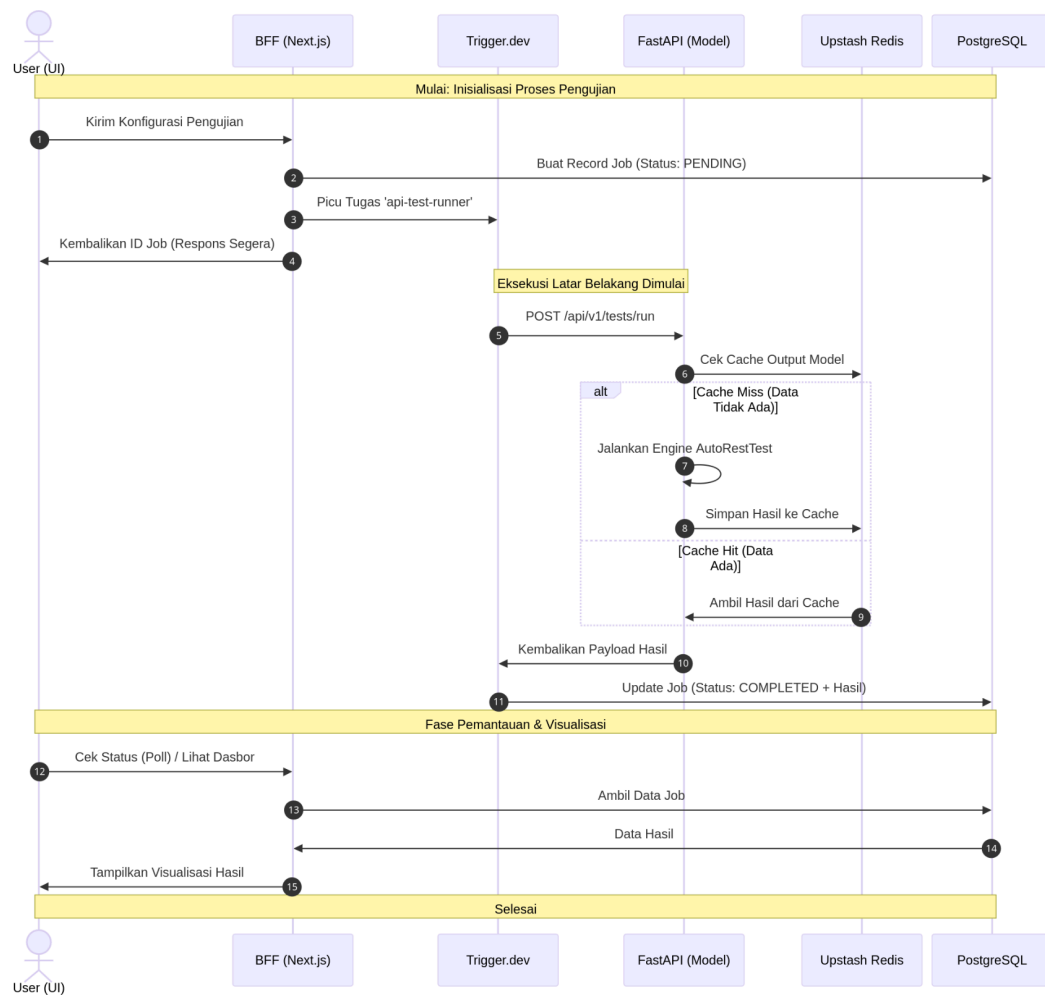
Gambar 4.3 Diagram arsitektur end-to-end sistem

1. **Lapisan Klien (*Client Side*):** Menggunakan Next.js sebagai *web client* yang menangani interaksi pengguna dan visualisasi data hasil pengujian.
2. **Lapisan Manajemen (*Management Backend*):** Bertindak sebagai *Backend-for-Frontend* (BFF) yang mengelola autentikasi pengguna melalui Clerk, manajemen basis data melalui Prisma ORM, serta penyediaan API internal untuk komunikasi dengan orkestrator tugas.
3. **Lapisan Orkestrasi Tugas (*Task Orchestration*):** Menggunakan Trigger.dev v3 sebagai mesin pengelola antrean tugas asinkron. Komponen ini bertanggung jawab untuk memicu eksekusi model di latar belakang, menangani pengulangan otomatis jika terjadi kegagalan (*retries*), dan memperbarui status pekerjaan di basis data.

4. **Lapisan Mesin AI (AI Model Engine):** Terdiri dari FastAPI sebagai pembungkus model yang mengekspos fungsi pengujian melalui HTTP. Di dalamnya terdapat *AutoRestTest Core Engine* yang menjalankan logika pengujian cerdas dengan dukungan Upstash Redis untuk strategi *caching* dan OpenAI API untuk kebutuhan pemrosesan semantik.

4.6.2. Alur Data dan Orkestrasi Asinkron

Mekanisme pertukaran data dalam sistem dirancang untuk bersifat non-blokir (*non-blocking*), di mana pengguna tidak perlu menunggu proses pengujian selesai untuk dapat terus berinteraksi dengan aplikasi. Urutan proses ini secara mendetail dipetakan dalam **Gambar 4.4**.

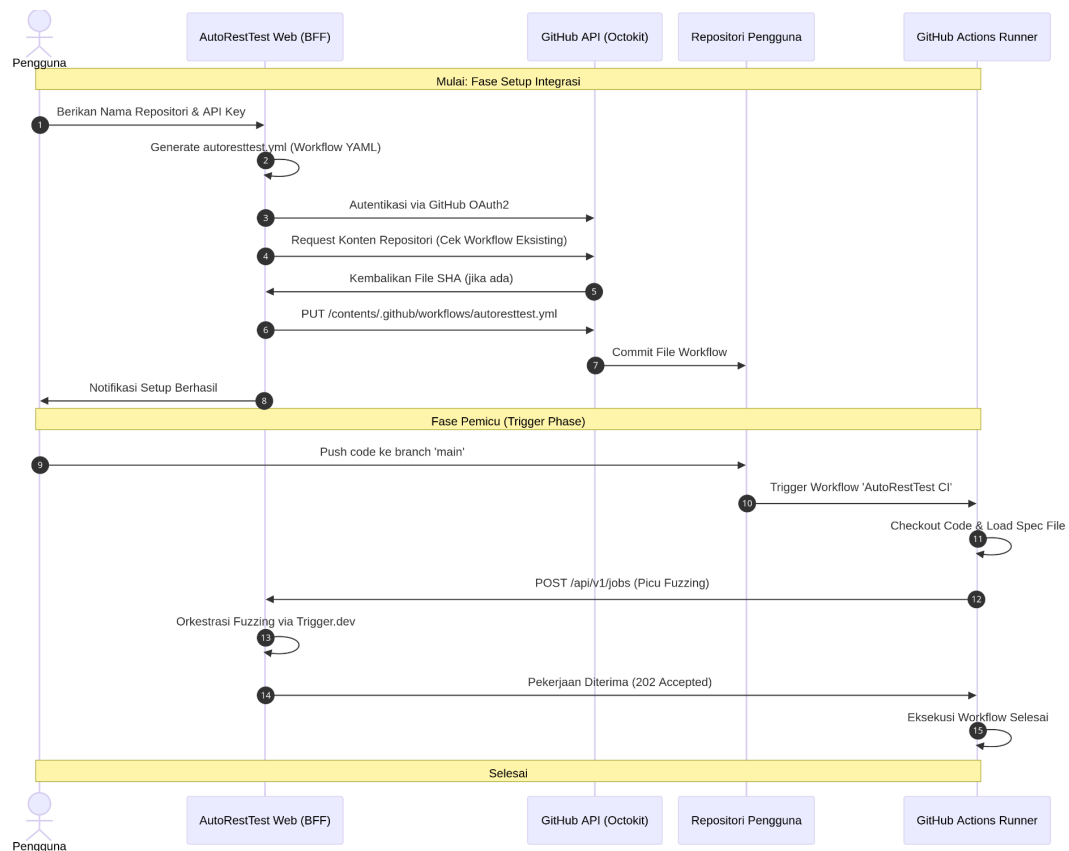


Gambar 4.4 Diagram *fuzzing task orchestration sequence*

Alur kerja dimulai ketika pengguna mengirimkan konfigurasi pengujian melalui antarmuka web. BFF Next.js akan segera membuat catatan pekerjaan baru di PostgreSQL dengan status awal **queued** dan memicu tugas pada Trigger.dev. Setelah pemicuan berhasil, sistem langsung mengembalikan ID pekerjaan kepada pengguna sehingga halaman web tetap responsif.

Di latar belakang, *task runner* Trigger.dev mengirimkan permintaan eksekusi ke *backend* FastAPI. Sebelum menjalankan model `AutoRestTest` yang memakan waktu lama, sistem akan memeriksa keberadaan data hasil di *cache* Upstash Redis untuk mempercepat respons jika pengujian serupa pernah dilakukan sebelumnya. Setelah proses *fuzzing* selesai, hasil akhir dikembalikan ke Trigger.dev yang kemudian bertugas memperbarui data ringkasan hasil (**summary**) dan mengubah status pekerjaan menjadi **completed** di basis data. Pengguna dapat melihat hasil akhir secara *real-time* melalui dasbor yang mengambil data terbaru dari PostgreSQL.

Selain alur pengujian manual, sistem juga menyediakan mekanisme integrasi otomatis yang menghubungkan platform dengan ekosistem GitHub. Proses ini dimulai dengan otomasi penyisipan berkas *workflow* ke repositori pengguna menggunakan API GitHub (Octokit). Secara teknis, interaksi antara sistem *web*, repositori pengguna, dan *runner* GitHub Actions dalam menjalankan sesi *fuzzing* proaktif dijelaskan secara mendetail melalui **Gambar 4.5**. Diagram tersebut memperlihatkan bagaimana sebuah aktivitas *push* kode dapat memicu *runner* untuk melakukan pemanggilan balik (*callback*) ke sistem guna memulai proses orkestrasi pengujian secara asinkron.



Gambar 4.5 Logika integrasi CI/CD

Pada fase **setup**, proses dimulai ketika pengguna memasukkan kredensial dan nama repositori target melalui antarmuka web. *Backend* (BFF) kemudian memanfaatkan pustaka Octokit untuk menjalin komunikasi dengan GitHub API. Sistem secara otomatis menyusun dan melakukan *commit* berkas konfigurasi *workflow* (`autoresttest.yml`) langsung ke dalam repositori pengguna, sehingga meniadakan kebutuhan konfigurasi manual yang rentan kesalahan.

Pada fase **trigger**, mekanisme berjalan secara reaktif. Setiap kali terdeteksi aktivitas *push* kode ke cabang utama (*main branch*), *runner* GitHub Actions akan mengeksekusi instruksi yang tertanam dalam berkas *workflow*. *Runner* tersebut bertindak sebagai klien yang mengirimkan permintaan HTTP POST ke *endpoint* sistem AutoRestTest. Setelah sistem memvalidasi permintaan dan menjadwalkan tugas ke orkestrator (Trigger.dev), sistem segera mengembalikan respons status

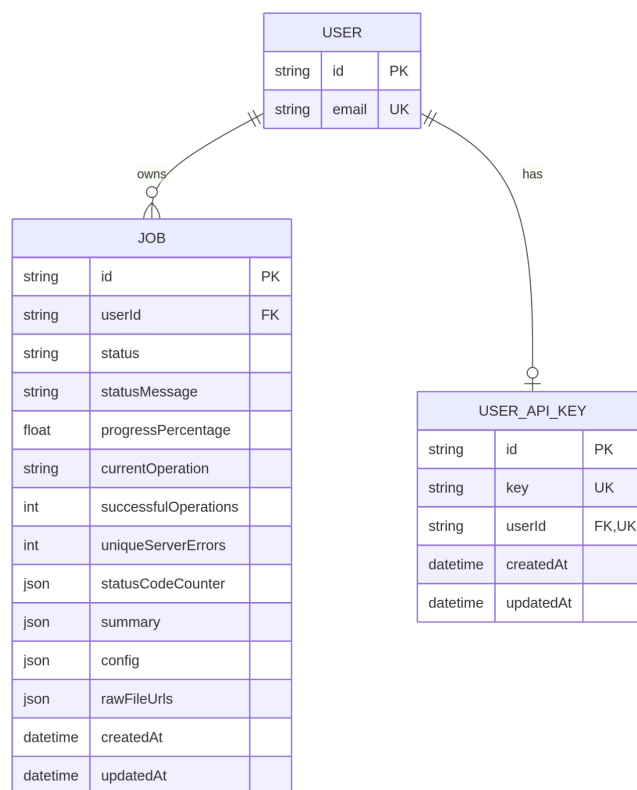
202 Accepted. Mekanisme *fire-and-forget* ini memastikan bahwa *pipeline* CI/CD pengguna tidak terblokir menunggu proses *fuzzing* yang lama, sementara pengujian berjalan secara asinkron di sisi server.

4.7. Perancangan Basis Data

Perancangan basis data bertujuan untuk menyediakan struktur penyimpanan yang efisien guna mengelola informasi pengguna, konfigurasi pengujian, serta riwayat hasil *fuzzing*. Sistem ini menggunakan PostgreSQL sebagai basis data relasional utama yang dikelola melalui abstraksi Prisma ORM.

4.7.1. Entity Relationship Diagram (ERD)

Hubungan antar entitas dalam sistem dirancang untuk menjamin integritas data dan kemudahan dalam pelacakan riwayat pekerjaan (*job history*). Struktur relasi antar tabel secara visual dipetakan di dalam **Gambar 4.6**.



Gambar 4.6 Diagram hubungan entitas (ERD)

Struktur basis data dirancang dengan memusatkan **User** sebagai entitas utama yang menyimpan identitas pengguna yang telah terautentikasi melalui layanan Clerk. Entitas ini memiliki hubungan *one-to-one* dengan **UserApiKey**, yang berfungsi menyimpan kredensial akses unik untuk setiap pengguna. Hubungan ini bersifat eksklusif, di mana satu pengguna hanya memiliki satu kunci API yang digunakan khusus untuk keperluan otorisasi pada skenario integrasi CI/CD. Selain itu, entitas User memiliki hubungan *one-to-many* terhadap entitas **Job**, yang berarti seorang pengguna dapat memiliki dan mengelola banyak catatan pekerjaan pengujian sekaligus. Entitas Job ini berperan penting sebagai penyimpan metadata lengkap dari setiap sesi pengujian API yang dijalankan, mencakup konfigurasi, status eksekusi, hingga hasil akhirnya.

4.7.2. Definisi Skema Data

Berdasarkan implementasi pada aplikasi, rincian dari setiap entitas basis data adalah sebagai berikut:

1. **Tabel User:** Bertindak sebagai entitas induk yang menyimpan **id** pengguna dan alamat **email** yang unik sebagai pengenalan utama dalam sistem.
2. **Tabel UserApiKey:** Digunakan untuk mendukung fitur integrasi eksternal. Tabel ini menyimpan string **key** yang dienkripsi atau bersifat rahasia, serta dihubungkan ke **userId** dengan batasan penghapusan bertingkat (*cascade delete*) guna menjaga konsistensi data saat akun pengguna dihapus.
3. **Tabel Job:** Merupakan entitas yang paling kompleks karena menyimpan seluruh konteks pengujian. Tabel ini memiliki atribut teknis untuk pemantauan *real-time* seperti **status**, **progressPercentage**, dan **currentOperation**. Selain itu, tabel ini memanfaatkan tipe data **Json** untuk atribut **config** (menyimpan parameter model), **summary** (statistik

hasil akhir), dan `rawFileUrls` (tautan unduhan log mentah) guna memberikan fleksibilitas tinggi dalam penyimpanan luaran dari model *AutoRestTest*. Setiap rekaman data juga dilengkapi dengan penanda waktu `createdAt` dan `updatedAt` untuk keperluan audit dan pengurutan pada antarmuka pengguna.

4.8. Strategi Evaluasi dan Validasi

Tahap evaluasi dan validasi merupakan langkah kritis untuk memastikan bahwa sistem yang telah dirancang dan diimplementasikan berfungsi sesuai dengan kebutuhan pengguna serta memiliki kualitas teknis yang memadai. Sebagaimana yang dipetakan pada fase ke-6 dalam **Diagram Metodologi Pengembangan Produk (Flowchart)**, strategi evaluasi ini dibagi ke dalam tiga kategori utama.

4.8.1. Pengujian Fungsional

Pengujian fungsional bertujuan untuk memvalidasi bahwa setiap fitur yang didefinisikan dalam rekayasa kebutuhan (*Functional Requirements*) berjalan dengan benar. Strategi yang digunakan meliputi:

1. **Unit Testing:** Melakukan pengujian pada tingkat kode program untuk memvalidasi fungsi-fungsi spesifik, seperti logika validasi skema OpenAPI, pemrosesan parameter *Reinforcement Learning*, dan fungsi pembuatan kunci API agar berjalan benar secara terisolasi.
2. **Integration Testing:** Memastikan kelancaran pertukaran data antara Next.js, Trigger.dev, dan FastAPI, terutama pada mekanisme *callback* dan pembaruan status pekerjaan di basis data.

4.8.2. Pengujian Non-Fungsional

Pengujian non-fungsional dilakukan untuk mengukur kualitas teknis sistem dari sisi performa dan kepatuhan terhadap standar pengembangan web modern.

1. **Performance Testing (Postman):** Dilakukan untuk mengukur responsivitas dan stabilitas titik akhir (*endpoint*) API pada sistem, baik di

sisi Next.js maupun FastAPI. Pengujian ini menggunakan Postman untuk memantau metrik seperti *response time* (latensi), *throughput*, dan *success rate* saat sistem menangani berbagai permintaan eksekusi pengujian.

2. **Quality Audit (Google Lighthouse):** Digunakan untuk memberikan penilaian objektif terhadap aspek *Performance*, *Accessibility*, *Best Practices*, dan *SEO* pada antarmuka web sistem.

4.8.3. Pengujian Usabilitas (*Task-Based Usability Testing*)

Evaluasi usabilitas dilakukan secara daring melalui platform konferensi video untuk mengobservasi interaksi pengguna secara langsung. Metodologi yang digunakan adalah *task-based usability testing* dengan menerapkan teknik *think-aloud*, di mana partisipan didorong untuk menyuarakan pikiran mereka saat menyelesaikan tugas.

Karakteristik pelaksanaan pengujian ini adalah sebagai berikut:

1. **Pemilihan Partisipan:** Melibatkan 5-10 partisipan yang terdiri dari praktisi industri (*Software Developer*, *QA Engineer*) dan akademisi (mahasiswa tingkat akhir) yang memiliki pemahaman mengenai REST API.
2. **Prosedur Sesi:** Setiap sesi diawali dengan impresi awal, diikuti oleh pengerjaan tugas utama (fuzzing manual dan integrasi CI/CD), dan diakhiri dengan pengisian kuesioner pasca-sesi.
3. **Skenario Pengujian:** Partisipan diminta menyelesaikan 8 tugas spesifik yang mencakup seluruh fungsionalitas utama sistem.

Rincian instruksi tugas dan pertanyaan pancingan (*probing questions*) yang digunakan dalam sesi ini dapat dilihat pada **Lampiran 1: Skenario Usability Testing**.

4.8.4. Kuesioner Persepsi Subjektif

Setelah menyelesaikan tugas observasi, partisipan mengisi kuesioner pasca-sesi melalui Google Form untuk menangkap data demografis, penilaian kuantitatif menggunakan Skala Likert (1-5), serta umpan balik kualitatif terbuka. Kuesioner ini dirancang untuk memvalidasi kemudahan penggunaan, kejelasan fungsi, dan kualitas penyajian informasi hasil pengujian. Daftar pertanyaan lengkap dan struktur kuesioner tersedia pada **Lampiran 2: Detail Kuesioner Pasca Usability Testing**.

4.8.5. Matriks Penilaian dan Mekanisme Penskalaan

Untuk menghasilkan evaluasi yang komprehensif, hasil observasi (data perilaku) dan kuesioner (data persepsi) diintegrasikan ke dalam sebuah matriks penilaian terstruktur. Data observasional yang memiliki skala keberhasilan 0-2 akan dinormalisasi ke skala 1-5 agar konsisten dengan skala kuesioner.

Penilaian akhir dihitung berdasarkan empat komponen utama dengan pembobotan tertentu:

1. **Skor Kualitas (40%):** Kemampuan aplikasi membantu pengguna memahami hasil pengujian.
2. **Skor Usability (30%):** Efektivitas sistem dalam menyelesaikan alur kerja utama.
3. **Skor Pengalaman (20%):** Kesan pertama dan kemudahan menemukan fitur.
4. **Skor Potensi Adopsi (10%):** Keinginan pengguna untuk menggunakan sistem kembali di masa depan.

4.8.6. Audit Kualitas Web (Google Lighthouse)

Validasi kualitas teknis dilakukan secara otomatis menggunakan alat audit **Google Lighthouse**. Evaluasi ini dilakukan untuk memberikan penilaian objektif terhadap aspek-aspek berikut:

1. **Performance:** Mengukur metrik kecepatan muat halaman dan responsivitas antarmuka.
2. **Accessibility:** Memastikan elemen UI dapat diakses dengan baik oleh pengguna dengan berbagai kebutuhan.
3. **Best Practices:** Memastikan kode program mengikuti standar keamanan dan pengembangan web modern.
4. **SEO:** Mengukur tingkat optimasi mesin pencari pada platform.

Hasil dari seluruh rangkaian evaluasi ini akan menjadi dasar utama dalam pembahasan pada bab selanjutnya untuk menentukan keberhasilan pengembangan sistem.

1. **Skor Kualitas (40%):** Kemampuan aplikasi membantu pengguna memahami hasil pengujian.
2. **Skor Usability (30%):** Efektivitas sistem dalam menyelesaikan alur kerja utama.
3. **Skor Pengalaman (20%):** Kesan pertama dan kemudahan menemukan fitur.
4. **Skor Potensi Adopsi (10%):** Keinginan pengguna untuk menggunakan sistem kembali di masa depan.

Skor akhir kuantitatif untuk setiap partisipan dihitung dengan rumus berikut:

$$\text{Skor Akhir} = (\text{avg}(\text{Skor Kualitas}) \times 0.40) + (\text{avg}(\text{Skor Usability}) \times 0.30) + (\text{avg}(\text{Skor Pengalaman}) \times 0.20) + (\text{Skor Potensi Adopsi} \times 0.10)$$

Detail matriks penilaian dan kriteria normalisasi skor dijelaskan secara mendalam pada **Lampiran 3: Matriks Penilaian Usability Testing**.

BAB V

PROSES PEMBUATAN PRODUK

5.1. Lingkungan Implementasi

Tahap implementasi dilakukan dengan mengintegrasikan berbagai komponen teknologi untuk memastikan sistem berjalan sesuai dengan rancangan asinkron yang telah ditetapkan. Lingkungan implementasi mencakup spesifikasi infrastruktur pengembangan dan platform *cloud* yang digunakan untuk menjamin stabilitas prototipe saat diakses oleh pengguna.

5.1.1. Spesifikasi Perangkat Keras

Pengembangan dan pengujian lokal sistem dilakukan menggunakan perangkat keras dengan spesifikasi sebagai berikut:

- **Model Laptop:** Asus TUF Gaming A15
- **Memori (RAM):** 16 GB LPDDR5.
- **Penyimpanan:** 512 GB SSD.
- **Perangkat Tambahan:** Mikrofon dan kamera web yang digunakan selama sesi evaluasi daring bersama partisipan.

5.1.2. Spesifikasi Perangkat Lunak

Implementasi sistem memanfaatkan tumpukan teknologi modern yang mendukung pengembangan aplikasi web asinkron dan integrasi model *machine learning*. Berikut adalah rincian perangkat lunak yang digunakan:

Bahasa Pemrograman & Framework Utama:

- **TypeScript & Next.js 15:** Digunakan untuk membangun antarmuka pengguna dan *Backend-for-Frontend* (BFF).
- **Python & FastAPI:** Digunakan untuk membangun *wrapper* model yang mengeksekusi *engine* AutoRestTest.

Layanan Cloud & Infrastruktur:

- **Dewacloud:** Platform untuk melakukan *deployment* untuk layanan *frontend* Next.js dan layanan *backend* FastAPI.
- **Trigger.dev v3:** Infrastruktur orkestrasi tugas asinkron untuk mengelola *long-running jobs*.
- **Upstash Redis:** Layanan *database in-memory* untuk mekanisme *caching* hasil model.
- **PostgreSQL:** Basis data relasional utama untuk persistensi data pekerjaan dan pengguna.

Layanan Pihak Ketiga & Alat Evaluasi:

- **Clerk:** Penyedia layanan autentikasi dan manajemen pengguna.
- **OpenAI API (GPT-4):** Digunakan sebagai mesin LLM untuk pemrosesan semantik data pengujian.
- **GitHub API (Octokit):** Digunakan untuk otomasi integrasi CI/CD pada repositori pengguna.
- **Google Meet:** Platform konferensi video untuk pelaksanaan *usability testing* dan perekaman sesi.
- **Google Forms:** Alat pengumpulan data kuesioner pasca-sesi dari partisipan.
- **Google Drive (Afiliasi UGM):** Media penyimpanan aman untuk data mentah, rekaman video, dan transkrip penelitian.

5.2. Implementasi Basis Data

Implementasi basis data pada sistem ini menggunakan PostgreSQL sebagai sistem manajemen basis data relasional. Untuk menjembatani antara logika aplikasi pada kode program dengan struktur tabel di basis data, digunakan Prisma ORM (*Object-Relational Mapping*). Penggunaan Prisma memungkinkan pendefinisian skema data yang bersifat *type-safe* dan mempermudah proses migrasi basis data.

5.2.1. Skema Data Prisma

Skema basis data didefinisikan dalam berkas `schema.prisma`. Definisi ini mencakup model untuk pengguna, kunci API, dan rincian pekerjaan pengujian (*job*). Berikut adalah potongan kode skema yang diimplementasikan:

```
TypeScript
model Job {
  id          String    @id @default(cuid())
  userId      String    @map("user_id")
  user        User      @relation(fields: [userId],
references: [id])
  status      String    @default("queued")
  statusMessage String? @map("status_message")
  progressPercentage Float? @map("progress_percentage")
  currentOperation String? @map("current_operation")
  successfulOperations Int? @map("successful_operations")
  uniqueServerErrors Int? @map("unique_server_errors")
  statusCodeCounter Json? @map("status_code_counter")
  summary     Json?
  config      Json?
  rawFileUrls Json? @map("raw_file_urls")
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model User {
  id      String @id
  email   String @unique
  jobs    Job[]
  apiKey  UserApiKey?
}

model UserApiKey {
  id      String    @id @default(cuid())
  key     String    @unique
  userId  String    @unique
  user    User      @relation(fields: [userId], references: [id],
onDelete: Cascade)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Kode 5.1 Definisi skema database dengan menggunakan Prisma ORM

5.2.2. Detail Struktur Tabel dan Relasi

Berdasarkan implementasi skema di atas, terdapat beberapa poin utama dalam struktur data sistem:

1. **Entitas Job:** Tabel ini dirancang untuk menampung data yang dinamis. Kolom `status`, `progressPercentage`, dan `currentOperation` diperbarui secara berkala oleh *worker* asinkron selama pengujian berjalan. Penggunaan tipe data `Json` pada kolom `summary`, `config`, dan `statusCodeCounter` bertujuan untuk menyimpan objek data kompleks yang dihasilkan oleh *engine* `AutoRestTest` tanpa harus mendefinisikan kolom tetap secara kaku.
2. **Manajemen Kunci API:** Tabel `UserApiKey` menggunakan relasi satu-ke-satu (*one-to-one*) dengan tabel `User`. Implementasi kunci API ini dilengkapi dengan fitur `onDelete: Cascade`, yang berarti jika akun pengguna dihapus, maka kunci API yang terkait akan secara otomatis terhapus dari sistem demi keamanan data.
3. **Identitas Unik:** Penggunaan `cuid()` sebagai generator ID pada tabel `Job` dan `UserApiKey` menjamin keunikan identitas di seluruh sistem dan lebih optimal untuk lingkungan terdistribusi dibandingkan dengan penggunaan integer *auto-increment*.

5.2.3. Transformasi ke PostgreSQL

Setelah skema didefinisikan, dilakukan proses migrasi menggunakan perintah `npx prisma migrate dev`. Proses ini menghasilkan struktur tabel fisik di dalam PostgreSQL yang siap digunakan oleh aplikasi. Setiap perubahan pada skema akan tercatat dalam berkas migrasi SQL untuk memastikan konsistensi antara lingkungan pengembangan dan lingkungan produksi.

5.3. Implementasi Backend dan Orkestrasi Tugas

Implementasi pada bagian *backend* difokuskan pada penyediaan layanan yang mampu menangani beban kerja komputasi tinggi secara asinkron. Hal ini dilakukan dengan memisahkan fungsi manajemen aplikasi (Next.js) dengan fungsi eksekusi model pengujian (FastAPI), yang dijembatani oleh orkestrator tugas (Trigger.dev).

5.3.1. Implementasi Task Runner (Trigger.dev)

Pekerjaan pengujian REST API sering kali memakan waktu yang lama, mulai dari hitungan menit hingga satu jam, tergantung pada kompleksitas spesifikasi API. Untuk mencegah terjadinya *timeout* pada server web, sistem mengimplementasikan *task runner* menggunakan Trigger.dev v3.

Kode 5.2 menunjukkan implementasi tugas `api-test-runner` di dalam berkas `src/trigger/api-test.ts`:


```

TypeScript
import { task } from "@trigger.dev/sdk/v3";
import { prisma } from "@lib/prisma";

export const apiTestRunner = task({
  id: "api-test-runner",
  run: async (payload: { jobId: string; config: any }) => {
    // 1. Perbarui status pekerjaan menjadi 'processing'
    await prisma.job.update({
      where: { id: payload.jobId },
      data: { status: "processing" },
    });

    // 2. Kirim permintaan ke Backend FastAPI
    const response = await
    fetch(`${process.env.MODEL_BACKEND_URL}/api/v1/run`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "X-API-Key": process.env.MODEL_API_KEY
      },
      body: JSON.stringify(payload.config),
    });

    const result = await response.json();

    // 3. Simpan hasil akhir ke PostgreSQL
    return await prisma.job.update({
      where: { id: payload.jobId },
      data: {
        status: "completed",
        summary: result.summary,
        uniqueServerError: result.total_errors,
        progressPercentage: 100,
      },
    });
  },
});

```

Kode 5.2 Implementasi *Task Runner* menggunakan SDK Trigger.dev v3

Implementasi ini memungkinkan sistem untuk menjalankan pengujian secara mandiri di latar belakang. Pengguna dapat menutup *browser* tanpa membatalkan proses pengujian yang sedang berjalan.

5.3.2. Implementasi API Wrapper (FastAPI)

Layanan mesin AI dibangun menggunakan FastAPI di lingkungan Python untuk memastikan kompatibilitas penuh dengan *engine* AutoRestTest. Layanan ini bertindak sebagai *wrapper* yang menerima parameter pengujian, menginisialisasi agen *Reinforcement Learning*, dan menghubungkan sistem dengan OpenAI LLM.

Kode 5.3 menunjukkan implementasi *endpoint* utama pada berkas `app/main.py`:

```

Python
from fastapi import FastAPI, Header, HTTPException
from app.services.model_service import run_fuzzing

app = FastAPI()

@app.post("/api/v1/run")
async def execute_test(config: TestConfig, x_api_key: str =
Header(None)):
    # Validasi API Key untuk keamanan komunikasi antar-layanan
    if x_api_key != settings.API_KEY:
        raise HTTPException(status_code=403, detail="Unauthorized
access")

    try:
        # Menjalankan mesin AutoRestTest dengan parameter yang
diberikan
        result = await run_fuzzing(
            spec_url=config.spec_url,
            llm_model=config.llm_model,
            rl_params=config.rl_params
        )
        return {"status": "success", "summary": result.summary,
"total_errors": result.errors}
    except Exception as e:
        return {"status": "failed", "message": str(e)}

```

Kode 5.3 Implementasi *API Wrapper* menggunakan FastAPI untuk eksekusi model

Pada implementasi ini, integrasi dengan model cerdas dilakukan melalui layanan `run_fuzzing`. Mesin akan memproses spesifikasi OpenAPI yang diunggah, membangun graf dependensi, dan melakukan *fuzzing* sesuai dengan tingkat eksplorasi yang telah diatur oleh pengguna melalui antarmuka web.

5.4. Implementasi Antarmuka Pengguna (Frontend)

Implementasi antarmuka pengguna dibangun menggunakan kerangka kerja Next.js 15 dengan memanfaatkan *App Router* untuk manajemen rute yang efisien.

Fokus utama implementasi ini adalah memberikan pengalaman pengguna yang responsif melalui komponen-komponen yang modular.

5.4.1. Formulir Konfigurasi Pengujian (TestForm.tsx)

Komponen `TestForm` merupakan pintu utama bagi pengguna untuk berinteraksi dengan sistem. Komponen ini diimplementasikan menggunakan pustaka `react-hook-form` untuk manajemen status formulir dan `zod` untuk validasi skema data di sisi klien.

Kode 5.4 menunjukkan logika penanganan unggah berkas OpenAPI (OAS) dan inisialisasi pekerjaan baru:

```

TypeScript
"use client";

import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { testSchema } from "@/lib/schema";

export function TestForm() {
  const form = useForm({
    resolver: zodResolver(testSchema),
    defaultValues: {
      type: "manual",
      engine: "gpt-4",
      temperature: 0.2,
    },
  });

  const onSubmit = async (values: any) => {
    // Logika pengiriman data konfigurasi ke API Next.js
    const response = await fetch("/api/v1/jobs", {
      method: "POST",
      body: JSON.stringify(values),
    });

    if (response.ok) {
      const data = await response.json();
      router.push(`/dashboard/jobs/${data.id}`);
    }
  };

  return (
    <form onSubmit={form.handleSubmit(onSubmit)}>
      {/* Implementasi input file OAS dan slider parameter RL/LLM */}
    </form>
  );
}

```

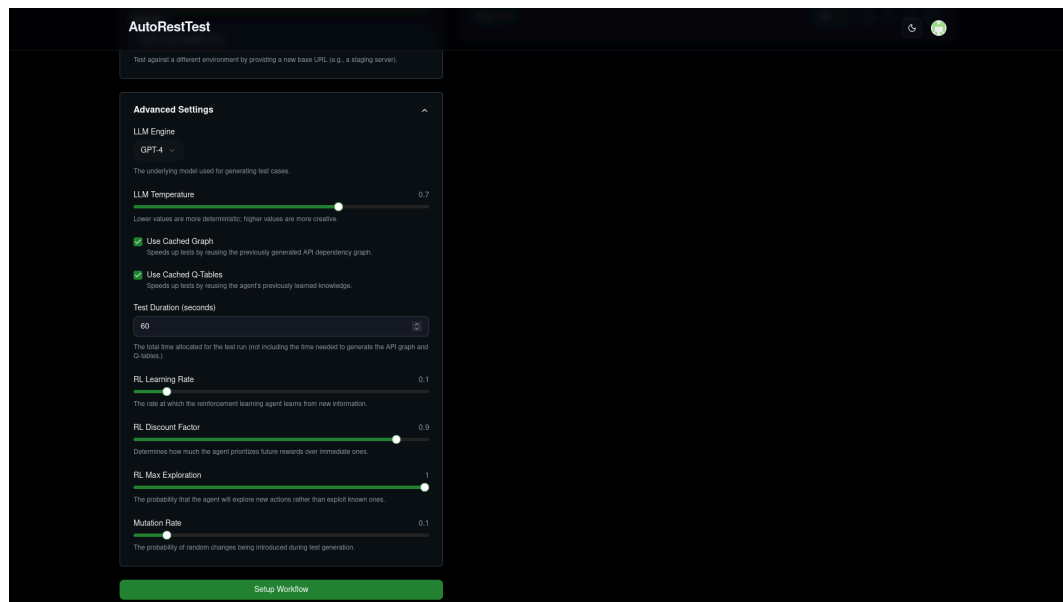
Kode 5.4 Implementasi komponen formulir konfigurasi pengujian menggunakan React Hook Form.

The screenshot shows the AutoRestTest dashboard. On the left, the 'Create a New Test' form is visible, featuring a 'Test Type' section with 'One-Time Test' selected and 'GitHub CI Setup' as an option. Below this is the 'Core Configuration' section, which includes an 'OpenAPI Specification' upload area with a file icon and instructions to upload a Swagger file. It also has an 'API URL' field with the placeholder 'https://api.example.com'. At the bottom of the form is an 'Advanced Settings' dropdown and a 'Create Job' button. On the right, the 'Job History' table displays a list of jobs with columns for Job ID, Status, Created At, Updated At, and Actions. The table shows 25 jobs, with the first job being 'cmjbaeyj0001lar3ey4ocf' in a 'completed' status.

Gambar 5.1 Antarmuka Halaman Dashboard yang Menampilkan Formulir Pembuatan Pekerjaan Baru (*Create a New Test*) dan Tabel Riwayat Pekerjaan (*Job History*).

This screenshot shows the AutoRestTest dashboard with the 'Create a New Test' form configured for CI. The 'Test Type' section now has 'GitHub CI Setup' selected. The 'CI Configuration' section is expanded, showing fields for 'Repository Name' (placeholder: owner/repo-name), 'Path to Spec File' (placeholder: swagger.json), 'Your Personal API Key' (placeholder: art_3da5c5e106738bbe97cb205b04c9495), and 'API Key Secret Name' (placeholder: AUTORESTTEST_API_KEY). An 'Action Required' message states: 'You must add your API Key to your repository's secrets as AUTORESTTEST_API_KEY for the workflow to run successfully.' Below this is a 'How to do that?' link. The 'Job History' table on the right remains the same, showing a list of 25 jobs with their respective IDs, statuses, and timestamps.

Gambar 5.2 Antarmuka Halaman Dashboard yang Menampilkan Formulir Pembuatan CI Setup dan Tabel Riwayat Pekerjaan (*Job History*).



Gambar 5.3 Antarmuka yang memungkinkan pengguna mengatur parameter cerdas seperti *temperature* dan *learning rate*.

5.4.2. Dasbor Riwayat dan Pemantauan (JobHistory.tsx)

Untuk memberikan informasi terkini mengenai proses pengujian yang sedang berjalan, sistem mengimplementasikan komponen **JobHistory**. Komponen ini melakukan pengambilan data secara berkala (*polling*) atau memanfaatkan pembaruan status dari basis data untuk menampilkan progres pengerjaan.

Job History

Job ID	Status	Created At	Updated At	Actions
cmjbaeyjp0001larr3eyf4ocf http://103.167.133.167/features	completed	12/18/2025, 5:18:16 PM	12/18/2025, 5:19:44 PM	...
cmjb7j462000b1gxb92qhzzxn http://103.167.133.167/features	failed ?	12/18/2025, 3:57:31 PM	12/18/2025, 3:57:31 PM	...
cmjb2nt8y0007la9iv35uq4ga http://103.167.133.167/features	completed	12/18/2025, 1:41:12 PM	12/18/2025, 1:42:38 PM	...
cmja1oy9p0001la9ikor79r4u http://103.167.133.167/features	failed ?	12/17/2025, 8:26:20 PM	12/17/2025, 8:26:57 PM	...
cmj9s87jp00031go2gkbv2tb http://103.167.133.167/features	completed	12/17/2025, 4:01:22 PM	12/17/2025, 4:02:42 PM	...
cmj9rrnjw0003laotx6glilj http://103.167.133.167/countries/rest	completed	12/17/2025, 3:48:29 PM	12/17/2025, 3:49:52 PM	...
cmj9rqxl80001laotdn2pxyfr http://103.167.133.167/features	completed	12/17/2025, 3:47:56 PM	12/17/2025, 3:49:23 PM	...
cmj9rac3t00011go2p1a7p200 http://103.167.133.167/features	completed	12/17/2025, 3:35:01 PM	12/17/2025, 3:36:26 PM	...
cmj9p9px6000b1ghev2agaufd http://103.167.133.167/features	completed	12/17/2025, 2:38:33 PM	12/17/2025, 2:48:44 PM	...
cmj9p1inj00091gheb69yefbi http://103.167.133.167/features	completed	12/17/2025, 2:32:11 PM	12/17/2025, 2:33:32 PM	...
cmj9ox2ld00071gheyf84ctf http://103.167.133.167/features	completed	12/17/2025, 2:28:43 PM	12/17/2025, 2:30:05 PM	...
cmj9otd9700051ghe9s1txxxx http://103.167.133.167/features	completed	12/17/2025, 2:25:51 PM	12/17/2025, 2:27:20 PM	...
cmj9nkao300031ghe1z2iy7ia http://103.167.133.167/features	completed	12/17/2025, 1:50:48 PM	12/17/2025, 1:52:11 PM	...
cmj9ir1jc00031go48xqcovot http://103.167.133.167/features	completed	12/17/2025, 11:36:04 AM	12/17/2025, 11:37:27 AM	...

Page 1 of 1

25 << < > >>

Gambar 5.4 Tabel yang berisi daftar pengujian yang pernah dilakukan, lengkap dengan label status (seperti *Queued*, *Processing*, atau *Completed*), dan waktu pembuatan.

5.4.3. Visualisasi Hasil Pengujian (JobDetailsDataExplorer.tsx)

Hasil pengujian yang dikembalikan oleh *engine* *AutoRestTest* disajikan melalui dasbor interaktif. Implementasi ini menggunakan pustaka grafik untuk memetakan distribusi kode status respons HTTP (2xx, 4xx, 5xx) sehingga pengguna dapat dengan cepat mengidentifikasi kerentanan pada API.

Kode 5.5 menunjukkan implementasi pemetaan data untuk grafik distribusi:


```

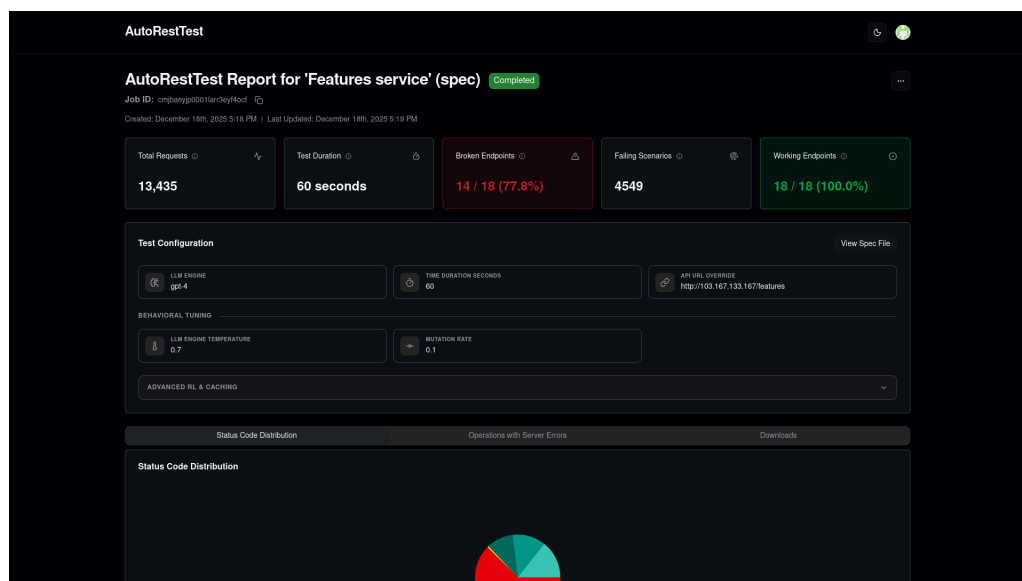
TypeScript
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer }
  from "recharts";

export function JobStatusChart({ data }: { data: any }) {
  // Transformasi data statusCodeCounter dari JSON menjadi array
  untuk Recharts
  const chartData = Object.entries(data).map(([code, count]) => ({
    name: code,
    value: count,
  }));

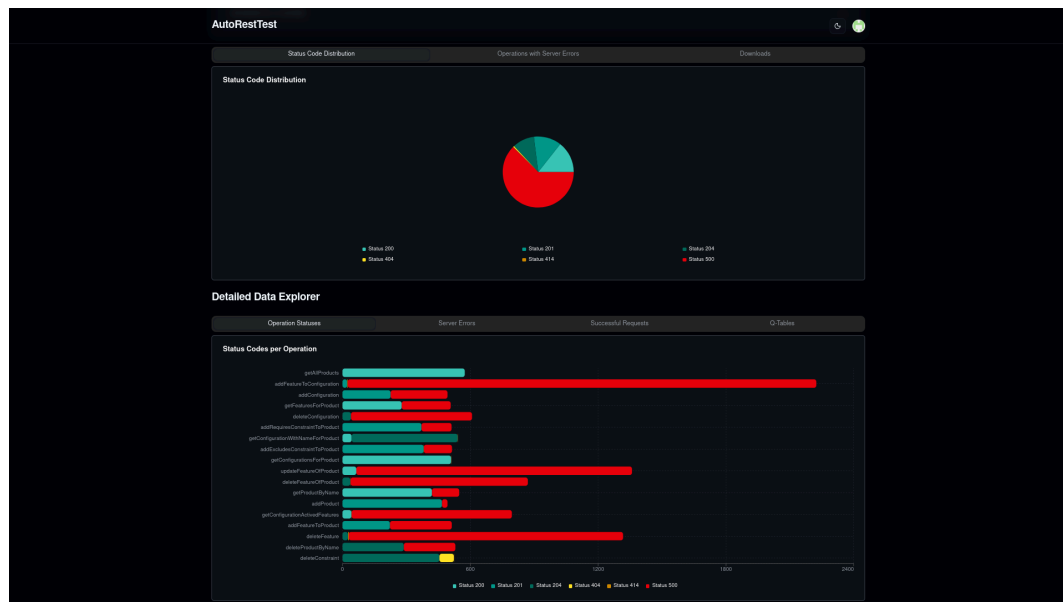
  return (
    <ResponsiveContainer width="100%" height={300}>
      <BarChart data={chartData}>
        <XAxis dataKey="name" />
        <YAxis />
        <Tooltip />
        <Bar dataKey="value" fill="#3b82f6" />
      </BarChart>
    </ResponsiveContainer>
  );
}

```

Kode 5.5 Implementasi komponen grafik distribusi kode status respons menggunakan pustaka Recharts.



Gambar 5.5 Tampilan landing halaman *job details* (JobDetailsDataExplorer.tsx)

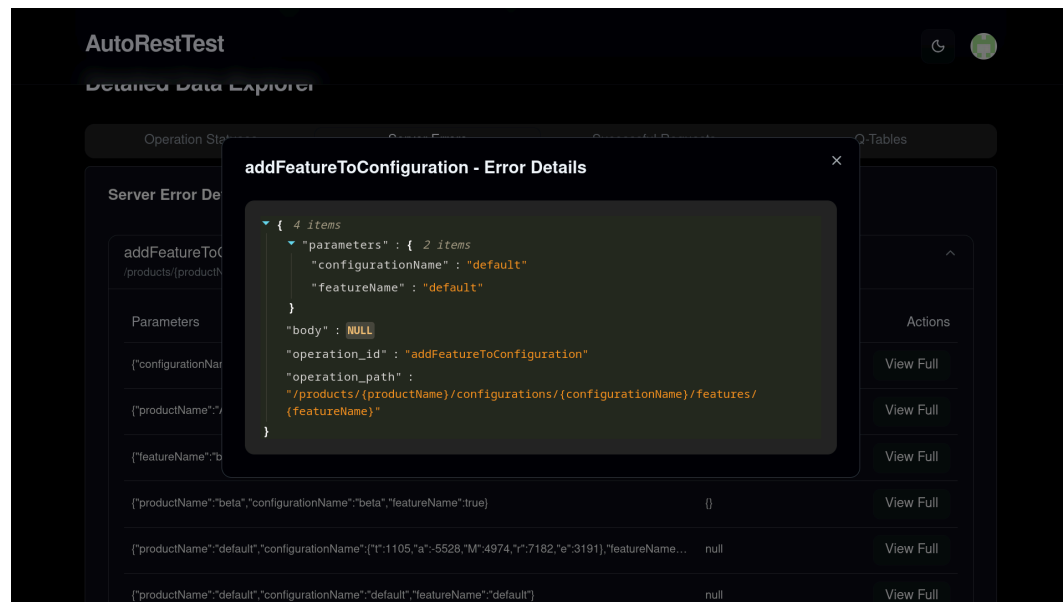


Gambar 5.6 Tampilan *data explorer job details* (JobDetailsDataExplorer.tsx)

The screenshot displays the AutoRestTest application interface, specifically the 'Server Error Details' section. The 'Server Errors' tab is active, showing a table of error details for the operation 'addFeatureToConfiguration'. The table has three columns: 'Parameters', 'Body', and 'Actions'. The 'Parameters' column contains JSON objects representing the request parameters. The 'Body' column contains the request body, which is null for most entries. The 'Actions' column contains a 'View Full' button for each entry. A red badge indicates '1265 unique errors'.

Parameters	Body	Actions
{"configurationName":"default","featureName":"default"}	null	View Full
{"productName":"AlphaProduct","configurationName":"default","featureName":"autoScaling"}	null	View Full
{"featureName":"beta"}	{}	View Full
{"productName":"beta","configurationName":"beta","featureName":true}	{}	View Full
{"productName":"default","configurationName":{"t":1105,"a":5528,"M":4974,"r":7182,"e":3191},"featureName":true}	null	View Full
{"productName":"default","configurationName":"default","featureName":"default"}	null	View Full

Gambar 5.7 Tampilan *server errors job details* (JobDetailsDataExplorer.tsx)



Gambar 5.8 Tampilan kombinasi body dan parameter *job details*
(JobDetailsDataExplorer.tsx)

5.5. Implementasi Integrasi CI/CD

Salah satu fitur unggulan dari sistem ini adalah kemampuannya untuk berintegrasi langsung dengan alur kerja pengembangan perangkat lunak melalui GitHub Actions. Implementasi ini memungkinkan pengujian *fuzzing* berjalan secara otomatis setiap kali terdapat pembaruan kode pada repositori pengguna.

5.5.1. Otomasi Setup Workflow (Octokit)

Untuk menjembatani komunikasi antara platform dan GitHub, sistem menggunakan pustaka Octokit. Pustaka ini memungkinkan aplikasi untuk berinteraksi dengan API GitHub guna membuat berkas konfigurasi *workflow* secara otomatis tanpa mengharuskan pengguna menulis kode YAML secara manual.

Implementasi logika ini dilakukan pada rute API `src/app/api/v1/ci-setup/route.ts`. Kode 5.6 menunjukkan bagaimana sistem membuat komit baru yang berisi berkas `.github/workflows/autoresttest.yml` ke repositori pengguna:

```

TypeScript
import { Octokit } from "@octokit/rest";

export async function POST(req: Request) {
  const { repoOwner, repoName, branch, apiKey } = await
  req.json();
  // Inisialisasi Octokit dengan token OAuth pengguna
  const octokit = new Octokit({ auth:
  process.env.GITHUB_ACCESS_TOKEN });

  const workflowYaml = `
name: AutoRestTest CI
on:
  push:
    branches: [${branch}]
jobs:
  fuzzing:
    runs-on: ubuntu-latest
    steps:
      - name: Trigger AutoRestTest
        run: |
          curl -X POST https://art.dvad.my.id/api/v1/jobs \\\
          -H "Authorization: Bearer ${apiKey}" \\\
          -d '{"type": "ci", "config": { "spec_url": "..." }}'
`;
  try {
    // Mengirimkan berkas workflow ke repositori GitHub
    await octokit.repos.createOrUpdateFileContents({
      owner: repoOwner,
      repo: repoName,
      path: ".github/workflows/autoresttest.yml",
      message: "chore: add AutoRestTest CI workflow",
      content: Buffer.from(workflowYaml).toString("base64"),
      branch: branch,
    });
    return Response.json({ message: "CI/CD setup successful" });
  } catch (error) {
    return Response.json({ error: "Failed to setup CI/CD" }, {
      status: 500 });
  }
}

```

Kode 5.6 Implementasi otomasi penyisipan berkas *workflow* GitHub Actions menggunakan Octokit.

5.5.2. Mekanisme Triggering CI/CD

Setelah berkas *workflow* berhasil disisipkan ke dalam repositori, mekanisme pengujian akan mengikuti alur berikut:

1. **Event Push:** Setiap kali pengguna melakukan *push* kode ke cabang (*branch*) yang ditentukan, GitHub Actions akan mendeteksi perubahan tersebut.
2. **Eksekusi Runner:** *Runner* GitHub Actions akan menjalankan perintah `curl` yang telah didefinisikan dalam berkas YAML untuk mengirimkan permintaan ke API sistem.
3. **Inisialisasi Job:** Sistem menerima permintaan tersebut, memvalidasi kunci API pengguna, dan memasukkan tugas pengujian ke dalam antrian Trigger.dev secara asinkron.

Dengan implementasi ini, pengguna mendapatkan kemudahan berupa pengujian yang bersifat proaktif, di mana setiap perubahan kode akan langsung divalidasi oleh model *AutoRestTest* untuk memastikan tidak ada kerentanan baru yang muncul pada API.

5.6. Implementasi Keamanan dan Autentikasi

Keamanan merupakan aspek vital dalam pengembangan sistem ini, mengingat platform mengelola data sensitif seperti spesifikasi API pengguna dan kredensial akses repositori GitHub. Implementasi keamanan dibagi menjadi dua lapis utama: autentikasi pengguna pada antarmuka web dan otorisasi akses pada layanan *backend*.

5.6.1. Manajemen Autentikasi Pengguna (Clerk)

Sistem menggunakan layanan Clerk untuk menangani seluruh siklus hidup autentikasi, mulai dari pendaftaran, masuk log (*login*), hingga manajemen sesi. Clerk dipilih karena menyediakan fitur keamanan standar industri seperti autentikasi multifaktor (MFA) dan integrasi OAuth yang memudahkan pengguna masuk menggunakan akun GitHub mereka.

Implementasi Clerk pada tingkat akar aplikasi dilakukan di dalam berkas `layout.tsx` seperti yang ditunjukkan Kode 5.7:

```
TypeScript
import { ClerkProvider, SignInButton, SignedIn, SignedOut,
  UserButton } from '@clerk/nextjs'

export default function RootLayout({ children }: { children:
  React.ReactNode }) {
  return (
    <ClerkProvider>
      <html lang="en">
        <body>
          <header>
            <SignedOut>
              <SignInButton />
            </SignedOut>
            <SignedIn>
              <UserButton />
            </SignedIn>
          </header>
          {children}
        </body>
      </html>
    </ClerkProvider>
  )
}
```

Kode 5.7 Implementasi Clerk Provider pada rute akar (root layout) aplikasi.

5.6.2. Perlindungan Rute dengan Middleware

Untuk memastikan bahwa fitur-fitur pengujian hanya dapat diakses oleh pengguna yang sah, sistem mengimplementasikan *middleware* pada Next.js. *Middleware* ini secara otomatis memeriksa status sesi pengguna sebelum memberikan akses ke rute-rute sensitif seperti `/dashboard` dan `/api/v1/jobs`.

Kode 5.8 menunjukkan implementasi perlindungan rute pada berkas `middleware.ts`:

```

TypeScript
import { clerkMiddleware, createRouteMatcher } from
"@clerk/nextjs/server";

// Mendefinisikan rute yang harus dilindungi (private)
const isProtectedRoute = createRouteMatcher(["/dashboard(.*)",
"/api/v1/(.*)"]);

export default clerkMiddleware((auth, req) => {
  // Jika pengguna mencoba mengakses rute terproteksi tanpa login,
  arahkan ke login
  if (isProtectedRoute(req)) auth().protect();
});

export const config = {
  matcher: ["/(?!.*\\.\\.\\.next).*", "/", "/(api|trpc)(.*)"],
};

```

Kode 5.8 Implementasi *Middleware* Next.js untuk proteksi rute dasbor dan API.

5.6.3. Keamanan Komunikasi Antar-Layanan (API Key)

Selain autentikasi pengguna, sistem juga harus mengamankan komunikasi antara *worker* (Trigger.dev) dan *backend* model (FastAPI). Hal ini dilakukan dengan menggunakan skema *API Key validation*. Setiap permintaan yang dikirimkan oleh sistem harus menyertakan kunci rahasia dalam *header* HTTP. Jika kunci yang dikirimkan tidak sesuai dengan yang tersimpan pada variabel lingkungan (*environment variables*) di sisi *backend*, maka permintaan akan ditolak.

Kode 5.9 menunjukkan logika validasi pada *backend* FastAPI:

```

TypeScript
from fastapi import Header, HTTPException

async def verify_api_key(x_api_key: str = Header(None)):
    if x_api_key != "RAHASIA_API_KEY_SISTEM":
        raise HTTPException(
            status_code=403,
            detail="Forbidden: Invalid API Key"
        )
    return x_api_key

```

Kode 5.9 Implementasi fungsi validasi *API Key* untuk keamanan komunikasi antar-layanan.

Dengan kombinasi Clerk untuk sisi pengguna dan API Key untuk sisi internal, sistem memiliki pertahanan berlapis yang meminimalisir risiko akses ilegal terhadap mesin pengujian cerdas *AutoRestTest*.

5.7. Akses Layanan dan Repositori

Sebagai bentuk transparansi hasil pengembangan dan untuk keperluan validasi produk secara langsung, sistem *AutoRestTest* ini telah disebarluaskan (*deployed*) pada lingkungan produksi yang dapat diakses melalui jaringan internet publik. Selain itu, seluruh kode sumber aplikasi juga telah diunggah ke repositori repositori kode untuk mempermudah peninjauan teknis. Informasi detail mengenai URL layanan, tautan repositori, serta instruksi akses bagi pengguna dan penguji dapat dilihat pada **Lampiran 6: Informasi Akses Sistem dan Repositori**.

BAB VI

PENGUJIAN DAN EVALUASI PRODUK

6.1. Hasil Pengujian Fungsional

Pengujian fungsional dilakukan untuk memvalidasi bahwa seluruh fitur yang telah diimplementasikan berjalan sesuai dengan spesifikasi kebutuhan dan bebas dari kesalahan logika pada tingkat kode.

6.1.1. Hasil Unit Testing

Unit testing dilakukan pada dua komponen utama sistem, yaitu *Backend-for-Frontend* (Next.js) dan *Core Engine Wrapper* (FastAPI). Pengujian ini bertujuan untuk memastikan setiap modul kode memiliki tingkat ketahanan yang tinggi.

1. **Next.js (Frontend & BFF):** Berdasarkan audit menggunakan instrumen Jest, seluruh berkas API utama mencapai tingkat *coverage* yang sangat tinggi dengan rata-rata keseluruhan sebesar 98,15% untuk pernyataan (*statements*) dan 100% untuk fungsi. Modul-modul krusial seperti manajemen pekerjaan (**jobs**), pengaturan CI/CD (**ci-setup**), dan manajemen kunci API (**user/api-key**) telah teruji sepenuhnya (100% *coverage*).

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	98.15	94.62	100	98.15	
app/api/v1/ci-setup	92.94	76.47	100	92.94	
route.ts	92.94	76.47	100	92.94	126-128,131-132,159,164-169
app/api/v1/jobs	97.84	94.44	100	97.84	
route.ts	97.84	94.44	100	97.84	75-77
app/api/v1/jobs/[job_id]	100	100	100	100	
route.ts	100	100	100	100	
app/api/v1/jobs/[job_id]/cancel	100	100	100	100	
route.ts	100	100	100	100	
app/api/v1/jobs/[job_id]/progress	100	100	100	100	
route.ts	100	100	100	100	
app/api/v1/jobs/[job_id]/replay	100	100	100	100	
route.ts	100	100	100	100	
app/api/v1/user/api-key	100	100	100	100	
route.ts	100	100	100	100	
app/api/webhooks/clerk	100	100	100	100	
route.ts	100	100	100	100	
lib	100	100	100	100	
schema.ts	100	100	100	100	

Gambar 6.1 Code coverage frontend dan backend for frontend

2. **FastAPI (Core Engine Wrapper):** Pengujian pada sisi Python menggunakan Pytest menunjukkan tingkat *coverage* keseluruhan sebesar 98%. Modul inti `model_service.py` yang menangani logika *fuzzing* memiliki cakupan pengujian sebesar 97%.

Name	Stmts	Miss	Cover
app/__init__.py	0	0	100%
app/api/__init__.py	0	0	100%
app/api/deps.py	12	0	100%
app/api/v1/__init__.py	0	0	100%
app/api/v1/api.py	4	0	100%
app/api/v1/endpoints/__init__.py	0	0	100%
app/api/v1/endpoints/test_runner.py	17	0	100%
app/core/__init__.py	0	0	100%
app/core/config.py	19	0	100%
app/main.py	5	0	100%
app/models/__init__.py	0	0	100%
app/models/test_config.py	27	0	100%
app/services/__init__.py	0	0	100%
app/services/model_service.py	95	3	97%
TOTAL	179	3	98%

Gambar 6.2 Code coverage model wrapper API

3. **Status Kelulusan:** Sebanyak 14 *test cases* yang mencakup validasi kunci API, penanganan kesalahan autentikasi, manajemen konfigurasi pengujian, hingga penanganan hasil pengujian yang malformasi telah dijalankan dengan status **PASSED** (100% lulus).

6.1.2. Hasil Validasi Fitur Utama

Validasi fitur utama dilakukan melalui skenario pengujian manual (*one-time job*) pada antarmuka web. Hasil pengujian menunjukkan bahwa sistem berhasil melakukan orkestrasi pengujian REST API secara *end-to-end*.

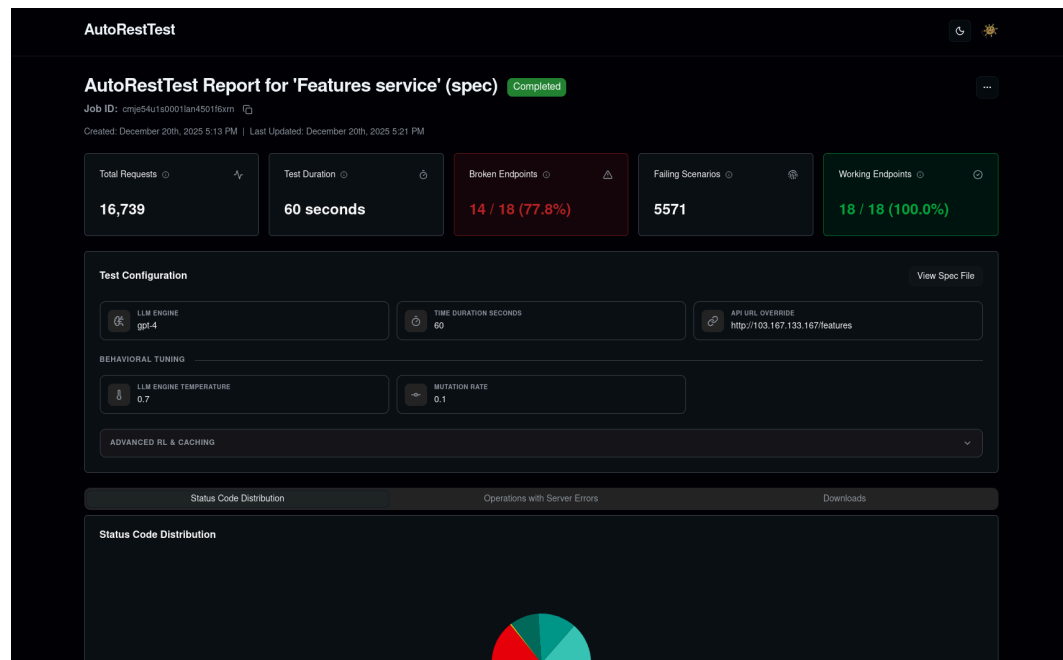
Sebagai contoh kasus, dilakukan pengujian pada '*Features service*' menggunakan spesifikasi OpenAPI yang diunggah secara manual. Sistem berhasil menjalankan konfigurasi cerdas dengan model GPT-4, *temperature* 0.7, dan *mutation rate* 0.1. Dalam durasi pengujian selama 60 detik, sistem mampu melakukan total **16.739 permintaan** (*requests*) ke API target.

The screenshot displays the AutoRestTest web interface. On the left, the 'Create a New Test' section includes a 'Test Type' selector with 'One-Time Test' selected, a 'Core Configuration' section for uploading an OpenAPI Specification (Swagger) file named 'features.json' and setting the API URL to 'http://103.167.133.167/features', and an 'Advanced Settings' dropdown. A green 'Create Job' button is at the bottom. On the right, the 'Job History' table shows two completed jobs.

Job ID	Status	Created At	Updated At	Actions
cmje5kpcb0005lan4dzyt5p52 http://103.167.133.167/market	completed	12/20/2025, 5:26:05 PM	12/20/2025, 5:27:29 PM	...
cmje54u1s0001lan450116xm http://103.167.133.167/features	completed	12/20/2025, 5:13:44 PM	12/20/2025, 5:21:20 PM	...

Page 1 of 1

Gambar 6.3 Tampilan form *one-time test* yang telah diisi oleh pengguna



Gambar 6.4 Visualisasi laporan hasil pengujian API (features service) yang menunjukkan statistik permintaan, durasi, dan broken endpoints

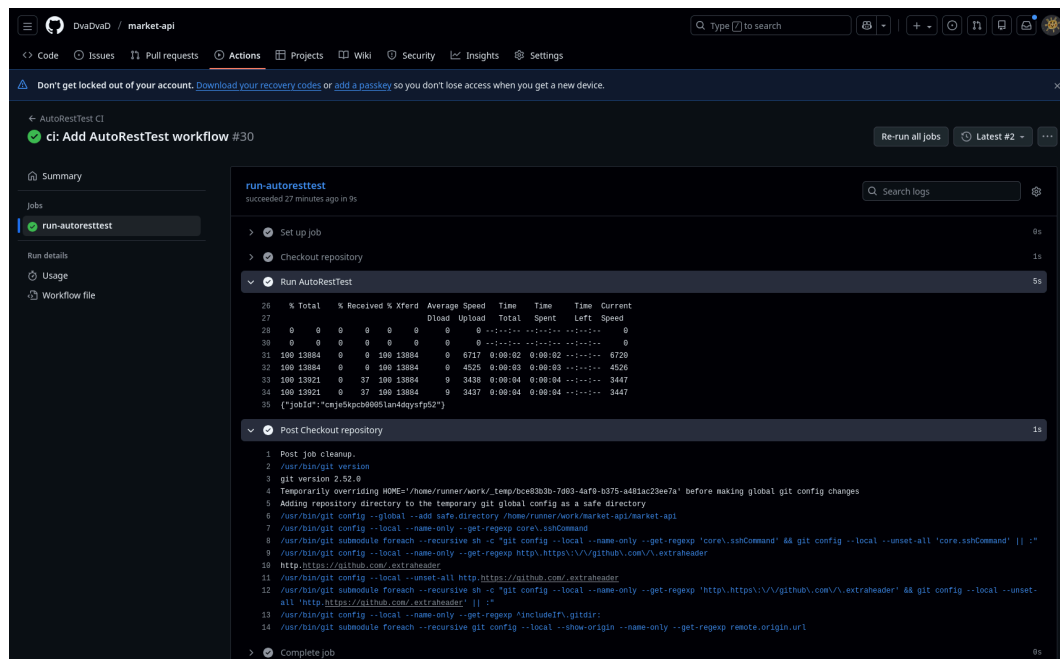
Dasbor laporan secara otomatis menyajikan ringkasan hasil yang menunjukkan bahwa sistem berhasil mengidentifikasi 14 dari 18 *endpoints* (77,8%) yang memiliki potensi masalah (*broken*), dengan total **5.571 skenario kegagalan** yang terdeteksi. Hal ini membuktikan bahwa fitur visualisasi dan interpretasi hasil telah berfungsi dengan baik.

6.1.3. Hasil Validasi Integrasi CI/CD

Pengujian integrasi CI/CD dilakukan untuk memvalidasi kemampuan sistem dalam melakukan otomatisasi pengujian melalui ekosistem GitHub Actions. Berdasarkan hasil pengujian pada repositori *market-api*:

1. **Penyisipan Workflow:** Sistem berhasil membuat dan menyisipkan berkas konfigurasi `.github/workflows/autoresttest.yml` ke repositori target.
2. **Pemicuan Otomatis:** Saat terdapat aksi *push* pada repositori, GitHub Actions berhasil menjalankan langkah *Run AutoRestTest* secara otomatis.

3. **Komunikasi API:** Log pada GitHub Actions menunjukkan bahwa perintah `curl` berhasil dieksekusi dan menerima respons sukses dari sistem berupa ID Pekerjaan unik (`cmje5kpcb00051an4dqysfp52`).



Gambar 6.5 Tampilan log keberhasilan eksekusi *workflow* AutoRestTest pada antarmuka Github Actions

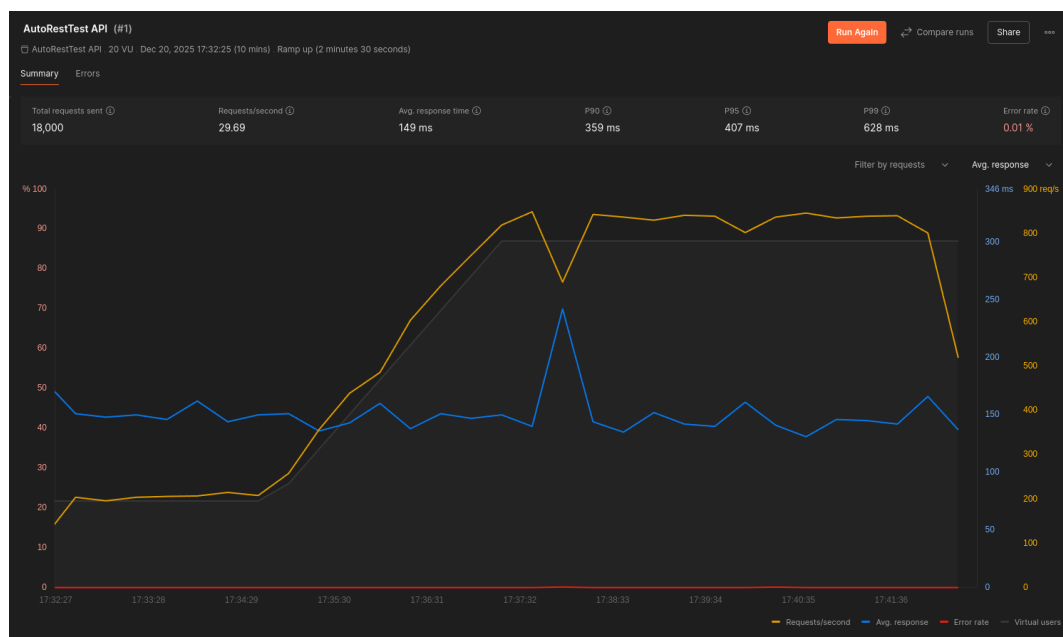
4. **Sinkronisasi Riwayat:** Pekerjaan yang dipicu melalui CI/CD tersebut muncul secara otomatis dalam daftar *Job History* pada aplikasi web dengan status *completed*, menunjukkan integrasi yang lancar antara repositori eksternal dan platform utama.

6.2. Hasil Pengujian Non-Fungsional

Pengujian non-fungsional dilakukan untuk menilai kualitas teknis sistem dari aspek performa, responsivitas, dan kepatuhan terhadap standar pengembangan web modern. Pengujian ini memastikan bahwa sistem tidak hanya berfungsi dengan benar, tetapi juga mampu menangani beban kerja secara stabil dan memiliki aksesibilitas yang baik.

6.3.1. Analisis Performa API (Postman)

Pengujian performa dilakukan pada titik akhir (*endpoint*) API utama sistem untuk mengukur stabilitas dan kecepatan respons di bawah beban kerja tertentu. Pengujian ini disimulasikan menggunakan Postman dengan skenario 20 pengguna virtual (*virtual users*) selama durasi 10 menit.



Gambar 6.6 Hasil pengujian performa API menggunakan Postman yang menunjukkan grafik waktu respons dan tingkat kesalahan

Tabel 6.1 menjabarkan ringkasan metrik performa yang diperoleh:

Tabel 6.1 Ringkasan metrik performa API (Waktu Respons, *Throughput*, dan *Error Rate*).

Metrik Performa	Nilai Hasil Pengujian
Total Permintaan Terkirim	18.000 permintaan
Rata-rata Permintaan per Detik	29,69 req/s
Rata-rata Waktu Respons (Average)	149 ms
Percentile 90 (P90)	359 ms
Percentile 95 (P95)	407 ms

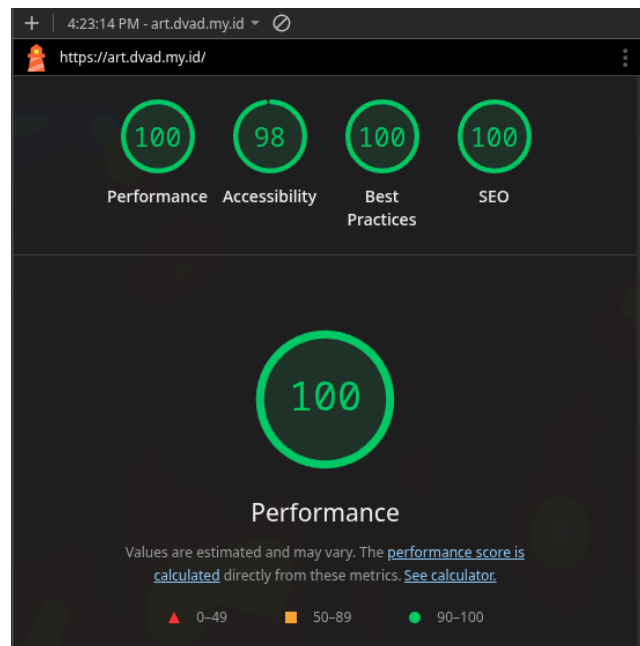
Percentile 99 (P99)	628 ms
Tingkat Kesalahan (Error Rate)	0,01%

Berdasarkan hasil pengujian di atas, sistem menunjukkan performa yang sangat stabil dengan rata-rata waktu respons sebesar 149 ms. Meskipun terdapat peningkatan latensi pada beban puncak (P99 mencapai 628 ms), tingkat kesalahan yang hanya sebesar 0,01% membuktikan bahwa sistem memiliki reliabilitas yang sangat tinggi dalam menangani ribuan permintaan secara simultan tanpa mengalami kegagalan layanan yang signifikan.

6.3.2. Audit Kualitas Web (Google Lighthouse)

Audit kualitas antarmuka web dilakukan untuk mengukur pengalaman pengguna dan optimalisasi teknis pada platform AutoRestTest. Audit ini dilakukan secara otomatis menggunakan instrumen Google Lighthouse pada lingkungan produksi. Hasil audit menunjukkan skor yang mendekati sempurna pada seluruh kategori utama:

1. **Performance (100):** Menunjukkan bahwa aplikasi memiliki kecepatan muat halaman yang optimal dan interaktivitas yang sangat responsif di sisi klien.
2. **Accessibility (98):** Menunjukkan bahwa elemen antarmuka pengguna telah dirancang dengan inklusif dan dapat diakses dengan baik oleh pengguna dengan berbagai kebutuhan.
3. **Best Practices (100):** Menunjukkan bahwa pengembangan kode mengikuti standar keamanan terbaru dan praktik terbaik dalam pengembangan web modern.
4. **SEO (100):** Menunjukkan bahwa platform telah teroptimasi dengan baik untuk mesin pencari, memastikan kemudahan penemuan aplikasi secara daring.



Gambar 6.7 Skor hasil audit kualitas web (*performance, accessibility, best practices, seo*) menggunakan Google Lighthouse

Secara keseluruhan, hasil audit non-fungsional ini menegaskan bahwa implementasi sistem AutoRestTest telah memenuhi standar kualitas perangkat lunak yang tinggi, baik dari sisi efisiensi mesin *backend* maupun optimalisasi antarmuka *frontend*.

6.3. Analisis Task-Based Usability Testing

Evaluasi usabilitas dilakukan melalui metode *task-based testing* untuk mengukur sejauh mana sistem dapat dioperasikan secara efektif dan efisien oleh pengguna target. Pengujian ini melibatkan lima partisipan (P-1 hingga P-5) yang merepresentasikan profil *developer* dan *QA engineer* untuk mengerjakan delapan tugas spesifik (S.1 hingga S.8) yang mencakup seluruh alur kerja inti aplikasi.

6.3.1. Profil Demografis Partisipan

Sebelum pelaksanaan pengujian usabilitas, dilakukan pendataan profil demografis partisipan untuk memastikan bahwa subjek uji memiliki latar belakang yang relevan dengan target pengguna aplikasi AutoRestTest. Pengujian ini

melibatkan lima orang partisipan yang terdiri dari mahasiswa rumpun ilmu komputer serta praktisi profesional di bidang pengembangan perangkat lunak.

Tabel 6.2 menunjukkan rincian profil demografis partisipan yang mengikuti rangkaian pengujian:

Tabel 6.2 Profil demografis partisipan *usability testing*

ID Partisipan	Peran / Status	Pengalaman Bidang Perangkat Lunak
P-1	<i>Developer</i>	4 Tahun
P-2	Mahasiswa	4 Tahun
P-3	Mahasiswa	1 Tahun
P-4	<i>Developer</i>	4 Tahun
P-5	Quality Assurance (QA) Engineer	2 Tahun

6.3.2. Hasil Observasi dan Tingkat Penyelesaian Tugas

Data observasi dikumpulkan dengan mencatat tingkat keberhasilan partisipan dalam menyelesaikan tugas menggunakan skala 0-2 (0: Gagal/Bingung, 1: Berhasil dengan bantuan/ragu, 2: Berhasil dengan lancar/paham). Tabel 6.3 menunjukkan rekapitulasi skor mentah dari sesi pengujian tersebut:

Tabel 6.3 Rekapitulasi skor mentah hasil observasi *task-based usability testing* (Skala 0-2)

Kode Tugas	Deskripsi Tugas	P-1	P-2	P-3	P-4	P-5
S.1	Impresi Halaman Utama	2	2	2	2	2
S.2	Unggah Berkas OpenAPI	2	1	2	2	2
S.3	Eksekusi Pekerjaan (Job)	2	2	2	2	2

S.4	Memeriksa Ringkasan Laporan	2	2	2	2	2
S.5	Memeriksa Detail Server Error	2	2	2	2	2
S.6	Konfigurasi Integrasi CI/CD	2	1	1	2	1
S.7	Finalisasi Setup CI/CD	2	2	2	2	2
S.8	Navigasi Riwayat Pengujian	2	2	2	2	2

6.3.3. Proses Normalisasi Skor Perilaku

Data observasi di atas kemudian dinormalisasi dari skala 0-2 menjadi skala 1-5 sesuai dengan ketentuan mekanisme penskalaan skor (skor 0=1, skor 1=3, skor 2=5). Tabel 6.4 menunjukkan hasil normalisasi skor setiap peserta *usability testing*.

Tabel 6.4 Hasil normalisasi skor perilaku partisipan ke dalam standar penilaian (Skala 1-5)

Kode Tugas	P-1	P-2	P-3	P-4	P-5	Rerata Skor
S.1 (Impresi)	5	5	5	5	5	5,0
S.2 (Upload)	5	3	5	5	5	4,6
S.3 (Execute)	5	5	5	5	5	5,0
S.4 (Summary)	5	5	5	5	5	5,0
S.5 (Error Detail)	5	5	5	5	5	5,0
S.6 (Config CI)	5	3	3	5	3	3,8
S.7 (Finalize)	5	5	5	5	5	5,0
S.8 (History)	5	5	5	5	5	5,0

6.3.4. Analisis Skor Komponen Berbobot

Berdasarkan hasil normalisasi tugas observasi, dilakukan pengelompokan skor ke dalam komponen utama penilaian . Analisis dilakukan pada aspek kualitas, usabilitas, dan pengalaman pengguna:

1. **Skor Kualitas (Bobot 40%):** Diturunkan dari kemampuan pemahaman laporan (S.4 dan S.5). Seluruh partisipan memperoleh skor sempurna (Rerata: 5,0), menunjukkan penyajian data pengujian sangat efektif.
2. **Skor Usability (Bobot 30%):** Mengevaluasi efektivitas alur kerja utama (S.2 dan S.3) . Komponen ini memperoleh rerata skor 4,8. Hambatan kecil ditemukan pada P-2 terkait masalah kontras pada *light mode*.
3. **Skor Pengalaman (Bobot 20%):** Menilai impresi awal dan navigasi fitur sekunder (S.1, S.6, S.8) . Komponen ini memperoleh rerata skor 4,6. P-2, P-3, dan P-5 mengalami sedikit kebingungan pada konfigurasi CI/CD karena urutan instruksi yang perlu diperjelas.
4. **Skor Potensi Adopsi (Bobot 10%):** Berdasarkan sentimen positif partisipan terhadap grafik laporan yang intuitif (Rerata: 5,0) .

6.3.5. Kalkulasi Skor Akhir Usabilitas

Skor akhir dihitung menggunakan rumus akumulasi rerata tiap komponen dikalikan bobotnya:

$$\text{Skor Akhir} = (5,0 \times 0,40) + (4,8 \times 0,30) + (4,6 \times 0,20) + (5,0 \times 0,10)$$

$$\text{Skor Akhir} = 2,0 + 1,44 + 0,92 + 0,5 = 4,86$$

Hasil akhir sebesar **4,86 dari 5,0** menunjukkan bahwa sistem memiliki tingkat usabilitas yang sangat tinggi. Sistem secara keseluruhan dinilai berhasil menyederhanakan proses pengujian REST API yang kompleks menjadi alur kerja yang dapat diterima oleh praktisi.

6.3.6. Analisis Masukan Peserta dan Perbaikan Fitur

Selain data kuantitatif, sesi *usability testing* juga menjangkit masukan kualitatif berupa kesan dan saran dari para partisipan. Berikut adalah ringkasan masukan partisipan untuk setiap fitur utama beserta perbaikan yang diimplementasikan untuk meningkatkan kegunaan sistem:

1. Homepage Secara keseluruhan, partisipan memberikan impresi awal yang sangat positif dan langsung memahami tujuan utama aplikasi melalui informasi yang disajikan di halaman utama.

- **Masukan Partisipan:**
 - **P-1:** "Testing kualitas API dan sekuritas."
 - **P-2:** "Testing API REST menggunakan LLM."
 - **P-4:** "Testing API yang melibatkan LLM dan MARL. AI agent yang akan mengeksplorasi API untuk mencari kesalahan tidak terduga."
- **Perbaikan yang Dilakukan:** Tidak dilakukan perubahan signifikan pada fitur ini karena seluruh partisipan sudah memiliki pemahaman yang akurat mengenai fungsi aplikasi pada pandangan pertama.

2. One-time Test Creation Form Meskipun alur kerja pembuatan tugas dinilai lancar, terdapat beberapa kendala teknis terkait aspek visual dan kejelasan komponen input.

- **Masukan Partisipan:**
 - **P-2:** "Ditemukan bahwa ada masalah kontras pada aplikasi ketika menggunakan light mode."
 - **P-3:** "Beri label required ke input field yang required. Kontras background dengan teks cukup buruk."
 - **P-4:** "Ekspektasi bahwa tidak diredirect langsung ke detail page karena dikira bakal bisa langsung menjalankan beberapa job secara sekaligus."
- **Perbaikan yang Dilakukan:**

- Memperbaiki rasio kontras warna pada tema terang (*light mode*) untuk memastikan seluruh teks dan elemen formulir memiliki tingkat keterbacaan yang tinggi.
- Menambahkan indikator visual (tanda bintang merah) pada setiap kolom input yang bersifat wajib diisi (*required*).

3. CI Setup Form Fitur integrasi CI/CD merupakan bagian yang paling banyak menerima masukan terkait kejelasan instruksi teknis dan bantuan visual.

- **Masukan Partisipan:**

- **P-1:** "Personal API key dikira dapat diubah padahal tidak bisa."
- **P-2:** "Urutan instruksi dalam form agak sedikit tidak sesuai dan ini dapat menyebabkan kebingungan untuk user."
- **P-4:** "Instruksi untuk membuat secrets di environment juga kurang jelas. Seharusnya perlu lebih banyak panduan untuk mempermudah penggunaan. Selain dari itu, error CI seharusnya lebih jelas karena error di CI tertutup oleh success message."

- **Perbaikan yang Dilakukan:**

- Menyusun ulang urutan langkah-langkah dalam formulir agar lebih logis dan mudah diikuti oleh pengguna dari sisi teknis.
- Menambahkan panduan visual berupa instruksi langkah-demi-langkah untuk membantu pengguna saat melakukan konfigurasi *secrets* pada repositori GitHub mereka.
- Memperbaiki logika sistem notifikasi sehingga pesan kesalahan tidak lagi tertutup oleh pesan keberhasilan.

4. Job History Component Komponen riwayat pekerjaan dinilai sangat intuitif dan mudah digunakan oleh seluruh partisipan untuk memantau aktivitas pengujian sebelumnya.

- **Masukan Partisipan:** Tidak ditemukan masukan atau keluhan spesifik pada komponen ini. Seluruh partisipan memberikan skor maksimal (skor 2/Mudah) dalam melakukan navigasi dan menemukan riwayat pengujian mereka.
- **Perbaikan yang Dilakukan:** Tidak diperlukan perbaikan berdasarkan hasil evaluasi usabilitas.

5. Job Detail Page Visualisasi hasil pengujian melalui grafik mendapatkan apresiasi tinggi, namun terdapat catatan mengenai kejelasan makna dari setiap metrik yang ditampilkan.

- **Masukan Partisipan:**
 - **P-1:** "Setiap section dari laporan dapat dipahami dengan jelas dan laporannya sangat membantu."
 - **P-2:** "Terdapat sedikit kebingungan tentang apa yang dilambangkan setiap data."
 - **P-4:** "Appreciate graph yang intuitive dan mudah dipahami. Bagi developer intuitif dan comfortable. Fitur preview JSON diapresiasi karena dapat digunakan untuk memeriksa kembali."
- **Perbaikan yang Dilakukan:** Menambahkan deskripsi bantuan atau *tooltips* pada setiap metrik utama di dasbor laporan untuk memberikan penjelasan mendalam mengenai arti dari setiap data statistik yang ditampilkan.

6.4. Analisis Kuesioner Pasca-Sesi

Setelah menyelesaikan seluruh rangkaian tugas dalam sesi observasi, partisipan diminta untuk mengisi kuesioner pasca-sesi guna memberikan persepsi subjektif mereka terhadap kualitas dan kemudahan penggunaan sistem. Kuesioner ini mencakup penilaian kuantitatif menggunakan Skala Likert (1-5) dan umpan balik kualitatif melalui pertanyaan terbuka.

6.4.1. Interpretasi Data Persepsi Subjektif

Penilaian kuantitatif dibagi menjadi tujuh parameter utama yang mewakili aspek kemudahan, kejelasan, dan manfaat sistem. Hasil rata-rata skor dari lima partisipan (P-1 hingga P-5) dirangkum dalam Tabel 6.5.

Tabel 6.5 Rekapitulasi skor kuesioner kepuasan pengguna berdasarkan parameter kepuasan dan kegunaan (Skala 1-5).

Kode	Parameter Penilaian	P-1	P-2	P-3	P-4	P-5	Rata
Q.4	Kemudahan penggunaan aplikasi secara keseluruhan	5	4	4	5	4	4,4
Q.5	Kejelasan tujuan dan fungsi utama aplikasi	5	5	4	5	5	4,8
Q.6	Kemudahan alur kerja pembuatan sesi baru	5	4	4	4	5	4,4
Q.7	Kemudahan dalam menemukan fitur riwayat	5	5	5	5	5	5,0
Q.8	Kejelasan penyajian hasil pada halaman laporan	5	4	4	5	4	4,4
Q.9	Kegunaan informasi hasil untuk identifikasi masalah	5	5	5	5	5	5,0
Q.10	Potensi penggunaan kembali di masa depan	5	5	5	5	4	4,8

Berdasarkan data di atas, sistem memperoleh skor sempurna (**5,0**) pada aspek **kemudahan navigasi riwayat (Q.7)** dan **kegunaan informasi hasil (Q.9)**. Hal ini menunjukkan bahwa sistem sangat efektif dalam menyajikan data yang relevan bagi kebutuhan pengembang untuk mengidentifikasi masalah pada API. Meskipun parameter kemudahan penggunaan (Q.4) dan penyajian hasil (Q.8) memiliki skor yang sedikit lebih rendah (4,4), nilai tersebut tetap berada pada kategori sangat baik, dengan catatan perlunya penyederhanaan pada tampilan yang dirasa terlalu padat informasi oleh beberapa partisipan.

6.4.2. Sintesis Masukan dan Sentimen Kualitatif

Umpan balik terbuka dari para partisipan memberikan gambaran mendalam mengenai aspek-aspek spesifik yang menjadi nilai tambah maupun hambatan dalam penggunaan platform AutoRestTest.

1. **Aspek yang Paling Disukai:** Partisipan memberikan apresiasi tinggi pada aspek visualisasi dan kedalaman informasi. **P-2** dan **P-4** secara spesifik menyukai fitur *JSON Previewer* dan grafik (*infographics*) yang intuitif pada halaman hasil. **P-1** menekankan bahwa aplikasi ini sangat intuitif dalam menyajikan banyak informasi yang berguna bagi pengujian kualitas dan keamanan API.
2. **Aspek yang Membingungkan atau Perlu Diperbaiki:** Masalah utama yang ditemukan berkaitan dengan konfigurasi teknis untuk integrasi eksternal. **P-2** dan **P-4** mencatat bahwa instruksi untuk pengaturan *Environment Secrets* di GitHub perlu dibuat lebih jelas secara visual. **P-3** juga menyoroti kebingungan pada proses penyiapan awal fitur *CI setup*. Selain itu, **P-1** menyarankan agar tampilan pada bagian *details* disederhanakan karena saat ini dirasa terlalu padat.
3. **Saran untuk Pengembangan Selanjutnya:** Beberapa saran kunci yang diberikan oleh partisipan untuk meningkatkan kualitas sistem di masa depan meliputi:
 - **P-3:** Memberikan kemudahan lebih pada proses *setup* yang harus dilakukan oleh pengguna.
 - **P-4:** Menambahkan panduan visual berupa tangkapan layar atau video tutorial untuk langkah-langkah integrasi CI/CD.
 - **P-5:** Menambahkan *tooltips* atau deskripsi singkat pada parameter teknis (seperti *Mutation Rate* dan *Temperature*) untuk membantu pengguna yang belum familiar dengan istilah AI/ML.

Secara keseluruhan, sentimen partisipan sangat positif, dengan mayoritas menyatakan ketertarikan yang besar untuk mengadopsi sistem ini ke dalam alur kerja pengembangan mereka jika perbaikan pada aspek instruksi visual telah diimplementasikan.

6.5. Pembahasan Hasil dan Dampak Penelitian

Setelah melalui rangkaian pengujian fungsional, non-fungsional, hingga evaluasi usability, hasil penelitian ini memberikan gambaran komprehensif mengenai posisi sistem AutoRestTest berbasis web dalam ekosistem pengembangan perangkat lunak. Bagian ini membahas bagaimana temuan-temuan tersebut menjawab permasalahan riset yang telah dirumuskan.

6.5.1. Penjembatanan Teori dan Praktik Melalui Antarmuka Web

Salah satu motivasi utama penelitian ini adalah adanya hambatan adopsi teknik *fuzzing* canggih karena antarmuka baris perintah (*command-line interface*) yang kompleks. Implementasi antarmuka berbasis web ini berhasil menjembatani kesenjangan tersebut dengan menyediakan abstraksi yang intuitif. Berdasarkan hasil observasi, partisipan (P-1 hingga P-5) mampu memahami fungsi utama aplikasi dalam waktu singkat dan menyelesaikan alur pengujian tanpa perlu memahami sintaks CLI yang rumit. Keberhasilan sistem dalam melakukan 16.739 permintaan hanya dalam waktu 60 detik membuktikan bahwa efisiensi mesin riset AutoRestTest tetap terjaga meskipun dibungkus dalam antarmuka grafis yang ramah pengguna.

6.5.2. Peran Integrasi CI/CD dalam Siklus Penjaminan Kualitas

Fitur integrasi CI/CD yang dikembangkan bertujuan untuk mengubah proses *fuzzing* dari aktivitas manual menjadi bagian proaktif dalam siklus pengembangan. Hasil pengujian fungsional menunjukkan bahwa sistem dapat secara otomatis menyisipkan berkas *workflow* dan menerima pemicu pengujian dari aksi *push* di GitHub. Meskipun partisipan sempat mengalami kendala pada urutan instruksi di formulir konfigurasi, hasil akhir menunjukkan bahwa setelah

konfigurasi selesai, pengembang mendapatkan nilai tambah berupa deteksi *bug* otomatis setiap kali terjadi perubahan kode. Hal ini secara signifikan mengurangi beban kerja manual tim *Quality Assurance* (QA) dalam melakukan pengujian regresi pada REST API.

6.5.3. Kualitas Teknis dan Keberterimaan Pengguna

Kualitas sistem secara keseluruhan didukung oleh performa teknis yang solid dan tingkat keberterimaan pengguna yang tinggi.

1. **Keandalan Kode:** Tingkat *coverage* unit testing yang mencapai rata-rata di atas 98% memastikan bahwa logika bisnis aplikasi memiliki risiko kegagalan internal yang minimal.
2. **Efisiensi Akses:** Waktu respons rata-rata API sebesar 149 ms dengan tingkat kesalahan hanya 0,01% menunjukkan bahwa arsitektur asinkron yang menggunakan Trigger.dev dan FastAPI sangat handal dalam menangani beban kerja tinggi.
3. **Optimalisasi Web:** Skor sempurna pada Google Lighthouse (Performance, Best Practices, SEO) membuktikan bahwa sistem telah memenuhi standar kualitas industri untuk aplikasi web modern.
4. **Keberterimaan:** Skor akhir usability sebesar 4,86 dari 5,0 menegaskan bahwa platform ini tidak hanya fungsional secara teknis, tetapi juga memberikan pengalaman pengguna yang memuaskan dan memiliki potensi adopsi yang besar di masa depan.

Secara keseluruhan, penelitian ini membuktikan bahwa penggabungan teknologi *Multi-Agent Reinforcement Learning* dengan antarmuka web yang dioptimalkan dapat menjadi solusi efektif dalam meningkatkan standar pengujian kualitas REST API di lingkungan industri maupun akademisi.

6.6. Kesimpulan dan Saran

6.6.1. Kesimpulan

Berdasarkan hasil pengembangan, pengujian, dan analisis yang telah dilakukan pada sistem fuzzing REST API berbasis web ini, dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Penelitian ini berhasil mengembangkan platform web end-to-end yang mengintegrasikan engine AutoRestTest ke dalam antarmuka intuitif menggunakan tumpukan teknologi Next.js, FastAPI, dan orkestrasi tugas asinkron Trigger.dev.
2. Sistem menunjukkan tingkat reliabilitas tinggi dengan cakupan unit testing mencapai rata-rata 98,15% pada sisi backend Next.js dan 98% pada sisi FastAPI, serta terbukti stabil dalam pengujian performa dengan rata-rata waktu respons 149 ms dan tingkat kesalahan hanya 0,01%.
3. Implementasi sistem terbukti efektif mengeksekusi pengujian secara masif dengan kemampuan melakukan 16.739 permintaan dalam durasi 60 detik serta berhasil mengidentifikasi kerentanan pada 77,8% endpoints yang diuji.
4. Sistem memperoleh skor akhir usability sebesar 4,86 dari 5,00 melalui task-based usability testing, yang menunjukkan keberhasilan dalam mengatasi hambatan kompleksitas antarmuka baris perintah (command-line) sehingga layak diadopsi oleh praktisi pengembang maupun QA engineer.
5. Fitur integrasi dengan GitHub Actions berfungsi efektif dalam menyisipkan konfigurasi dan memicu pekerjaan pengujian secara otomatis setiap kali terdapat pembaruan kode pada repositori pengguna, menjadikan proses fuzzing lebih proaktif.

6.6.2. Saran

Penelitian ini telah menghasilkan purwarupa fungsional yang stabil, namun terdapat beberapa peluang pengembangan di masa depan untuk meningkatkan kapabilitas sistem:

1. Memperluas cakupan pengujian tidak hanya pada REST API, tetapi juga pada protokol lain seperti GraphQL, gRPC, atau WebSockets untuk mengakomodasi arsitektur perangkat lunak yang lebih bervariasi.
2. Melakukan studi komparatif dengan mengintegrasikan berbagai Large Language Models (LLM) lain, seperti Anthropic Claude, Google Gemini, atau model lokal (self-hosted) seperti Llama 3, untuk menganalisis efektivitas biaya dan akurasi deteksi kesalahan.
3. Menambahkan modul pengujian yang lebih spesifik pada aspek keamanan, khususnya pendeteksian otomatis terhadap OWASP Top 10 API Security Risks seperti SQL Injection, Broken Object Level Authorization, atau Sensitive Data Exposure.
4. Mengembangkan fitur analisis perbandingan (benchmarking) yang memungkinkan pengguna membandingkan hasil pengujian antar versi API (versioning) secara berdampingan untuk memantau peningkatan atau penurunan kualitas kode secara historis.
5. Mengimplementasikan teknologi kontainerisasi dinamis seperti Kubernetes untuk mengelola skalabilitas sumber daya komputasi secara otomatis saat sistem menangani banyak permintaan pengujian dari berbagai pengguna secara bersamaan.

DAFTAR PUSTAKA

- Adamjak, T. (2024) *Task-based usability testing + example task scenario*, *Articles on everything UX: Research, Testing & Design*. Available at: <https://blog.uxtweak.com/task-based-usability-testing/> (Accessed: 23 December 2025).
- Atlidakis, V., Godefroid, P. and Polishchuk, M. (2019) 'RESTler: Stateful REST API Fuzzing', *Proceedings - International Conference on Software Engineering*, 2019-May, pp. 748–758. Available at: <https://doi.org/10.1109/ICSE.2019.00083>.
- Chow, K.H. et al. (2022) 'DeepRest: Deep Resource Estimation for Interactive Microservices', *EuroSys 2022 - Proceedings of the 17th European Conference on Computer Systems*, pp. 181–198. Available at: <https://doi.org/10.1145/3492321.3519564>.
- Clerk (2025) *Clerk Documentation*. Available at: <https://clerk.com/docs>.
- FastAPI (2025) *FastAPI Documentation*. Available at: <https://fastapi.tiangolo.com/>.
- Godefroid, P. (2020) 'Fuzzing', *Communications of the ACM*, 63(2), pp. 70–76. doi:10.1145/3363824.
- Hatfield-Dodds, Z. and Dygalo, D. (2022) 'Deriving semantics-aware fuzzers from web API schemas', pp. 345–346. Available at: <https://doi.org/10.1145/3510454.3528637>.
- Kim, M. et al. (2024) 'A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs'. Available at: <http://arxiv.org/abs/2411.07098> (Accessed: 22 April 2025).

- Liu, Y. et al. (2022) ‘Morest: Model-based RESTful API Testing with Execution Feedback’, *Proceedings - International Conference on Software Engineering*, 2022-May, pp. 1406–1417. Available at: <https://doi.org/10.1145/3510003.3510133>.
- Miller, D. et al. (2021) *OpenAPI Specification v3.1.0*. Available at: <https://spec.openapis.org/oas/v3.1.0> (Accessed: 23 December 2025).
- Next.js (2025) *Next.js Documentation*. Available at: <https://nextjs.org/docs>.
- Postman (2023) *What Is a REST API? Examples, Uses & Challenges* | Postman Blog. Available at: <https://blog.postman.com/rest-api-examples/> (Accessed: 27 April 2025).
- Prisma (2025) *Prisma Documentation*. Available at: <https://www.prisma.io/docs>.
- Richardson, L. and Ruby, S. (2007) *RESTful web services: Web services for the real world*. Beijing: O’Reilly.
- Stennett, T. et al. (2025) ‘AutoRestTest: A Tool for Automated REST API Testing Using LLMs and MARL’. Available at: <https://arxiv.org/abs/2501.08600v2> (Accessed: 22 April 2025).
- Sutton, R.S. and Barto, A.G. (2020) *Reinforcement learning: An introduction*. Cambridge, MA: The MIT Press.
- Trigger.dev (2025) *Trigger.dev Documentation*. Available at: <https://trigger.dev/docs>.
- Viglianisi, E., Dallago, M. and Ceccato, M. (2020) ‘RESTTESTGEN: Automated Black-Box Testing of RESTful APIs’, *Proceedings - 2020 IEEE 13th International Conference on Software Testing, Verification and Validation, ICST 2020*, pp. 142–152. Available at: <https://doi.org/10.1109/ICST46399.2020.00024>.

LAMPIRAN

Lampiran 1: Skenario Usability Testing

Proses *usability testing* ini akan dilaksanakan sepenuhnya secara daring (online) menggunakan platform konferensi video (Google Meet). Peneliti dan partisipan akan berinteraksi secara sinkron. Panduan berikut merinci alur sesi daring tersebut.

1. Pembukaan & Persetujuan (Estimasi Durasi: 4-5 Menit)

Tahap ini bertujuan membangun suasana nyaman, menjelaskan konteks, dan memperoleh persetujuan formal.

- Sapa partisipan, perkenalkan diri dan afiliasi peneliti.
- Jelaskan tujuan sesi: untuk mengevaluasi prototipe aplikasi, bukan menguji kemampuan partisipan. Tekankan bahwa tidak ada jawaban yang salah.
- Dorong partisipan untuk menerapkan metode *think-aloud*.
- Informasikan logistik: durasi sekitar 40-50 menit, sesi akan direkam (layar dan suara), dan data akan dijaga kerahasiaannya.
- Menjelaskan mekanisme *informed consent* secara daring. Peneliti akan membagikan dua tautan Google Docs terpisah (dengan akses *view only*) melalui fitur chat pada platform konferensi video:
 - Tautan pertama: Lembar Penjelasan kepada Calon Subjek.
 - Tautan kedua: Informed Consent Form (ICF).
- Peneliti akan memberikan waktu kepada partisipan untuk membaca kedua dokumen tersebut. Setelah partisipan mengonfirmasi telah membaca dan memahami keduanya, peneliti akan meminta persetujuan secara lisan (verbal). Peneliti akan secara eksplisit menyatakan bahwa sesi akan direkam, dan persetujuan lisan ini akan terekam sebagai bukti persetujuan resmi. Sesi perekaman akan dimulai setelah partisipan memberikan persetujuan verbal

yang jelas (contoh: "Ya, saya telah membaca dan saya setuju untuk berpartisipasi").

2. Sesi *Task-Based Usability Testing* (Estimasi Durasi: 30-35 Menit)

a. Bagian I: Impresi Awal & Pemahaman Konteks

1. **Tujuan:** Mengukur pemahaman awal pengguna terhadap fungsi aplikasi.

2. Instruksi Inti (Wajib):

- (S.1) Minta partisipan membuka tautan aplikasi dan membagikan layar. Beri instruksi: "Silakan amati halaman utama ini sejenak. Berdasarkan apa yang Anda lihat, coba deskripsikan apa fungsi dari aplikasi ini?"

3. Pertanyaan Pancingan (Opsional):

- (SO.1) "Apa yang pertama kali menarik perhatian Anda?"
- (SO.2) "Menurut Anda, untuk siapa aplikasi ini ditujukan?"

b. Bagian II: Penggunaan Fungsi Utama (*Fuzzing Manual*)

- **Tujuan:** Mengobservasi alur kerja pengguna dalam mengonfigurasi dan menjalankan pengujian.

• Instruksi Inti (Wajib):

1. (S.2) Beri tugas pertama: "Bayangkan Anda ingin melakukan pengujian pada sebuah REST API. **Silakan mulai dengan membuat sesi pengujian baru dengan mengunggah file spesifikasi OpenAPI yang telah disediakan.**"
2. (S.3) Setelah file terunggah, lanjutkan instruksi: "**Sekarang, silakan mulai sesi pengujian dengan menekan tombol 'Create Job'.** Sambil menunggu, coba jelaskan apa yang Anda harapkan terjadi pada antarmuka setelah proses ini dimulai?"
3. (S.4) Setelah sesi selesai, beri instruksi berikutnya: "**Sesi pengujian telah selesai. Silakan buka halaman detail untuk melihat hasilnya.** Pada halaman ini, coba **temukan ringkasan**

umum (General Summary) dari hasil pengujian dan jelaskan informasi apa yang menurut Anda paling penting."

4. (S.5) Beri tugas terakhir pada bagian ini: "Selain ringkasan, sistem ini juga mendeteksi error spesifik. **Coba temukan bagian yang menampilkan daftar *server errors* (kesalahan 5xx) yang ditemukan selama pengujian.**"

- **Pertanyaan Pancingan (Opsional):**

1. (SO.3) Saat partisipan melihat form konfigurasi: "Apakah opsi-opsi seperti 'Quick Scan' atau parameter terkait RL dan LLM cukup bisa dipahami tujuannya dari deskripsi yang ada?"
2. (SO.4) Setelah menekan 'Create Job' dan melihat entri baru di riwayat: "Bagaimana pendapat Anda tentang cara aplikasi menampilkan status pekerjaan yang sedang berjalan?"
3. (SO.5) Saat melihat halaman hasil detail: "Apakah penyajian informasi di dasbor ini secara keseluruhan mudah untuk dinavigasi dan dipahami?"
4. (SO.6) Saat melihat detail *server error*: "Apakah informasi yang ditampilkan untuk setiap error (misalnya *request* dan *response*) cukup untuk membantu Anda melakukan investigasi awal?"

c. **Bagian III: Integrasi dengan Alur Kerja CI/CD**

- **Tujuan:** Menguji kemudahan pengguna dalam mengotomatisasi pengujian melalui integrasi GitHub Actions.

- **Instruksi Inti (Wajib):**

1. (S.6) Beri tugas: "Sekarang, bayangkan Anda ingin mengotomatisasi pengujian ini agar berjalan setiap kali ada perubahan kode. **Silakan** konfigurasi pengujian baru, namun kali ini pilih tipe 'Integrate to GitHub CI'."
2. (S.7) Lanjutkan instruksi: "**Lengkapi formulir konfigurasi**

CI/CD. Anda bisa memilih salah satu repositori Anda, tentukan *branch* **main**, dan gunakan pengaturan *default* lainnya. Setelah itu, **selesaikan proses penyiapan.**"

- **Pertanyaan Pancingan (Opsional):**

1. (SO.7) "Menurut Anda, seberapa jelas proses untuk mengintegrasikan pengujian ini ke dalam alur kerja GitHub Actions?"
2. (SO.8) "Setelah Anda menyelesaikan penyiapan, apa yang Anda harapkan akan terjadi selanjutnya di dalam repositori GitHub Anda?"

d. Bagian III: Penggunaan Fitur Sekunder (Riwayat Pengujian)

- **Tujuan:** Menguji kemudahan pengguna dalam menemukan dan menggunakan fitur riwayat.

- **Instruksi Inti (Wajib):**

1. (S.8) Beri tugas: "Sekarang, coba kembali ke halaman utama dan **temukan kembali hasil pengujian yang baru saja Anda jalankan** melalui daftar riwayat."

- **Pertanyaan Pancingan (Opsional):**

1. (SO.9) "Apakah proses untuk menemukan dan mengakses kembali hasil pengujian dari riwayat terasa intuitif?"
2. (SO.10) "Apakah informasi yang ditampilkan pada daftar riwayat (status, durasi, dll.) sudah sesuai dengan ekspektasi Anda?"

3. Pengisian Kuesioner dan Penutup (Estimasi Durasi: 5 Menit)

Tahap ini bertujuan untuk mengakhiri sesi secara profesional dan memberikan informasi mengenai langkah berikutnya. Adapun, poin-poin kunci yang wajib disampaikan peneliti:

- a. Konfirmasi bahwa seluruh skenario telah selesai dan mengucapkan terima kasih atas waktu dan kontribusi partisipan.

- b. Mengonfirmasi bahwa **tautan kuesioner pasca-sesi (Google Form)** akan segera dikirimkan kepada partisipan melalui saluran komunikasi pribadi (email atau pesan *chat* yang disepakati) untuk diisi segera setelah sesi berakhir.
- c. Memberi kesempatan kepada partisipan untuk mengajukan pertanyaan terakhir sebelum sesi diakhiri.

Lampiran 2: Detail Kuesioner Pasca Usability Testing

Kuesioner ini dirancang sebagai instrumen pengumpulan data komplementer untuk menangkap persepsi subjektif partisipan setelah berinteraksi langsung dengan sistem. Partisipan dapat mengakses dan mengisi kuesioner melalui *platform* Google Form dengan link <https://forms.gle/8pJBxqyNJ7Qkox1A8>. Dengan mengadopsi pendekatan *mixed-method*, instrumen ini terstruktur secara sistematis untuk menjaring data kuantitatif dan kualitatif.

Struktur kuesioner mencakup pengumpulan data demografis partisipan untuk segmentasi, dilanjutkan dengan evaluasi kuantitatif mengenai fungsionalitas keseluruhan melalui Skala Likert, serta penilaian spesifik terhadap kualitas dan relevansi rekomendasi yang merupakan fungsionalitas inti sistem. Instrumen ini diakhiri dengan pertanyaan terbuka yang bersifat opsional untuk menangkap umpan balik kualitatif dan saran yang mendalam. Berikut penjabaran desainnya:

Kode.	Pertanyaan	Tipe
Bagian I: Informasi Partisipan		
Q.1	Nama Lengkap	Teks Singkat
Q.2	Apa peran/status Anda saat ini?	Pilihan Ganda (Mahasiswa, Akademisi, Praktisi Industri, Frontend Developer, Backend Developer, dan Lainnya)
Q.3	Lama pengalaman di bidang pengembangan perangkat lunak?	Angka (dalam tahun)
Bagian II: Pengalaman & Fungsionalitas Keseluruhan		
Q.4	Secara keseluruhan, seberapa mudah aplikasi ini untuk digunakan?	Skala Likert (1-5)
Q.5	Seberapa jelaskah tujuan dan fungsi utama aplikasi saat pertama kali Anda melihat halaman utama?	Skala Likert (1-5)

Q.6	Alur kerja untuk membuat sesi pengujian baru mudah untuk diikuti.	Skala Likert (1-5)
Q.7	Saya tidak mengalami kesulitan saat mencoba menemukan fitur riwayat pengujian.	Skala Likert (1-5)
Bagian III: Kualitas Penyajian Hasil & Potensi Adopsi		
Q.8	Penyajian hasil pengujian pada halaman hasil mudah untuk dipahami.	Skala Likert (1-5)
Q.9	Informasi yang disajikan pada halaman hasil terasa berguna untuk mengidentifikasi potensi masalah pada API.	Skala Likert (1-5)
Q.10	Seberapa besar kemungkinan Anda akan menggunakan aplikasi ini dalam pekerjaan Anda di masa depan?	Skala Likert (1-5)
Bagian IV: Umpan Balik Terbuka (Opsional)		
Q.11	Aspek atau fitur apa yang paling Anda sukai dari aplikasi ini?	Paragraf
Q.12	Aspek atau fitur apa yang paling membingungkan atau perlu diperbaiki?	Paragraf
Q.13	Apakah ada saran atau masukan lain untuk pengembangan aplikasi selanjutnya?	Paragraf

Lampiran 3: Matriks Penilaian Usability Testing

Kode	Aktivitas / Pertanyaan	Metrik & Skala Penilaian	Catatan
Bagian I. Informasi Partisipan			
Q.1 - Q.3	Identitas & Demografi	Teks & Pilihan Ganda	
Bagian II. Sesi Uji Usabilitas (Data Observasional)			
S.1	Pemahaman Konteks Awal	Tingkat Pemahaman (Skala 0-2: Bingung, Cukup, Paham)	Sintesis dari jawaban S.1 dan kesan dari SO.1, SO.2
S.2	Tugas: Unggah Spesifikasi	Tingkat Pemahaman (Skala 0-2: Bingung, Cukup, Paham)	Catatan observasi alur & kesan dari SO.3
S.3	Tugas: Mulai Pengujian	Tingkat Pemahaman (Skala 0-2: Bingung, Cukup, Paham)	Catatan observasi alur & kesan dari SO.4
S.4	Tugas: Temukan General Summary	Tingkat Pemahaman (Skala 0-2: Bingung, Cukup, Paham)	Catatan observasi alur & kesan dari SO.5
S.5	Tugas: Temukan Server Errors	Tingkat Pemahaman (Skala 0-2: Bingung, Cukup, Paham)(Skala 0-2)	Catatan observasi alur & kesan dari SO.6
S.6	Tugas: Akses Riwayat	Tingkat Pemahaman (Skala 0-2: Bingung, Cukup, Paham)	Catatan observasi alur & kesan dari SO.9, SO.10
Bagian III. Kuesioner Pasca-Sesi (Data Persepsi Subjektif)			
Q.4	Kemudahan Penggunaan Keseluruhan	Skala Likert (1-5)	Validasi kuantitatif umum
Q.5	Kemudahan Alur Kerja Utama	Skala Likert (1-5)	Validasi kuantitatif untuk S.2 & S.3
Q.6	Kemudahan Menemukan Riwayat	Skala Likert (1-5)	Validasi kuantitatif untuk S.6
Q.7	Kejelasan Halaman Hasil	Skala Likert (1-5)	Validasi kuantitatif

			untuk S.4 & S.5
Q.8	Kegunaan Informasi Hasil	Skala Likert (1-5)	Validasi kuantitatif untuk tujuan aplikasi
Q.9	Potensi Adopsi	Skala Likert (1-5)	Skor untuk potensi penggunaan
Bagian IV. Sintesis Kualitatif			
Q.10 - Q.12	Umpan Balik Terbuka	Rangkuman Umpan Balik	Sintesis dari jawaban paragraf

Matriks ini dirancang untuk memberikan kerangka penilaian yang terstruktur, menggabungkan skor kuantitatif berbobot dengan analisis kualitatif untuk menghasilkan profil evaluasi yang komprehensif bagi setiap partisipan.

1. Komponen Penilaian dan Pembobotan

Melalui pendekatan kualitatif, hasil perilaku dan aksi pengguna dalam *task-based usability testing* serta pengisian kuesioner melalui Google Form dapat dinilai berdasarkan empat komponen utama yang mencerminkan prioritas penelitian seperti yang dijabarkan berikut:

a. Skor Kualitas (Bobot: 40%)

Mengukur sejauh mana aplikasi membantu pengguna mencapai tujuan utama: menjalankan pengujian dan memahami hasilnya. Komponen ini diturunkan dari sumber data **S.4, S.5, Q.7,** dan **Q.8.**

b. Skor *Usability* (Bobot: 30%)

Skor *usability* atau tingkat tepat-guna bertujuan untuk mengevaluasi efektivitas penggunaan sistem dalam menyelesaikan alur kerja utama. Komponen ini diturunkan dari sumber data **S.2, S.3,** dan **Q.5.**

c. Skor Pengalaman (Bobot: 20%)

Skor pengalaman menilai kesan pertama pengguna terhadap

aplikasi, kemudahan penggunaan aplikasi, serta kemudahan dalam menemukan fitur sekunder. Komponen ini diturunkan dari sumber data **S.1, S.6, dan Q.4, Q.6.**

d. Skor Potensi Adopsi (Bobot: 10%)

Skor potensi adopsi mengukur potensi pengguna untuk menggunakan kembali sistem terkait di masa depan. Didasarkan pada sumber data **Q.9.**

2. Mekanisme Penskalaan Skor

Untuk memperoleh nilai kualitatif akhir dari perilaku pengguna yang konsisten, data observasional akan dinormalisasi ke skala 1-5 dengan kalkulasi yang dijabarkan sebagai berikut.

- a. Data Kuesioner (dinotasikan dengan Q.x) tetap menggunakan skor asli Skala Likert (1 s.d. 5).
- b. Data *Task-Based Usability Testing* (dinotasikan dengan S.x dan SO.x) yang terkait dengan tugas dinilai dari tingkat keberhasilan (skala 0-2) dan akan dinormalisasi menjadi:
 - 0 (Gagal) menjadi Skor 1
 - 1 (Berhasil dengan kesulitan) menjadi Skor 3
 - 2 (Berhasil dengan mudah) menjadi Skor 5
- c. Data *Task-Based Usability Testing* (dinotasikan dengan S.x dan SO.x) yang terkait dengan pemahaman dan sentimen pengguna dinilai dari tingkat keberhasilan (skala 0-2) dan akan dinormalisasi menjadi:
 - 0 (Bingung) menjadi Skor 1
 - 1 (Cukup Paham) menjadi Skor 3
 - 2 (Paham) menjadi Skor 5

3. Hasil Penilaian Partisipan

Hasil penilaian dan pencatatan masukan partisipan dalam

melaksanakan *task-based usability testing* dan pengisian kuesioner melalui Google Form dapat disintesis menjadi simpulan yang terdiri dari skor kuantitatif dan rangkuman masukan serta sentimen pengguna secara kualitatif. Berikut penjabaran bentuk hasil akhir penilaian terhadap partisipan.

a. Skor Kuantitatif

Skor akhir secara kuantitatif didefinisikan sebagai akumulasi dari rerata tiap skor komponen utama yaitu Skor Kualitas, Skor *Usability*, Skor Pengalaman dan Skor Potensi Adopsi serta nilai pembobotan masing-masing komponen. Secara matematis, rumus skor akhir kuantitatif didefinisikan sebagai berikut:

$$\text{Skor Akhir} = (\text{avg}(\text{Skor Kualitas}) \times 0.40) + (\text{avg}(\text{Skor Usability}) \times 0.30) + (\text{avg}(\text{Skor Pengalaman}) \times 0.20) + (\text{Skor Potensi Adopsi} \times 0.10)$$

b. Ringkasan Masukan dan Sentimen Kualitatif

- Poin Kuat: 2-3 poin utama yang disukai partisipan (disintesis dari observasi dan Q.10).
- Area Masalah (*Pain Points*): 2-3 kendala atau kebingungan utama yang dialami (disintesis dari observasi dan Q.11).
- Saran Kunci: 1-2 saran paling berdampak dari partisipan (disintesis dari Q.12).
- Kutipan Penting: Satu kutipan *verbatim* yang paling mewakili pengalaman partisipan selama sesi.

Lampiran 4: Surat Persetujuan Komisi Etik



UNIVERSITAS GADJAH MADA

Bulaksumur, Yogyakarta 55281, Telp. +62 274 588688, +62 274 562011, Fax. +62 274 565223
http://ugm.ac.id, E-mail : setr@ugm.ac.id

PERSETUJUAN KOMISI ETIK

NO: KE/UGM/211/EC/2025

Judul Protokol Penelitian : *Usability Testing* pada Prototipe Sistem *Fuzzing* REST API dengan Model *Multi Agent Reinforcement Learning* *AutoRestTest*

Dokumen yang telah disetujui : Proposal Penelitian versi 02 2025
Penjelasan kepada Calon Subyek dan *Informed Consent Form* versi 02 2025

Peneliti utama : David Lois

Peneliti yang berpartisipasi : Arif Nurwidyantoro, S.Kom., M.Cs., Ph.D.

Tanggal disetujui : 10 November 2025 (valid selama 1 tahun sejak tanggal disetujui)

Institusi/tempat penelitian : Indonesia (daring)

Komisi Etik Penelitian Universitas Gadjah Mada menyatakan bahwa dokumen tersebut di atas telah memenuhi standar prinsip-prinsip etika penelitian.

Komisi Etik Penelitian Universitas Gadjah Mada memiliki wewenang untuk mengawasi aktivitas penelitian kapanpun.

Peneliti Utama diwajibkan untuk menyerahkan:

- ☐ Laporan kemajuan penelitian
- ☐ Laporan khusus jika ada kejadian serius
- ☒ Laporan akhir ketika penelitian telah selesai.

Ketua Panel

Dr. biol.hum. Nasuti Wijayanti, M.Si.

Wakil Ketua Panel

Dr. Devi Oktaviana Latif, S.T., M.Eng.

Lampiran 5: Sampel Laporan Hasil Usability Testing

LEMBAR OBSERVASI & LOG USABILITY TESTING

Informasi Sesi

- Subjek ID: P-1
- Tanggal: 18 Desember 2025
- Peran: [] Mahasiswa [v] Developer [] QA Engineer
- Pengalaman: 4 Tahun

BAGIAN I: IMPRESI AWAL (Context)

Kode	Instruksi & Skenario	Skor (Lingkari)	Catatan Verbal (Think-Aloud) & Perilaku
S.1	Halaman Utama	0 (Bingung)	Tebakan pertama user:
	Instruct: "Amati halaman ini. Deskripsikan apa fungsi aplikasi ini?"	1 (Ragu) 2 (Paham)	API testing yang melibatkan LLM dan MARL. AI agent yang akan mengeksplorasi API untuk mencari kesalahan tidak terduga

BAGIAN II: FUNGSI UTAMA (Fuzzing Manual)

Kode	Instruksi & Skenario	Skor (Lingkari)	Catatan Verbal (Think-Aloud) & Perilaku
S.2	Upload File	0 (Gagal)	Kendala saat upload:
	Instruct: "Buat sesi baru, unggah file OpenAPI yang disediakan."	1 (Bantuan) 2 (Lancar)	Tidak ada

S.3	Eksekusi Job	0 (Bingung)	<i>Ekspektasi vs Realita User:</i>
	Instruct: "Tekan <i>Create Job</i> . Apa yang Anda harapkan terjadi?"	1 (Ragu) 2 (Paham)	ekspektasi bahwa tidak <u>diredirect</u> langsung ke detail page karena dikira bakal bisa langsung menjalankan beberapa job secara sekaligus.
S.4	Cek Summary	0 (Bingung)	<i>Info pertama yang dilihat:</i>
	Instruct: "Sesi selesai. Temukan ringkasan umum (Summary)."	1 (Lama) 2 (Cepat)	jumlah requests yang ditembak ke API server dan ditemukan banyak error
S.5	Cek Error	0 (Gagal)	<i>Pemahaman detail error:</i>
	Instruct: "Temukan daftar Server Errors (5xx)."	1 (Sulit) 2 (Mudah)	paham bahwa kombinasi parameter dan body dapat menyebabkan server error pada API

BAGIAN III: INTEGRASI CI/CD (GitHub)

Kode	Instruksi & Skenario	Skor (Lingkari)	Catatan Verbal (Think-Aloud) & Perilaku
S.6	Config CI/CD	0 (Gagal)	<i>Kejelasan form config:</i>
	Instruct: "Buat tes baru, pilih tipe 'Integrate to GitHub CI'."	1 (Bingung) 2 (Lancar)	sudah bisa membuat CI github sendiri tetapi bingung kenapa tidak langsung terbuatkan jobnya. Mungkin bisa diperjelas lagi notice dari aplikasinya menggunakan dialog. Instruksi untuk membuat secrets di environment juga kurang jelas. Seharusnya perlu lebih banyak panduan screenshot untuk mempermudah penggunaan. Selain dari itu, error dari CI seharusnya lebih jelas karena error di CI tertutup oleh success message.

Stabilitas infrastruktur juga perlu diperhatikan karena user sempat terganggu oleh itu.

S.7	Finalisasi Setup	0 (Bingung)	<i>Ekspektasi output di GitHub:</i>
	Instruct: "Selesaikan setup. Apa ekspektasi di Repo Anda?"	1 (Ragu) 2 (Paham)	Langsung terbuatkan github actions sesuai dengan config

BAGIAN IV: RIWAYAT (History)

Kode	Instruksi & Skenario	Skor (Lingkari)	Catatan Verbal (Think-Aloud) & Perilaku
S.8	Cek History	0 (Gagal)	<i>Navigasi user:</i>
	Instruct: "Kembali ke menu utama, temukan riwayat tes tadi."	1 (Sulit) 2 (Mudah)	

CATATAN KUALITATIF (Diisi segera setelah sesi)

1. Apa hal yang paling disukai user? (Poin Kuat)

Appreciate graph yang intuitive dan mudah dipahami. Bagi developer intuitif dan comfortable.

Fitur preview JSON diapresiasi karena dapat digunakan untuk memeriksa kembali

2. Di mana user terlihat paling frustrasi/bingung? (Pain Points)

3. Kutipan/Komentar paling menarik (Verbatim):

Checklist Akhir: [v] Stop Recording [v] Kirim Link Kuesioner (Google Form) [v] Ucapkan Terima Kasih

Lampiran 6: Informasi Akses Sistem dan Repositori

Berikut adalah informasi teknis untuk mengakses platform AutoRestTest dan meninjau kode sumber hasil pengembangan:

1. Tautan Layanan (Deployment) Seluruh komponen sistem telah di-*deploy* menggunakan infrastruktur berbasis *cloud* untuk memastikan ketersediaan layanan:

- **Alamat Aplikasi (Frontend):** <https://art.dvad.my.id>
- **Alamat API Backend:** <https://art-api.user.cloudjkt01.com>

2. Repositori Kode Sumber Pengguna dapat meninjau implementasi logika program dan struktur basis data melalui tautan berikut:

- **Tautan Repositori *Frontend* dan *BFF*:**
<https://github.com/DvaDvaD/autoresttest-web>
- **Tautan Repositori *Model Wrapper API* dan *Dummy API*:**
<https://github.com/DvaDvaD/autoresttest-api>

3. Kredensial Uji Coba Jika penguji memerlukan akun dengan data riwayat yang sudah terisi untuk keperluan peninjauan, silakan gunakan kredensial berikut:

- **Email:** `test@account.com`
- **Password:** `!Tester1234`